Example

Let's start with an example. Suppose we are charged with providing automated access control to a building. Before entering the building each person has to look into a camera so we can take a still image of their face. For our purposes it suffices just to decide based on the image whether the person can enter the building. It might be helpful to (try to) also identify each person but this might require type of information we do not have (e.g., names or whether any two face images correspond to the same person). We only have face images of people recorded while access control was still provided manually. As a result of this experience we have *labeled* images. An image is labeled *positive* if the person in question should gain entry and *negative* otherwise. To supplement the set of negatively labeled images (as we would expect only few cases of refused entries under normal circumstances) we can use any other face images of people who we do not expect to be permitted to enter the building. Images taken with similar camera-face orientation (e.g., from systems operational in other buildings) would be preferred. Our task then is to come up with a function – a *classifier* – that maps pixel images to binary (± 1) labels. And we only have the small set of labeled images (the *training set*) to constrain the function.

Let's make the task a bit more formal. We assume that each image (grayscale) is represented as a column vector $\mathbf{x}$ of dimension d. So, the pixel intensity values in the image, column by column, are concatenated into a single column vector. If the image has 100 by 100 pixels, then $d = 10000$. We assume that all the images are of the same size. Our classifier is a binary valued function $f : \mathcal{R}^d \to \{-1, 1\}$ chosen on the basis of the training set alone. For our task here we assume that the classifier knows nothing about images (or faces for that matter) beyond the labeled training set. So, for example, from the point of view of the classifier, the images could have been measurements of weight, height, etc. rather than pixel intensities. The classifier only has a set of n training vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n$ with binary ± 1 labels $y_1, \ldots, y_n$. This is the only information about the task that we can use to constraint what the function f should be.

What kind of solution would suffice?

Suppose now that we have $n = 50$ labeled pixel images that are 128 by 128, and the pixel intensities range from 0 to 255. It is therefore possible that we can find a single pixel, say pixel i, such that each of our n images have a distinct value for that pixel. We could then construct a simple binary function based on this single pixel that perfectly maps the training images to their labels. In other words, if x_{ti} refers to pixel i in the t^{th} training

image, and x'_i is the i^{th} pixel in any image $\mathbf{x}'$, then

$$f_i(\mathbf{x}') = \begin{cases} y_t, \text{ if } x_{ti} = x'_i \text{ for some } t = 1, \ldots, n \text{ (in this order)} \\ -1, \text{ otherwise} \end{cases} \quad (1)$$

would appear to solve the task. In fact, it is always possible to come up with such a "perfect" binary function if the training images are distinct (no two images have identical pixel intensities for all pixels). But do we expect such rules to be useful for images not in the training set? Even an image of the same person varies somewhat each time the image is taken (orientation is slightly different, lighting conditions may have changed, etc). These rules provide no sensible predictions for images that are not identical to those in the training set. The primary reason for why such trivial rules do not suffice is that our task is *not* to correctly classify the training images. Our task is to find a rule that works well for all new images we would encounter in the access control setting; the training set is merely a helpful source of information to find such a function. To put it a bit more formally, we would like to find classifiers that *generalize* well, i.e., classifiers whose performance on the training set is representative of how well it works for yet unseen images.

Model selection

So how can we find classifiers that generalize well? The key is to constrain the set of possible binary functions we can entertain. In other words, we would like to find a class of binary functions such that if a function in this class works well on the training set, it is also likely to work well on the unseen images. The "right" class of functions to consider cannot be too large in the sense of containing too many clearly different functions. Otherwise we are likely to find rules similar to the trivial ones that are close to perfect on the training set but do not generalize well. The class of function should not be too small either or we run the risk of not finding any functions in the class that work well even on the training set. If they don't work well on the training set, how could we expect them to work well on the new images? Finding the class of functions is a key problem in machine learning, also known as the *model selection* problem.

Linear classifiers through origin

Let's just fix the function class for now. Specifically, we will consider only a type of *linear classifiers*. These are thresholded linear mappings from images to labels. More formally, we only consider functions of the form

$$f(\mathbf{x}; \theta) = \text{sign}\big(\theta_1 x_1 + \ldots + \theta_d x_d\big) = \text{sign}\big(\theta^T \mathbf{x}\big) \quad (2)$$

where $\theta = [\theta_1, \ldots, \theta_d]^T$ is a column vector of real valued parameters. Different settings of the parameters give different functions in this class, i.e., functions whose value or *output* in $\{-1, 1\}$ could be different for some *input* images $\mathbf{x}$. Put another way, the functions in our class are parameterized by $\theta \in \mathcal{R}^d$.

We can also understand these linear classifiers geometrically. The classifier changes its prediction only when the argument to the sign function changes from positive to negative (or vice versa). Geometrically, in the space of image vectors, this transition corresponds to crossing the *decision boundary* where the argument is exactly zero: all $\mathbf{x}$ such that $\theta^T\mathbf{x} = 0$. The equation defines a plane in d-dimensions, a plane that goes through the origin since $\mathbf{x} = 0$ satisfies the equation. The parameter vector θ is normal (orthogonal) to this plane; this is clear since the plane is defined as all $\mathbf{x}$ for which $\theta^T\mathbf{x} = 0$. The θ vector as the normal to the plane also specifies the direction in the image space along which the value of $\theta^T\mathbf{x}$ would increase the most. Figure 1 below tries to illustrate these concepts.

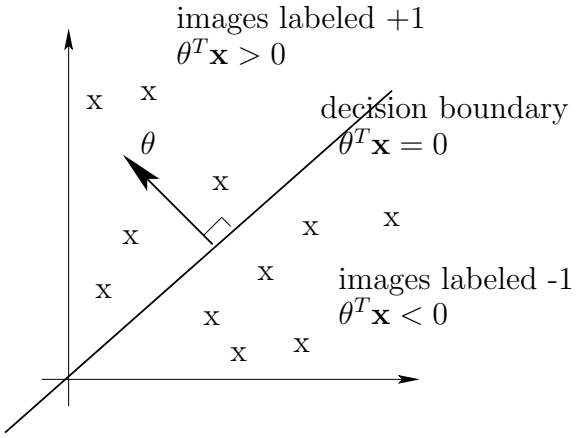


Figure 1: A linear classifier through origin.

Before moving on let's figure out whether we lost some useful properties of images as a result of restricting ourselves to linear classifiers? In fact, we did. Consider, for example, how nearby pixels in face images relate to each other (e.g., continuity of skin). This information is completely lost. The linear classifier is perfectly happy (i.e., its ability to classify images remains unchanged) if we get images where the pixel positions have been reordered provided that we apply the same transformation to all the images. This permutation of pixels merely reorders the terms in the argument to the sign function in Eq. (2). A linear classifier therefore does not have access to information about which pixels are close to each other in the image.

Learning algorithm: the perceptron

Now that we have chosen a function class (perhaps suboptimally) we still have to find a specific function in this class that works well on the training set. This is often referred to as the *estimation* problem. Let's be a bit more precise. We'd like to find a linear classifier that makes the fewest mistakes on the training set. In other words, we'd like to find θ that minimizes the *training error*

$$\hat{E}(\theta) = \frac{1}{n}\sum_{t=1}^{n}\left(1 - \delta(y_t, f(\mathbf{x}_t;\theta))\right) = \frac{1}{n}\sum_{t=1}^{n}\text{Loss}(y_t, f(\mathbf{x}_t;\theta)) \tag{3}$$

where $\delta(y, y') = 1$ if $y = y'$ and 0 otherwise. The training error merely counts the average number of training images where the function predicts a label different from the label provided for that image. More generally, we could compare our predictions to labels in terms of a loss function $\text{Loss}(y_t, f(\mathbf{x}_t;\theta))$. This is useful if errors of a particular kind are more costly than others (e.g., letting a person enter the building when they shouldn't). For simplicity, we use the *zero-one loss* that is 1 for mistakes and 0 otherwise.

What would be a reasonable algorithm for setting the parameters θ? Perhaps we can just incrementally adjust the parameters so as to correct any mistakes that the corresponding classifier makes. Such an algorithm would seem to reduce the training error that counts the mistakes. Perhaps the simplest algorithm of this type is the *perceptron* update rule. We consider each training image one by one, cycling through all the images, and adjust the parameters according to

$$\theta' \leftarrow \theta + y_t\mathbf{x}_t \;\text{ if }\; y_t \neq f(\mathbf{x}_t;\theta) \tag{4}$$

In other words, the parameters (classifier) is changed only if we make a mistake. These updates tend to correct mistakes. To see this, note that when we make a mistake the sign of $\theta^T\mathbf{x}_t$ disagrees with y_t and the product $y_t\theta^T\mathbf{x}_t$ is negative; the product is positive for correctly classified images. Suppose we make a mistake on $\mathbf{x}_t$. Then the updated parameters are given by $\theta' = \theta + y^t\mathbf{x}_t$, written here in a vector form. If we consider classifying the same image $\mathbf{x}_t$ after the update, then

$$y_t\theta'^T\mathbf{x}^t = y_t(\theta + y_t\mathbf{x}_t)^T\mathbf{x}_t = y_t\theta^T\mathbf{x}_t + y_t^2\mathbf{x}_t^T\mathbf{x}_t = y_t\theta^T\mathbf{x}_t + \|\mathbf{x}_t\|^2 \tag{5}$$

In other words, the value of $y_t\theta^T\mathbf{x}_t$ increases as a result of the update (becomes more positive). If we consider the same image repeatedly, then we will necessarily change the parameters such that the image is classified correctly, i.e., the value of $y_t\theta^T\mathbf{x}_t$ becomes positive. Mistakes on other images may steer the parameters in different directions so it may not be clear that the algorithm converges to something useful if we repeatedly cycle through the training images.

Analysis of the perceptron algorithm

The perceptron algorithm ceases to update the parameters only when all the training images are classified correctly (no mistakes, no updates). So, if the training images are possible to classify correctly with a linear classifier, will the perceptron algorithm find such a classifier? Yes, it does, and it will converge to such a classifier in a finite number of updates (mistakes). We'll show this in lecture 2.

Perceptron, convergence, and generalization

Recall that we are dealing with linear classifiers through origin, i.e.,

$$f(\mathbf{x};\theta) = \text{sign}\big(\,\theta^T\mathbf{x}\,\big) \tag{1}$$

where $\theta \in \mathcal{R}^d$ specifies the parameters that we have to estimate on the basis of training examples (images) $\mathbf{x}_1, \ldots, \mathbf{x}_n$ and labels $y_1, \ldots, y_n$.

We will use the perceptron algorithm to solve the estimation task. Let k denote the number of parameter updates we have performed and $\theta^{(k)}$ the parameter vector after k updates. Initially $k = 0$ and $\theta^{(k)} = 0$. The algorithm then cycles through all the training instances $(\mathbf{x}_t, y_t)$ and updates the parameters only in response to mistakes, i.e., when the label is predicted incorrectly. More precisely, we set $\theta^{(k+1)} = \theta^{(k)} + y_t\mathbf{x}_t$ when $y_t(\theta^{(k)})^T\mathbf{x}_t < 0$ (mistake), and otherwise leave the parameters unchanged.

Convergence in a finite number of updates

Let's now show that the perceptron algorithm indeed convergences in a finite number of updates. The same analysis will also help us understand how the linear classifier generalizes to unseen images. To this end, we will assume that all the (training) images have bounded Euclidean norms, i.e., $\|\mathbf{x}_t\| \leq R$ for all t and some finite R. This is clearly the case for any pixel images with bounded intensity values. We also make a much stronger assumption that there exists a linear classifier in our class with finite parameter values that correctly classifies all the (training) images. More precisely, we assume that there is some $\gamma > 0$ such that $y_t(\theta^*)^T\mathbf{x}_t \geq \gamma$ for all $t = 1, \ldots, n$. The additional number $\gamma > 0$ is used to ensure that each example is classified correctly with a *finite margin*.

The convergence proof is based on combining two results: 1) we will show that the inner product $(\theta^*)^T\theta^{(k)}$ increases at least linearly with each update, and 2) the squared norm $\|\theta^{(k)}\|^2$ increases at most linearly in the number of updates k. By combining the two we can show that the cosine of the angle between $\theta^{(k)}$ and θ^* has to increase by a finite increment due to each update. Since cosine is bounded by one, it follows that we can only make a finite number of updates.

Part 1: we simply take the inner product $(\theta^*)^T\theta^{(k)}$ before and after each update. When making the k^{th} update, say due to a mistake on image $\mathbf{x}_t$, we get

$$(\theta^*)^T\theta^{(k)} = (\theta^*)^T\theta^{(k-1)} + y_t(\theta^*)^T\mathbf{x}_t \geq (\theta^*)^T\theta^{(k-1)} + \gamma \tag{2}$$

since, by assumption, $y_t(\theta^*)^T\mathbf{x}_t \geq \gamma$ for all t (θ^* is always correct). Thus, after k updates,

$$(\theta^*)^T\theta^{(k)} \geq k\gamma \tag{3}$$

Part 2: Our second claim follows simply from the fact that updates are made only on mistakes:

$$\begin{aligned}
\|\theta^{(k)}\|^2 &= \|\theta^{(k-1)} + y_t\mathbf{x}_t\|^2 && (4)\\
&= \|\theta^{(k-1)}\|^2 + 2y_t(\theta^{(k-1)})^T\mathbf{x}_t + \|\mathbf{x}_t\|^2 && (5)\\
&\leq \|\theta^{(k-1)}\|^2 + \|\mathbf{x}_t\|^2 && (6)\\
&\leq \|\theta^{(k-1)}\|^2 + R^2 && (7)
\end{aligned}$$

since $y_t(\theta^{(k-1)})^T\mathbf{x}^t < 0$ whenever an update is made and, by assumption, $\|\mathbf{x}_t\| \leq R$. Thus,

$$\|\theta^{(k)}\|^2 \leq kR^2 \tag{8}$$

We can now combine parts 1) and 2) to bound the cosine of the angle between θ^* and $\theta^{(k)}$:

$$cos(\theta^*, \theta^{(k)}) = \frac{(\theta^*)^T\theta^{(k)}}{\|\theta^{(k)}\|\,\|\theta^*\|} \overset{1)}{\geq} \frac{k\gamma}{\|\theta^{(k)}\|\,\|\theta^*\|} \overset{2)}{\geq} \frac{k\gamma}{\sqrt{kR^2}\,\|\theta^*\|} \tag{9}$$

Since cosine is bounded by one, we get

$$1 \geq \frac{k\gamma}{\sqrt{kR^2}\,\|\theta^*\|} \quad \text{or } k \leq \frac{R^2\|\theta^*\|^2}{\gamma^2} \tag{10}$$

Margin and geometry

It is worthwhile to understand this result a bit further. For example, does $\|\theta^*\|^2/\gamma^2$ relate to how difficult the classification problem is? Indeed, it does. We claim that its inverse, i.e., $\gamma/\|\theta^*\|$ is the smallest distance in the image space from any example (image) to the decision boundary specified by θ^*. In other words, it serves as a measure of how well the two classes of images are separated (by a linear boundary). We will call this the geometric margin or γ_{geom} (see figure 1). γ_{geom}^{-1} is then a fair measure of how difficult the problem is: the smaller the geometric margin that separates the training images, the more difficult the problem.

To calculate γ_{geom} we measure the distance from the decision boundary ${\theta^*}^T\mathbf{x} = 0$ to one of the images $\mathbf{x}_t$ for which $y_t{\theta^*}^T\mathbf{x}_t = \gamma$. Since θ^* specifies the normal to the decision boundary,

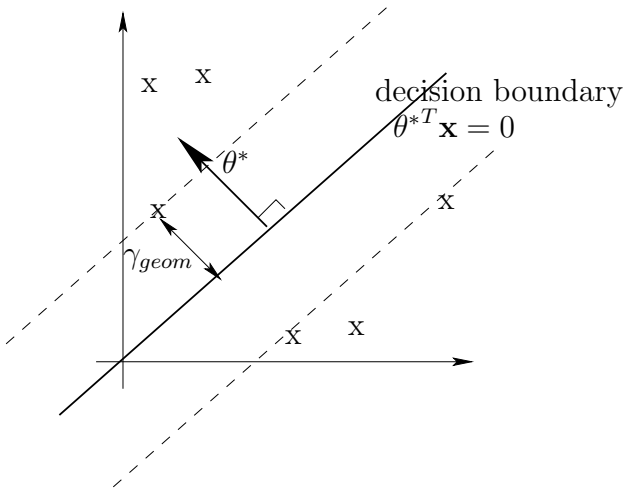


Figure 1: Geometric margin

the shortest path from the boundary to the image $\mathbf{x}_t$ will be parallel to the normal. The image for which $y_t \theta^{*T} \mathbf{x}_t = \gamma$ is therefore among those closest to the boundary. Now, let's define a line segment from $\mathbf{x}(0) = \mathbf{x}_t$, parallel to θ^*, towards the boundary. This is given by

$$\mathbf{x}(\xi) = \mathbf{x}(0) - \xi \frac{y_t \theta^*}{\|\theta^*\|} \tag{11}$$

where ξ defines the length of the line segment since it multiplies a unit length vector. It remains to find the value of ξ such that $\theta^{*T}\mathbf{x}(\xi) = 0$, or, equivalently, $y_t \theta^{*T}\mathbf{x}(\xi) = 0$. This is the point where the segment hits the decision boundary. Thus

$$\begin{aligned} y_t \theta^{*T}\mathbf{x}(\xi) &= y_t \theta^{*T}\left[\mathbf{x}(0) - \xi \frac{y_t \theta^*}{\|\theta^*\|}\right] && (12)\\ &= y_t \theta^{*T}\left[\mathbf{x}_t - \xi \frac{y_t \theta^*}{\|\theta^*\|}\right] && (13)\\ &= y_t \theta^{*T}\mathbf{x}_t - \xi \frac{\|\theta^*\|^2}{\|\theta^*\|} && (14)\\ &= \gamma - \xi\|\theta^*\| = 0 && (15) \end{aligned}$$

implying that the distance is exactly $\xi = \gamma/\|\theta^*\|$ as claimed. As a result, the bound on the number of perceptron updates can be written more succinctly in terms of the geometric

margin γ_{geom} (distance to the boundary):

$$k \leq \left(\frac{R}{\gamma_{geom}} \right)^2 \tag{16}$$

with the understanding that γ_{geom} is the largest geometric margin that could be achieved by a linear classifier for this problem. Note that the result does not depend (directly) on the dimension d of the examples, nor the number of training examples n. It is nevertheless tempting to interpret $\left(\frac{R}{\gamma_{geom}} \right)^2$ as a measure of difficulty (or complexity) of the problem of learning linear classifiers in this setting. You will see later in the course that this is exactly the case, cast in terms of a measure known as *VC-dimension*.

Generalization guarantees

We have so far discussed the perceptron algorithm only in relation to the training set but we are more interested in how well the perceptron classifies images we have not yet seen, i.e., how well it generalizes to new images. Our simple analysis above actually provides some information about generalization. Let's assume then that all the images and labels we could possibly encounter satisfy the same two assumptions. In other words, 1) $\|\mathbf{x}_t\| \leq R$ and 2) $y_t \theta^{*T} \mathbf{x}_t \geq \gamma$ for all t and some finite θ^*. So, in essence, we assume that there is a linear classifier that works for all images and labels in this problem, we just don't know what this linear classifier is to start with. Let's now imagine getting the images and labels one by one and performing only a single update per image, if misclassified, and move on. The previous situation concerning the training set corresponds to encountering the same set of images repeatedly. How many mistakes are we now going to make in this infinite arbitrary sequence of images and labels, subject only to the two assumptions? The same number $k \leq (R/\gamma_{geom})^2$. Once we have made this many mistakes we would classify all the new images correctly. So, provided that the two assumptions hold, especially the second one, we obtain a nice guarantee of generalization. One caveat here is that the perceptron algorithm does need to know when it has made a mistake. The bound is after all cast in terms of the number of updates based on mistakes.

Maximum margin classifier?

We have so far used a simple on-line algorithm, the perceptron algorithm, to estimate a linear classifier. Our reference assumption has been, however, that there exists a linear classifier that has a large geometric margin, i.e., whose decision boundary is well separated

from all the training images (examples). Can't we find such a large margin classifier directly? Yes, we can. The classifier is known as the Support Vector Machine or SVM for short. See the next lecture for details.

The Support Vector Machine

So far we have used a reference assumption that there exists a linear classifier that has a large geometric margin, i.e., whose decision boundary is well separated from all the training images (examples). Such a large margin classifier seems like one we would like to use. Can't we find it more directly? Yes, we can. The classifier is known as the Support Vector Machine or SVM for short.

You could imagine finding the maximum margin linear classifier by first identifying any classifier that correctly classifies all the examples (Figure 2a) and then increasing the geometric margin until the classifier "locks in place" at the point where we cannot increase the margin any further (Figure 2b). The solution is unique.

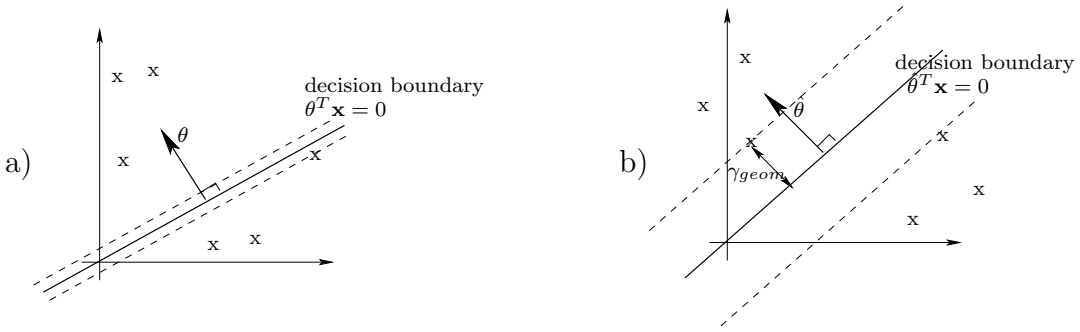


Figure 1: a) A linear classifier with a small geometric margin, b) maximum margin linear classifier.

More formally, we can set up an optimization problem for directly maximizing the geometric margin. We will need the classifier to be correct on all the training examples or $y_t \theta^T \mathbf{x}_t \geq \gamma$ for all $t = 1, \ldots, n$. Subject to these constraints, we would like to maximize $\gamma / \|\theta\|$, i.e., the geometric margin. We can alternatively minimize the inverse $\|\theta\|/\gamma$ or the inverse squared $\frac{1}{2}(\|\theta\|/\gamma)^2$ subject to the same constraints (the factor 1/2 is included merely for later convenience). We then have the following optimization problem for finding $\hat{\theta}$:

$$\text{minimize } \frac{1}{2}\|\theta\|^2/\gamma^2 \text{ subject to } y_t \theta^T \mathbf{x}_t \geq \gamma \text{ for all } t = 1, \ldots, n \tag{1}$$

We can simplify this a bit further by getting rid of γ. Let's first rewrite the optimization problem in a way that highlights how the solution depends (or doesn't depend) on γ:

$$\text{minimize } \frac{1}{2}\|\theta/\gamma\|^2 \text{ subject to } y_t (\theta/\gamma)^T \mathbf{x}_t \geq 1 \text{ for all } t = 1, \ldots, n \tag{2}$$

In other words, our classification problem (the data, setup) only tells us something about the ratio θ/γ, not θ or γ separately. For example, the geometric margin is defined only on the basis of this ratio. Scaling θ by a constant also doesn't change the decision boundary. We can therefore fix $\gamma = 1$ and solve for θ from

$$\text{minimize } \frac{1}{2}\|\theta\|^2 \text{ subject to } y_t\theta^T\mathbf{x}_t \geq 1 \text{ for all } t = 1, \ldots, n \tag{3}$$

This optimization problem is in the standard SVM form and is a *quadratic programming* problem (objective is quadratic in the parameters with linear constraints). The resulting geometric margin is $1/\|\hat{\theta}\|$ where $\hat{\theta}$ is the unique solution to the above problem. The decision boundary (separating hyper-plane) nor the value of the geometric margin were affected by our choice $\gamma = 1$.

General formulation, offset parameter

We will modify the linear classifier here slightly by adding an offset term so that the decision boundary does not have to go through the origin. In other words, the classifier that we consider has the form

$$f(\mathbf{x}; \theta, \theta_0) = \text{sign}(\, \theta^T\mathbf{x} + \theta_0 \,) \tag{4}$$

with parameters θ (normal to the separating hyper-plane) and the offset parameter θ_0, a real number. As before, the equation for the separating hyper-plane is obtained by setting the argument to the sign function to zero or $\theta^T\mathbf{x} + \theta_0 = 0$. This is a general equation for a hyper-plane (line in 2-dim). The additional offset parameter can lead to a classifier with a larger geometric margin. This is illustrated in Figures 2a and 2b. Note that the vector $\hat{\theta}$ corresponding to the maximum margin solution is different in the two figures.

The offset parameter changes the optimization problem only slightly:

$$\text{minimize } \frac{1}{2}\|\theta\|^2 \text{ subject to } y_t(\theta^T\mathbf{x}_t + \theta_0) \geq 1 \text{ for all } t = 1, \ldots, n \tag{5}$$

Note that the offset parameter only appears in the constraints. This is different from simply modifying the linear classifier through origin by feeding it with examples that have an additional constant component, i.e., $\mathbf{x}' = [\mathbf{x}; 1]$. In the above formulation we do not bias in any way where the separating hyper-plane should appear, only that it should maximize the geometric margin.

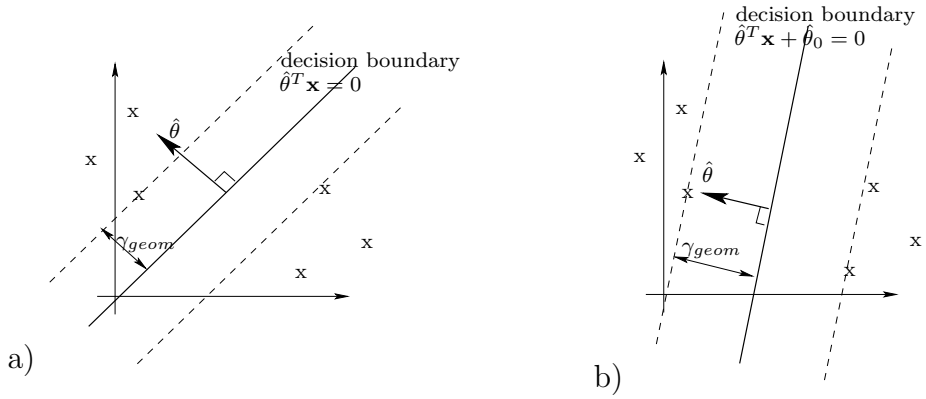


Figure 2: a) Maximum margin linear classifier through origin; b) Maximum margin linear classifier with an offset parameter

Properties of the maximum margin linear classifier

The maximum margin classifier has several very nice properties, and some not so advantageous features.

Benefits. On the positive side, we have already motivated these classifiers based on the perceptron algorithm as the "best reference classifiers". The solution is also unique based on any linearly separable training set. Moreover, drawing the separating boundary as far from the training examples as possible makes it robust to noisy examples (though not noisy labels). The maximum margin linear boundary also has the curious property that the solution depends only on a subset of the examples, those that appear exactly on the margin (the dashed lines parallel to the boundary in the figures). The examples that lie exactly on the margin are called *support vectors* (see Figure 3). The rest of the examples could lie anywhere outside the margin without affecting the solution. We would therefore get the same classifier if we had only received the support vectors as training examples. Is this is a good thing? To answer this question we need a bit more formal (and fair) way of measuring how good a classifier is.

One possible "fair" performance measure evaluated only on the basis of the training examples is *cross-validation*. This is simply a method of retraining the classifier with subsets of training examples and testing it on the remaining held-out (and therefore fair) examples, pretending we had not seen them before. A particular version of this type of procedure is called *leave-one-out cross-validation*. As the name suggests, the procedure is defined as follows: select each training example in turn as the single example to be held-out, train the

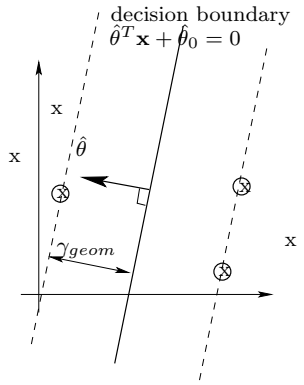


Figure 3: Support vectors (circled) associated the maximum margin linear classifier

classifier on the basis of all the remaining training examples, test the resulting classifier on the held-out example, and count the errors. More precisely, let the superscript '$-i$' denote the parameters we would obtain by finding the maximum margin linear separator without the i^{th} training example. Then

$$\text{leave-one-out CV error} = \frac{1}{n}\sum_{i=1}^{n} \text{Loss}\Big(y_i, f(\mathbf{x}_i; \theta^{-i}, \theta_0^{-i})\Big) \tag{6}$$

where $\text{Loss}(y, y')$ is the zero-one loss. We are effectively trying to gauge how well the classifier would generalize to each training example if it had not been part of the training set. A classifier that has a low leave-one-out cross-validation error is likely to generalize well though it is not guaranteed to do so.

Now, what is the leave-one-out CV error of the maximum margin linear classifier? Well, examples that lie outside the margin would be classified correctly regardless of whether they are part of the training set. Not so for support vectors. They are key to defining the linear separator and thus, if removed from the training set, may be misclassified as a result. We can therefore derive a simple upper bound on the leave-one-out CV error:

$$\text{leave-one-out CV error} \leq \frac{\#\text{ of support vectors}}{n} \tag{7}$$

A small number of support vectors – a sparse solution – is therefore advantageous. This is another argument in favor of the maximum margin linear separator.

Problems. There are problems as well, however. Even a single training example, if mislabeled, can radically change the maximum margin linear classifier. Consider, for example,

what would happen if we switched the label of the top right support vector in Figure 3.

Allowing misclassified examples, relaxation

Labeling errors are common in many practical problems and we should try to mitigate their effect. We typically do not know whether examples are difficult to classify because of labeling errors or because they simply are not linearly separable (there isn't a linear classifier that can classify them correctly). In either case we have to articulate a trade-off between misclassifying a training example and the potential benefit for other examples.

Perhaps the simplest way to permit errors in the maximum margin linear classifier is to introduce "slack" variables for the classification/margin constraints in the optimization problem. In other words, we measure the degree to which each margin constraint is violated and associate a cost for the violation. The costs of violating constraints are minimized together with the norm of the parameter vector. This gives rise to a simple relaxed optimization problem:

$$\text{minimize } \frac{1}{2}\|\theta\|^2 + C\sum_{t=1}^{n}\xi_t \tag{8}$$

$$\text{subject to } \; y_t(\theta^T\mathbf{x}_t + \theta_0) \geq 1 - \xi_t \;\text{ and } \xi_t \geq 0 \;\text{ for all } t = 1, \ldots, n \tag{9}$$

where ξ_t are the slack variables. The margin constraint is violated when we have to set $\xi_t > 0$ for some example. The penalty for this violation is $C\xi_t$ and it is traded-off with the possible gain in minimizing the squared norm of the parameter vector or $\|\theta\|^2$. If we increase the penalty C for margin violations then at some point all $\xi_t = 0$ and we get back the maximum margin linear separator (if possible). On the other hand, for small C many margin constraints can be violated. Note that the relaxed optimization problem specifies a particular *quantitative* trade-off between the norm of the parameter vector and margin violations. It is reasonable to ask whether this is indeed the trade-off we want.

Let's try to understand the setup a little further. For example, what is the resulting margin when some of the constraints are violated? We can still take $1/\|\hat{\theta}\|$ as the geometric margin. This is indeed the geometric margin based on examples for which $\xi_t^* = 0$ where '*' indicates the optimized value. So, is it the case that we get the maximum margin linear classifier for the subset of examples for which $\xi_t^* = 0$? No, we don't. The examples that violate the margin constraints, including those training examples that are actually misclassified (larger violation), do affect the solution. In other words, the parameter vector $\hat{\theta}$ is defined on the basis of examples right at the margin, those that violate the constraint but not enough to

be misclassified, and those that are misclassified. All of these are support vectors in this sense.

The Support Vector Machine and regularization

We proposed a simple relaxed optimization problem for finding the maximum margin separator when some of the examples may be misclassified:

$$\text{minimize } \frac{1}{2}\|\theta\|^2 + C\sum_{t=1}^{n}\xi_t \tag{1}$$

$$\text{subject to } \; y_t(\theta^T\mathbf{x}_t + \theta_0) \geq 1 - \xi_t \;\text{ and } \xi_t \geq 0 \;\text{ for all } t = 1, \ldots, n \tag{2}$$

where the remaining parameter C could be set by cross-validation, i.e., by minimizing the leave-one-out cross-validation error.

The goal here is to briefly understand the relaxed optimization problem from the point of view of *regularization*. Regularization problems are typically formulated as optimization problems involving the desired objective (classification loss in our case) and a regularization penalty. The regularization penalty is used to help stabilize the minimization of the objective or infuse prior knowledge we might have about desirable solutions. Many machine learning methods can be viewed as regularization methods in this manner. For later utility we will cast SVM optimization problem as a regularization problem.

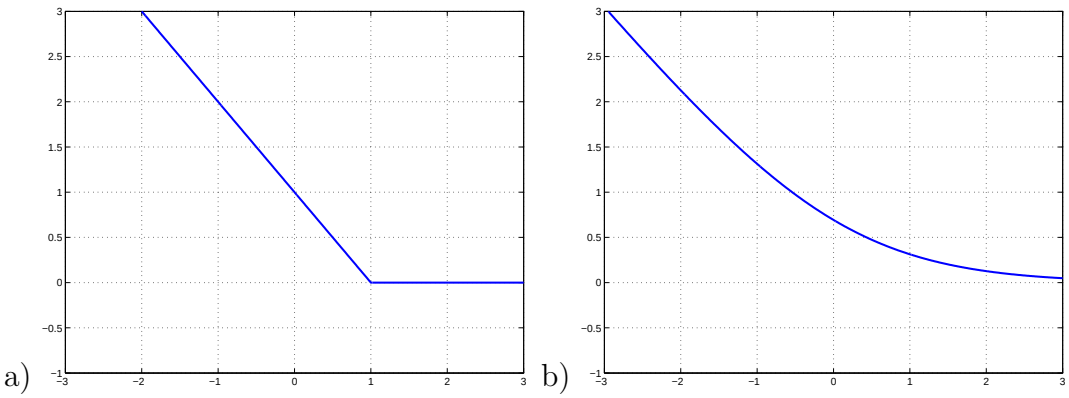


Figure 1: a) The hinge loss $(1-z)^+$ as a function of z. b) The logistic loss $\log[1+\exp(-z)]$ as a function of z.

To turn the relaxed optimization problem into a regularization problem we define a loss function that corresponds to individually optimized ξ_t values and specifies the cost of violating each of the margin constraints. We are effectively solving the optimization problem with respect to the ξ values for a fixed θ and θ_0. This will lead to an expression of $C\sum_t \xi_t$ as a function of θ and θ_0.

The loss function we need for this purpose is based on the *hinge loss* $\text{Loss}_h(z)$ defined as the positive part of $1 - z$, written as $(1 - z)^+$ (see Figure 1a). The relaxed optimization problem can be written using the hinge loss as

$$\text{minimize } \frac{1}{2}\|\theta\|^2 + C \sum_{t=1}^{n} \overbrace{\left(1 - y_t(\theta^T \mathbf{x}_t + \theta_0)\right)^+}^{=\hat{\xi}_t} \tag{3}$$

Here $\|\theta\|^2/2$, the inverse squared geometric margin, is viewed as a regularization penalty that helps stabilize the objective

$$C \sum_{t=1}^{n} \left(1 - y_t(\theta^T \mathbf{x}_t + \theta_0)\right)^+ \tag{4}$$

In other words, when no margin constraints are violated (zero loss), the regularization penalty helps us select the solution with the largest geometric margin.

Logistic regression, maximum likelihood estimation

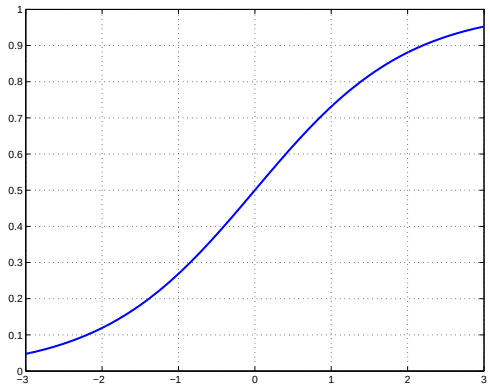


Figure 2: The logistic function $g(z) = (1 + \exp(-z))^{-1}$.

Another way of dealing with noisy labels in linear classification is to model how the noisy labels are generated. For example, human assigned labels tend to be very good for "typical examples" but exhibit some variation in more difficult cases. One simple model of noisy labels in linear classification is a logistic regression model. In this model we assign a

probability distribution over the two labels in such a way that the labels for examples further away from the decision boundary are more likely to be correct. More precisely, we say that

$$P(y = 1|\mathbf{x}, \theta, \theta_0) = g\left(\theta^T\mathbf{x} + \theta_0\right) \tag{5}$$

where $g(z) = (1 + \exp(-z))^{-1}$ is known as the logistic function (Figure 2). One way to derive the form of the logistic function is to say that the *log-odds* of the predicted class probabilities should be a linear function of the inputs:

$$\log \frac{P(y = 1|\mathbf{x}, \theta, \theta_0)}{P(y = -1|\mathbf{x}, \theta, \theta_0)} = \theta^T\mathbf{x} + \theta_0 \tag{6}$$

So for example, when we predict the same probability (1/2) for both classes, the log-odds term is zero and we recover the decision boundary $\theta^T\mathbf{x} + \theta_0 = 0$. The precise functional form of the logistic function, or, equivalently, the fact that we chose to model log-odds with the linear prediction, may seem a little arbitrary (but perhaps not more so than the hinge loss used with the SVM classifier). We will derive the form of the logistic function later on in the course based on certain assumptions about *class-conditional* distributions $P(\mathbf{x}|y = 1)$ and $P(\mathbf{x}|y = -1)$.

In order to better compare the logistic regression model with the SVM we will write the conditional probability $P(y|\mathbf{x}, \theta, \theta_0)$ a bit more succinctly. Specifically, since $1 - g(z) = g(-z)$ we get

$$P(y = -1|\mathbf{x}, \theta, \theta_0) = 1 - P(y = 1|\mathbf{x}, \theta, \theta_0) = 1 - g(\,\theta^T\mathbf{x} + \theta_0\,) = g\left(-(\theta^T\mathbf{x} + \theta_0)\right) \tag{7}$$

and therefore

$$P(y|\mathbf{x}, \theta, \theta_0) = g\left(\,y(\theta^T\mathbf{x} + \theta_0)\,\right) \tag{8}$$

So now we have a linear classifier that makes probabilistic predictions about the labels. How should we train such models? A sensible criterion would seem to be to maximize the probability that we predict the correct label in response to each example. Assuming each example is labeled independently from others, this probability of assigning correct labels to examples is given by the product

$$L(\theta, \theta_0) = \prod_{t=1}^{n} P(y_t|\mathbf{x}_t, \theta, \theta_0) \tag{9}$$

$L(\theta, \theta_0)$ is known as the (conditional) likelihood function and is interpreted as a function of the parameters for a fixed data (labels and examples). By maximizing this conditional likelihood with respect to θ and θ_0 we obtain *maximum likelihood estimates* of the parameters. Maximum likelihood *estimators*[1] have many nice properties. For example, assuming we have selected the right model class (logistic regression model) and certain regularity conditions hold, then the ML estimator is a) *consistent* (we will get the right parameter values in the limit of a large number of training examples), and b) *efficient* (no other estimator will converge to the correct parameter values faster in the mean squared sense). But what if we do not have the right model class? Neither property may hold as a result. More robust estimators can be found in a larger class of estimators called *M-estimators* that includes maximum likelihood. We will nevertheless use the maximum likelihood principle to set the parameter values.

The product form of the conditional likelihood function is a bit difficult to work with directly so we will maximize its logarithm instead:

$$l(\theta, \theta_0) = \sum_{t=1}^{n} \log P(y_t | \mathbf{x}_t, \theta, \theta_0) \tag{10}$$

Alternatively, we can *minimize* the negative logarithm

$$\begin{aligned} -l(\theta, \theta_0) &= \sum_{t=1}^{n} \overbrace{-\log P(y_t | \mathbf{x}_t, \theta, \theta_0)}^{\text{log-loss}} && (11) \\ &= \sum_{t=1}^{n} -\log g\left(y_t(\theta^T \mathbf{x}_t + \theta_0) \right) && (12) \\ &= \sum_{t=1}^{n} \log \left[1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0) \right) \right] && (13) \end{aligned}$$

We can interpret this similarly to the sum of the hinge losses in the SVM approach. As before, we have a base loss function, here $\log[1+\exp(-z)]$ (Figure 1b), similar to the hinge loss (Figure 1a), and this loss depends only on the value of the "margin" $y_t(\theta^T \mathbf{x}_t + \theta_0)$ for each example. The difference here is that we have a clear probabilistic interpretation of the "strength" of the prediction, i.e., how high $P(y_t | \mathbf{x}_t, \theta, \theta_0)$ is for any particular example. Having a probabilistic interpretation does not, however, mean that the probability values are in any way sensible or *calibrated*. Predicted probabilities are *calibrated* when they

[1] An *estimator* is a function that maps data to parameter values. An *estimate* is the value obtained in response to specific data.

correspond to observed frequencies. So, for example, if we group together all the examples for which we predict positive label with probability 0.5, then roughly half of them should be labeled +1. Probability estimates are rarely well-calibrated but can nevertheless be useful.

The minimization problem we have defined above is convex and there are a number of optimization methods available for finding the minimizing $\hat{\theta}$ and $\hat{\theta}_0$ including simple gradient descent. In a simple (stochastic) gradient descent, we would modify the parameters in response to each term in the sum (based on each training example). To specify the updates we need the following derivatives

$$\frac{d}{d\theta_0} \log \left[1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0) \right)\right] = -y_t \frac{\exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0)\right)}{1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0) \right)} \quad (14)$$

$$= -y_t[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)] \quad (15)$$

and

$$\frac{d}{d\theta} \log \left[1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0) \right)\right] = -y_t \mathbf{x}_t[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)] \quad (16)$$

The parameters are then updated by selecting training examples at random and moving the parameters in the opposite direction of the derivatives:

$$\theta_0 \leftarrow \theta_0 + \eta \cdot y_t[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)] \quad (17)$$

$$\theta \leftarrow \theta + \eta \cdot y_t \mathbf{x}_t[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)] \quad (18)$$

where η is a small (positive) learning rate. Note that $P(y_t|\mathbf{x}_t, \theta, \theta_0)$ is the probability that we predict the training label correctly and $[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)]$ is the probability of making a mistake. The stochastic gradient descent updates in the logistic regression context therefore strongly resemble the perceptron mistake driven updates. The key difference here is that the updates are graded, made in proportion to the probability of making a mistake.

The stochastic gradient descent algorithm leads to no significant change on average when the gradient of the full objective equals zero. Setting the gradient to zero is also a necessary condition of optimality:

$$\frac{d}{d\theta_0}(-l(\theta, \theta_0) = -\sum_{t=1}^{n} y_t[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)] = 0 \quad (19)$$

$$\frac{d}{d\theta}(-l(\theta, \theta_0)) = -\sum_{t=1}^{n} y_t \mathbf{x}_t[1 - P(y_t|\mathbf{x}_t, \theta, \theta_0)] = 0 \quad (20)$$

The sum in Eq.(19) is the difference between mistake probabilities associated with positively and negatively labeled examples. The optimality of θ_0 therefore ensures that the mistakes are balanced in this (soft) sense. Another way of understanding this is that the vector of mistake probabilities is orthogonal to the vector of labels. Similarly, the optimal setting of θ is characterized by mistake probabilities that are orthogonal to all rows of the label-example matrix $\tilde{\mathbf{X}} = [y_1\mathbf{x}_1, \ldots, y_n\mathbf{x}_n]$. In other words, for each dimension j of the example vectors, $[y_1 x_{1j}, \ldots, y_n x_{nj}]$ is orthogonal to the mistake probabilities. Taken together, these orthogonality conditions ensure that there's no further linearly available information in the examples to improve the predicted probabilities (or mistake probabilities). This is perhaps a bit easier to see if we first map ± 1 labels into 0/1 labels: $\tilde{y}_t = (1 + y_t)/2$ so that $\tilde{y}_t \in \{0, 1\}$. Then the above optimality conditions can be rewritten in terms of prediction errors $[\tilde{y}_t - P(y = 1|\mathbf{x}_t, \theta, \theta_0)]$ rather than mistake probabilities as

$$\sum_{t=1}^{n} [\tilde{y}_t - P(y = 1|\mathbf{x}_t, \theta, \theta_0)] = 0 \tag{21}$$

$$\sum_{t=1}^{n} \mathbf{x}_t [\tilde{y}_t - P(y = 1|\mathbf{x}_t, \theta, \theta_0)] = 0 \tag{22}$$

and

$$\theta_0' \sum_{t=1}^{n} [\tilde{y}_t - P(y = 1|\mathbf{x}_t, \theta, \theta_0)] + \theta'^T \sum_{t=1}^{n} \mathbf{x}_t [\tilde{y}_t - P(y = 1|\mathbf{x}_t, \theta, \theta_0)] \tag{23}$$

$$= \sum_{t=1}^{n} (\theta'^T \mathbf{x}_t + \theta_0)[\tilde{y}_t - P(y = 1|\mathbf{x}_t, \theta, \theta_0)] = 0 \tag{24}$$

meaning that the prediction errors are orthogonal to any linear function of the inputs.

Let's try to briefly understand the type of predictions we could obtain via maximum likelihood estimation of the logistic regression model. Suppose the training examples are linearly separable. In this case we can find parameter values such that $y_t(\theta^T \mathbf{x}_t + \theta_0)$ are positive for all training examples. By scaling up the parameters, we make these values larger and larger. This is beneficial as far as the likelihood model is concerned since the log of the logistic function is strictly increasing as a function of $y_t(\theta^T \mathbf{x}_t + \theta_0)$ (the loss $\log\left[1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0)\right)\right]$ is strictly decreasing). Thus, as a result, the maximum likelihood parameter values would become unbounded, and infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood (likelihood of exactly one or the loss exactly zero). The resulting probability values, predicting each training label correctly with probability one, are hardly accurate in the sense of reflecting

our uncertainty about what the labels might be. So, when the number of training examples is small we would need to add the regularizer $\|\theta\|^2/2$ just as in the SVM model. The regularizer helps select reasonable parameters when the available training data fails to sufficiently constrain the linear classifier.

To estimate the parameters of the logistic regression model with regularization we would minimize instead

$$\frac{1}{2}\|\theta\|^2 + C\sum_{t=1}^{n} \log\left[1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0)\right)\right] \tag{25}$$

where the constant C again specifies the trade-off between correct classification (the objective) and the regularization penalty. The regularization problem is typically written (equivalently) as

$$\frac{\lambda}{2}\|\theta\|^2 + \sum_{t=1}^{n} \log\left[1 + \exp\left(-y_t(\theta^T \mathbf{x}_t + \theta_0)\right)\right] \tag{26}$$

since it seems more natural to vary the strength of regularization with λ while keeping the objective the same.

Linear regression, active learning

We arrived at the logistic regression model when trying to explicitly model the uncertainty about the labels in a linear classifier. The same general modeling approach permits us to use linear predictions in various other contexts as well. The simplest of them is regression where the goal is to predict a continuous response $y_t \in \mathcal{R}$ to each example vector. Here too focusing on linear predictions won't be inherently limiting as linear predictions can be easily extended (next lecture).

So, how should we model continuous responses? The linear function of the input already produces a "mean prediction" or $\theta^T \mathbf{x} + \theta_0$. By treating this as a mean prediction more formally, we are stating that the expected value of the response variable, conditioned on $\mathbf{x}$, is $\theta^T \mathbf{x} + \theta_0$. More succinctly, we say that $E\{y|\mathbf{x}\} = \theta^T \mathbf{x} + \theta_0$. It remains to associate a distribution over the responses around such mean prediction. The simplest symmetric distribution is the normal (Gaussian) distribution. In other words, we say that the responses y follow the normal pdf

$$N(y;\, \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y-\mu)^2 \right) \tag{1}$$

where $\mu = \theta^T \mathbf{x} + \theta_0$. Our response model is therefore defined as

$$P(y|\mathbf{x}, \theta, \theta_0) = N(y;\, \theta^T \mathbf{x} + \theta_0,\, \sigma^2) \tag{2}$$

So, when the input is 1-dimensional, we predict a mean response that is a line in (x, y) space, and assume that noise in y is normally distributed with zero mean and variance σ^2. Note that the noise variance σ^2 in the model does not depend on the input x. Moreover, we only model variation in the y-direction while expecting to know x with perfect precision. Taking into account the effect of potential noise in x on the responses y would tie parameters θ and θ_0 to the noise variance σ^2, potentially in an input dependent manner. The specifics of this coupling depend on the form of noise added to x. We will discuss this in a bit more detail later on.

We can also write the linear regression model in another way to explicate how exactly the additive noise appears in the responses:

$$y = \theta^T \mathbf{x} + \theta_0 + \epsilon \tag{3}$$

where $\epsilon \sim N(0, \sigma^2)$ (meaning that noise ϵ is distributed normally with mean zero and variance σ^2). Clearly for this model $E\{y|\mathbf{x}\} = \theta^T \mathbf{x} + \theta_0$ since ϵ has zero mean. Moreover, adding Gaussian noise to a deterministic prediction $\theta^T \mathbf{x} + \theta_0$ makes y normally distributed

with mean $\theta^T \mathbf{x} + \theta_0$ and variance σ^2, as before. So, in particular, for the training inputs $\mathbf{x}_1, \ldots, \mathbf{x}_n$ and outputs $y_1, \ldots, y_n$, the model relating them is

$$y_t = \theta^T \mathbf{x}_t + \theta_0 + \epsilon_t, \quad t = 1, \ldots, n \tag{4}$$

where $e_t \sim N(0, \sigma^2)$ and e_i is independent of e_j for any $i \neq i$.

Regardless of how we choose the write the model (both forms are useful) we can find the parameter estimates by maximizing the conditional likelihood. Similarly to the logistic regression case, the conditional likelihood is written as

$$L(\theta, \theta_0, \sigma^2) = \prod_{t=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \right) \tag{5}$$

Note that σ^2 is also a parameter we have to estimate. It accounts for errors not captured by the linear model. In terms of the log-likelihood, we try to maximize

$$\begin{aligned} l(\theta, \theta_0, \sigma^2) &= \sum_{t=1}^{n} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \right) \right] &(6)\\ &= \sum_{t=1}^{n} \left[-\frac{1}{2}\log(2\pi) - \frac{1}{2}\log\sigma^2 - \frac{1}{2\sigma^2}(y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \right] &(7)\\ &= \text{const.} - \frac{n}{2}\log\sigma^2 - \frac{1}{2\sigma^2}\sum_{t=1}^{n}(y_t - \theta^T \mathbf{x}_t - \theta_0)^2 &(8) \end{aligned}$$

where 'const.' absorbs terms that do not depend on the parameters. Now, the problem of estimating the parameters θ and θ_0 is nicely decoupled from estimating σ^2. In other words, we can find the *maximizing* $\hat{\theta}$ and $\hat{\theta}_0$ by simply *minimizing* the mean squared error

$$\sum_{t=1}^{n}(y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \tag{9}$$

It is perhaps easiest to write the solution based on a bit of matrix calculation. Let $\mathbf{X}$ be a matrix whose rows, indexed by training examples, are given by $[\mathbf{x}_t^T, 1]$ ($\mathbf{x}_t$ turned into a row vector and 1 added at the end). In terms of this matrix, the minimization problem

becomes

$$\sum_{t=1}^{n}\left(y_t - \begin{bmatrix}\theta\\ \theta_0\end{bmatrix}^T \begin{bmatrix}\mathbf{x}_t\\ 1\end{bmatrix}\right)^2 = \sum_{t=1}^{n}\left(y_t - [\mathbf{x}_t^T, 1]\begin{bmatrix}\theta\\ \theta_0\end{bmatrix}\right)^2 \quad (10)$$

$$= \left\| \begin{bmatrix} y_1\\ \cdots\\ y_n\end{bmatrix} - \begin{bmatrix}\mathbf{x}_1^T, 1\\ \cdots\\ \mathbf{x}_n^T, 1\end{bmatrix}\begin{bmatrix}\theta\\ \theta_0\end{bmatrix}\right\|^2 \quad (11)$$

$$= \left\|\mathbf{y} - \mathbf{X}\begin{bmatrix}\theta\\ \theta_0\end{bmatrix}\right\|^2 \quad (12)$$

$$= \mathbf{y}^T\mathbf{y} - 2\begin{bmatrix}\theta\\ \theta_0\end{bmatrix}^T\mathbf{X}^T\mathbf{y} + \begin{bmatrix}\theta\\ \theta_0\end{bmatrix}^T\mathbf{X}^T\mathbf{X}\begin{bmatrix}\theta\\ \theta_0\end{bmatrix} \quad (13)$$

where $\mathbf{y} = [y_1, \ldots, y_n]^T$ is a vector of training responses. Solving it yields

$$\begin{bmatrix}\hat{\theta}\\ \hat{\theta}_0\end{bmatrix} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \quad (14)$$

Note that the optimal parameter values are linear functions of the observed responses $\mathbf{y}$. We will make use of this property later on. The dependence on the training inputs $\mathbf{x}_1, \ldots, \mathbf{x}_n$ (or the matrix $\mathbf{X}$) is non-linear, however.

The noise variance can be subsequently set to account for the remaining prediction errors. Indeed, the the maximizing value of σ^2 is given by

$$\hat{\sigma}^2 = \frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{\theta}^T\mathbf{x}_t - \hat{\theta}_0)^2 \quad (15)$$

which is the average squared prediction error. Note that we cannot compute $\hat{\sigma}^2$ before knowing how well the linear model explains the responses.

Bias and variance of the parameter estimates

We can make use of the closed form parameter estimates in Eq.(14) to analyze how good these estimates are. For this purpose let's make the strong assumption that the actual relation between $\mathbf{x}$ and y follows a linear model of the same type that we are estimating (we just don't know the correct parameter values θ^*, θ_0^*, and σ^{*2}). We can therefore describe the observed responses y_t as

$$y_t = \theta^{*T}\mathbf{x}_t + \theta_0^* + \epsilon_t, \quad t = 1, \ldots, n \quad (16)$$

where $\epsilon_t \sim N(0, \sigma^{*2})$. In a matrix form

$$\mathbf{y} = \mathbf{X} \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} + \mathbf{e} \tag{17}$$

where $\mathbf{e} = [\epsilon_1, \ldots, \epsilon_n]^T$, $E\{\mathbf{e}\} = 0$ and $E\{\mathbf{e}\mathbf{e}^T\} = \sigma^{*2}\,\mathbf{I}$. The noise vector $\mathbf{e}$ is also independent of the inputs or $\mathbf{X}$. Plugging this form of responses into Eq.(14) we get

$$\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T(\mathbf{X} \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} + \mathbf{e}) \tag{18}$$

$$= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X} \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} + (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{e} \tag{19}$$

$$= \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} + (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{e} \tag{20}$$

In other words, our parameter estimates can be decomposed into the sum of correct underlying parameters and estimates based on noise alone (i.e., based on $\mathbf{e}$). Thus, on average with fixed inputs

$$E\{\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} |\mathbf{X}\} = \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} + (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T E\{\mathbf{e}|\mathbf{X}\} = \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \tag{21}$$

Our parameter estimates are therefore *unbiased* or correct on average when averaging is over possible training sets we could generate. The averaging here is conditioned on the specific training inputs.

Using Eq.(20) and Eq.(21) we can also evaluate the conditional co-variance of the parameter estimates where the expectation is again over the noise in the outputs:

$$Cov\{\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} |\mathbf{X}\} = E\left\{ \left(\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \right) \left(\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \right)^T |\mathbf{X} \right\} \tag{22}$$

$$= E\left\{ \left((\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{e}\right) \left((\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{e}\right)^T |\mathbf{X} \right\} \tag{23}$$

$$= E\left\{ (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{e}\mathbf{e}^T\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} | \mathbf{X} \right\} \tag{24}$$

$$= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\, E\{\mathbf{e}\mathbf{e}^T|\mathbf{X}\}\, \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} \tag{25}$$

$$= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\, (\sigma^{*2}\,\mathbf{I})\, \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} \tag{26}$$

$$= \sigma^{*2}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} \tag{27}$$

$$= \sigma^{*2}(\mathbf{X}^T\mathbf{X})^{-1} \tag{28}$$

So the way in which the parameters vary in response to noise in the outputs is a function of the inputs or $\mathbf{X}$. We will use this property in the next section to select inputs so as to improve the quality of the parameter estimates or to reduce the variance of predictions.

Based on the bias and variance calculations we can evaluate the mean squared error of the parameter estimates. To this end, we use the fact that the expectation of the squared norm of any vector valued random variable can be decomposed into a bias and variance components as follows:

$$
\begin{aligned}
E\left\{ \|\mathbf{z} - \mathbf{z}^*\|^2 \right\} &= E\left\{ \|\mathbf{z} - E\{\mathbf{z}\} + E\{\mathbf{z}\} - \mathbf{z}^*\|^2 \right\} && (29)\\
&= E\left\{ \|\mathbf{z} - E\{\mathbf{z}\}\|^2 + 2(\mathbf{z} - E\{\mathbf{z}\})^T (E\{\mathbf{z}\} - \mathbf{z}^*) + \|E\{\mathbf{z}\} - \mathbf{z}^*\|^2 \right\} && (30)\\
&= E\left\{ \|\mathbf{z} - E\{\mathbf{z}\}\|^2 \right\} + 2E\left\{ (\mathbf{z} - E\{\mathbf{z}\})^T \right\} (E\{\mathbf{z}\} - \mathbf{z}^*) + \|E\{\mathbf{z}\} - \mathbf{z}^*\|^2 \\
&= \overbrace{E\left\{ \|\mathbf{z} - E\{\mathbf{z}\}\|^2 \right\}}^{\text{variance}} + \overbrace{\|E\{\mathbf{z}\} - \mathbf{z}^*\|^2}^{\text{bias}^2} && (31)
\end{aligned}
$$

where we have assumed that $\mathbf{z}^*$ is fixed. Make sure you understand how this decomposition is derived. We will further elaborate the variance part to better use the result in our context:

$$
\begin{aligned}
E\left\{ \|\mathbf{z} - E\{\mathbf{z}\}\|^2 \right\} &= E\left\{ (\mathbf{z} - E\{\mathbf{z}\})^T (\mathbf{z} - E\{\mathbf{z}\}) \right\} && (32)\\
&= E\left\{ Tr\left[(\mathbf{z} - E\{\mathbf{z}\})^T (\mathbf{z} - E\{\mathbf{z}\}) \right] \right\} && (33)\\
&= E\left\{ Tr\left[(\mathbf{z} - E\{\mathbf{z}\})(\mathbf{z} - E\{\mathbf{z}\})^T \right] \right\} && (34)\\
&= Tr\left[E\left\{ (\mathbf{z} - E\{\mathbf{z}\})(\mathbf{z} - E\{\mathbf{z}\})^T \right\} \right] && (35)\\
&= Tr\left[Cov\{\mathbf{z}\} \right] && (36)
\end{aligned}
$$

where $Tr[\cdot]$ is the matrix *trace*, the sum of its diagonal components, and therefore a linear operation (exchangeable with the expectation). We have also used the fact that $Tr[\mathbf{a}^T\mathbf{b}] = Tr[\mathbf{a}\mathbf{b}^T]$ for any vectors $\mathbf{a}$ and $\mathbf{b}$.

Now, adapting the result to our setting, we get

$$E\left\{ \left\| \begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \right\|^2 |\mathbf{X} \right\} = \overbrace{Tr\left[Cov\left\{ \begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} |\mathbf{X} \right\} \right]}^{\text{variance}} + \overbrace{\left\| E\left\{ \begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} |\mathbf{X} \right\} - \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \right\|^2}^{\text{bias}^2=0}$$

$$= \sigma^{*2} Tr\left[(\mathbf{X}^T\mathbf{X})^{-1} \right] \quad (37)$$

Let's understand this result a bit further. How does it depend on n, the number of training examples? In other words, how quickly does the mean squared error decrease as the number of training examples increases, assuming the input examples $\mathbf{x}$ are sampled independently from some underlying distribution $P(\mathbf{x})$? To answer this let's start by analyzing what happens to the matrix $\mathbf{X}^T\mathbf{X}$:

$$\mathbf{X}^T\mathbf{X} = \sum_{t=1}^{n} \begin{bmatrix} \mathbf{x}_t \\ 1 \end{bmatrix} [\mathbf{x}_t^T, 1] \quad (38)$$

$$= n \cdot \frac{1}{n} \sum_{t=1}^{n} \begin{bmatrix} \mathbf{x}_t \\ 1 \end{bmatrix} [\mathbf{x}_t^T, 1] \quad (39)$$

$$\approx n \cdot E_{\mathbf{x}\sim P}\left\{ \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} [\mathbf{x}^T, 1] \right\} = n \cdot \mathbf{C} \quad (40)$$

where for large n the average will be close to the corresponding expected value. For large n the mean squared error of the parameter estimates will therefore be close to

$$\frac{\sigma^{*2}}{n} \cdot Tr[\mathbf{C}^{-1}] \quad (41)$$

The variance of simply averaging the (noise in the) outputs would behave as σ^{*2}/n. Since we are estimating $d+1$ parameters where d is the input dimension, this dependence would have to be in $Tr[\mathbf{C}^{-1}]$. Indeed it is. This term, a trace of a $(d+1) \times (d+1)$ matrix $\mathbf{C}^{-1}$, is directly proportional to $d+1$.

Penalized log-likelihood and Ridge regression

When the number of training examples is small, i.e., not too much larger than the number of parameters (dimension of the inputs), it is often beneficial to *regularize* the parameter estimates. We will derive the form of regularization here by assigning a prior distribution over the parameters $P(\theta, \theta_0)$. The purpose of the prior is to prefer small parameter values

(predict values close to zero) in the absence of data. Specifically, we will look at simple zero mean Gaussian distributions

$$P(\theta, \theta_0; \sigma'^2) = N(\begin{bmatrix} \theta \\ \theta_0 \end{bmatrix}; \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \sigma'^2 \mathbf{I}) = N(\theta_0; 0, \sigma'^2) \prod_{j=1}^{d} N(\theta_j; 0, \sigma'^2) \tag{42}$$

where the variance parameter σ'^2 in the prior distribution specifies how strongly we wish to bias the parameters towards zero.

By combining the log-likelihood criterion with the prior we obtain a *penalized log-likelihood* function (penalized by the prior):

$$\begin{aligned} l'(\theta, \theta_0, \sigma^2) &= \sum_{t=1}^{n} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \right) \right] + \log P(\theta, \theta_0; \sigma'^2) \quad (43) \\ &= \text{const.} - \frac{n}{2} \log \sigma^2 - \frac{1}{2\sigma^2} \sum_{t=1}^{n} (y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \\ &\quad - \frac{1}{2\sigma'^2}(\theta_0^2 + \sum_{j=1}^{d} \theta_j^2) - \frac{d+1}{2} \log \sigma'^2 \quad (44) \end{aligned}$$

It is convenient to tie the prior variance σ'^2 to the noise variance σ^2 according to $\sigma'^2 = \sigma^2/\lambda$. This has the effect that if the noise variance σ^2 is large, we penalize the parameters very little (permit large deviations from zero by assuming a large σ'^2). On the other hand, if the noise variance is small, we could be *over-fitting* the linear model. This happens, for example, when the number of training examples is small. In this case most of the responses can be explained directly by the linear model making the noise variance very small. In such cases our penalty for the parameters will be larger as well (prior variance is smaller).

Incorporating this parameter tie into the penalized log-likelihood function gives

$$\begin{aligned} l'(\theta, \theta_0, \sigma^2) &= \text{const.} - \frac{n}{2} \log \sigma^2 - \frac{1}{2\sigma^2} \sum_{t=1}^{n} (y_t - \theta^T \mathbf{x}_t - \theta_0)^2 \\ &\quad - \frac{\lambda}{2\sigma^2}(\theta_0^2 + \sum_{j=1}^{d} \theta_j^2) - \frac{d+1}{2} \log(\sigma^2/\lambda) \quad (45) \\ &= \text{const.} - \frac{n+d+1}{2} \log \sigma^2 + \frac{d+1}{2} \log \lambda \quad (46) \\ &\quad - \frac{1}{2\sigma^2} \left[\sum_{t=1}^{n} (y_t - \theta^T \mathbf{x}_t - \theta_0)^2 + \lambda(\theta_0^2 + \sum_{j=1}^{d} \theta_j^2) \right] \quad (47) \end{aligned}$$

where again the estimation of θ and θ_0 separates from setting the noise variance σ^2. Note that this separation is achieved because we tied the prior and noise variance parameters. The above regularized problem of finding the parameter estimates $\hat{\theta}$ and $\hat{\theta}_0$ is known as *Ridge regression*.

As before, we can get closed form estimates for the parameters (we omit the analogous derivation):

$$\left[\begin{array}{c} \hat{\theta} \\ \hat{\theta}_0 \end{array} \right] = (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \tag{48}$$

It is now useful to understand how the properties of these parameter estimates depend on λ. For example, are the parameter estimates *unbiased*? No, they are not:

$$\begin{aligned} E\left\{ \left[\begin{array}{c} \hat{\theta} \\ \hat{\theta}_0 \end{array} \right] | \mathbf{X} \right\} &= (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} \left[\begin{array}{c} \theta^* \\ \theta_0^* \end{array} \right] & (49) \\ &= (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} - \lambda \mathbf{I}) \left[\begin{array}{c} \theta^* \\ \theta_0^* \end{array} \right] & (50) \\ &= \left[\begin{array}{c} \theta^* \\ \hat{\theta}_0^* \end{array} \right] \overbrace{-\lambda (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \left[\begin{array}{c} \theta^* \\ \theta_0^* \end{array} \right]}^{\text{bias}} & (51) \\ &= \left(\mathbf{I} - \lambda (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \right) \left[\begin{array}{c} \theta^* \\ \theta_0^* \end{array} \right] & (52) \end{aligned}$$

It is straightforward to check that $\left(\mathbf{I} - \lambda (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \right)$ is a positive definite matrix with eigenvalues all less than one. The parameter estimates are therefore shrunk towards zero and more so the larger the value of λ. This is what we would expect since we explicitly favored small parameter values with the prior penalty. What do we gain from such *biased* parameter estimates? Let's evaluate the mean squared error, starting with the covariance:

$$\begin{aligned} Cov\left\{ \left[\begin{array}{c} \hat{\theta} \\ \hat{\theta}_0 \end{array} \right] | \mathbf{X} \right\} &= \sigma^{*2} (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} & (53) \\ &= \sigma^{*2} (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X} - \lambda \mathbf{I}) (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} & (54) \\ &= \sigma^{*2} (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} - \lambda \sigma^{*2} (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-2} & (55) \end{aligned}$$

The mean squared error in the parameters is therefore given by (we again omit the deriva-

tion that can be obtained similarly to previous expressions):

$$E\left\{\left\|\begin{bmatrix}\hat{\theta}\\ \hat{\theta}_0\end{bmatrix}-\begin{bmatrix}\theta^*\\ \theta_0^*\end{bmatrix}\right\|^2|\mathbf{X}\right\} = \sigma^{*2}\cdot Tr\left[(\lambda\mathbf{I}+\mathbf{X}^T\mathbf{X})^{-1}-\lambda(\lambda\mathbf{I}+\mathbf{X}^T\mathbf{X})^{-2}\right] + \lambda^2\begin{bmatrix}\theta^*\\ \theta_0^*\end{bmatrix}^T(\lambda\mathbf{I}+\mathbf{X}^T\mathbf{X})^{-2}\begin{bmatrix}\theta^*\\ \theta_0^*\end{bmatrix} \quad (56)$$

Can this be smaller than the mean squared error corresponding to the unregularized estimates $\sigma^{*2}\cdot Tr\left[(\mathbf{X}^T\mathbf{X})^{-1}\right]$? Yes, it can. This is indeed the benefit from regularization: we can reduce large variance at the cost of introducing a bit of bias. We will get back to this trade-off in the context of *model selection*.

Let's exemplify the effect of λ on the mean squared error in a context of a very simple 1-dimensional example. Suppose, we have observed responses for only two points, $x=-1$ and $x=1$. In this case,

$$\mathbf{X}=\begin{bmatrix}-1 & 1\\ 1 & 1\end{bmatrix},\ \mathbf{X}^T\mathbf{X}=\begin{bmatrix}2 & 0\\ 0 & 2\end{bmatrix},\ (\lambda\mathbf{I}+\mathbf{X}^T\mathbf{X})^{-1}=\begin{bmatrix}1/(2+\lambda) & 0\\ 0 & 1/(2+\lambda)\end{bmatrix} \quad (57)$$

The expression for the mean squared error therefore becomes

$$\begin{aligned}E\left\{\left\|\begin{bmatrix}\hat{\theta}\\ \hat{\theta}_0\end{bmatrix}-\begin{bmatrix}\theta^*\\ \theta_0^*\end{bmatrix}\right\|^2|\mathbf{X}\right\} &= \sigma^{*2}\left(\frac{2}{(2+\lambda)}-\frac{2\lambda}{(2+\lambda)^2}\right)+\frac{\lambda^2}{(2+\lambda)^2}(\theta^{*2}+\theta_0^{*2})\\ &= \frac{4\sigma^{*2}}{(2+\lambda)^2}+\frac{\lambda^2}{(2+\lambda)^2}(\theta^{*2}+\theta_0^{*2}) \quad (58)\end{aligned}$$

We should compare this to $\sigma^{*2}Tr\left[(\mathbf{X}^T\mathbf{X})^{-1}\right]=\sigma^{*2}$ obtained without regularization (corresponds to setting $\lambda=0$). In the noisy case $\sigma^{*2}>\theta^{*2}+\theta_0^{*2}$ we can set $\lambda=2$ and obtain

$$E\left\{\left\|\begin{bmatrix}\hat{\theta}\\ \hat{\theta}_0\end{bmatrix}-\begin{bmatrix}\theta^*\\ \theta_0^*\end{bmatrix}\right\|^2|\mathbf{X}\right\}=\frac{4\sigma^{*2}}{16}+\frac{4}{16}(\theta^{*2}+\theta_0^{*2})<\frac{8\sigma^{*2}}{16}=\frac{1}{2}\sigma^{*2} \quad (59)$$

The mean squared error of the parameters is therefore clearly smaller than without regularization.

Active learning

We can use the expressions for the mean squared error to actively select input points $\mathbf{x}_1,\ldots,\mathbf{x}_n$, when possible, so as to reduce the resulting estimation error. This is an *active*

learning (*experiment design*) problem. The goal is to get by with as few responses as possible without sacrificing the estimation accuracy. For example, in training a classifier we would like to minimize the number of training images we would have to label in order obtain a certain classification accuracy. In the regression context, we may be selecting among possible experiments to carry out (e.g., choosing different operating points for a factory or gauging market/customer responses to different strategies available to us). By letting the method guide the selection of the training examples (inputs), we will generally need far fewer examples in comparison to selecting them at random from some underlying distribution, database, or trying available experiments at random. Fewer responses means less human effort, less cost, or both.

To develop this further let's go back to the unregularized case where

$$E\left\{ \left\| \begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \right\|^2 |\mathbf{X} \right\} = \sigma^{*2} Tr\left[(\mathbf{X}^T\mathbf{X})^{-1} \right] \tag{60}$$

We do not know the noise variance σ^{*2} for the correct model but it only appears as a multiplicative constant in the above expression and therefore won't affect how we should choose the inputs. When the choice of inputs is indeed up to us (e.g., which experiments to carry out) we can select them so as to minimize $Tr\left[(\mathbf{X}^T\mathbf{X})^{-1} \right]$. One caveat of this approach is that it relies on the underlying relationship between the inputs and the responses to be linear. When this is no longer the case we may end up with clearly suboptimal selections.

We will discuss next time how we can find say n input examples $\mathbf{x}_1, \ldots, \mathbf{x}_n$ that minimize the criterion.

Lecture topics:

- Active learning
- Non-linear predictions, kernels

Active learning

We can use the expressions for the mean squared error to actively select input points $\mathbf{x}_1, \ldots, \mathbf{x}_n$, when possible, so as to reduce the resulting estimation error. This is an *active learning* (*experiment design*) problem. By letting the method guide the selection of the training examples (inputs), we will generally need far fewer examples in comparison to selecting them at random from some underlying distribution, database, or trying available experiments at random.

To develop this further, recall that we continue to assume that the responses y come from some linear model $y = \theta^{*T}\mathbf{x} + \theta_0^* + \epsilon$ where $\epsilon \sim N(0, \sigma^{*2})$. Nothing is assumed about the distribution of $\mathbf{x}$ as the choice of the inputs is in our control. For any given set of inputs, $\mathbf{x}_1, \ldots, \mathbf{x}_n$, we derived last time an expression for the mean squared error of the maximum likelihood parameter estimates $\hat{\theta}$ and $\hat{\theta}_0$:

$$E\left\{ \left\| \begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta_0^* \end{bmatrix} \right\|^2 | \mathbf{X} \right\} = \sigma^{*2} Tr\left[(\mathbf{X}^T\mathbf{X})^{-1} \right] \tag{1}$$

where the expectation is relative to the responses generated from the underlying linear model, i.e., over different training sets generated from the linear model. We do not know the noise variance σ^{*2} for the correct model but it only appears as a multiplicative constant in the above expression and therefore won't affect how we should choose the inputs. When the choice of inputs is indeed up to us (e.g., which experiments to carry out) we can select them so as to minimize $Tr\left[(\mathbf{X}^T\mathbf{X})^{-1} \right]$. One caveat of this approach is that it relies on the underlying relationship between the inputs and the responses to be linear. When this is no longer the case we may end up with clearly suboptimal selections.

Given the selection criterion, how should we find say n input examples $\mathbf{x}_1, \ldots, \mathbf{x}_n$ that minimize it? One simple approach is to select them one after the other, merely optimizing the selection of the next one in light of what we already have. Let's assume then that we already have $\mathbf{X}$ and thus have $\mathbf{A} = (\mathbf{X}^T\mathbf{X})^{-1}$ (assuming it is already invertible). We are trying to select another input example $\mathbf{x}$ that adds a row $[\mathbf{x}^T, 1]$ to $\mathbf{X}$. In an applied context we are typically constrained by what $\mathbf{x}$ can be (e.g., due to the experimental setup). We

will discuss simple constraints below. Let's now evaluate the effect of adding a new (valid) row:

$$\left[\begin{array}{c} \mathbf{X} \\ \mathbf{x}^T\ 1 \end{array}\right]^T \left[\begin{array}{c} \mathbf{X} \\ \mathbf{x}^T\ 1 \end{array}\right] = (\mathbf{X}^T\mathbf{X}) + \left[\begin{array}{c} \mathbf{x} \\ 1 \end{array}\right]\left[\begin{array}{c} \mathbf{x} \\ 1 \end{array}\right]^T = \mathbf{A}^{-1} + \mathbf{v}\mathbf{v}^T \tag{2}$$

where $\mathbf{v} = [\mathbf{x}^T, 1]^T$. We would like to find a valid $\mathbf{v}$ that minimizes

$$Tr\left[(\mathbf{A}^{-1} + \mathbf{v}\mathbf{v}^T)^{-1}\right] \tag{3}$$

The matrix inverse can actually be carried out in closed form (easy enough to check)

$$(\mathbf{A}^{-1} + \mathbf{v}\mathbf{v}^T)^{-1} = \mathbf{A} - \frac{1}{(1 + \mathbf{v}^T\mathbf{A}\mathbf{v})}\mathbf{A}\mathbf{v}\mathbf{v}^T\mathbf{A} \tag{4}$$

so that the trace becomes

$$\begin{aligned} Tr\left[(\mathbf{A}^{-1} + \mathbf{v}\mathbf{v}^T)^{-1}\right] &= Tr\left[\mathbf{A}\right] - \frac{1}{(1 + \mathbf{v}^T\mathbf{A}\mathbf{v})}Tr\left[\mathbf{A}\mathbf{v}\mathbf{v}^T\mathbf{A}\right] &(5)\\ &= Tr\left[\mathbf{A}\right] - \frac{1}{(1 + \mathbf{v}^T\mathbf{A}\mathbf{v})}Tr\left[\mathbf{v}^T\mathbf{A}\mathbf{A}\mathbf{v}\right] &(6)\\ &= Tr\left[\mathbf{A}\right] - \frac{\mathbf{v}^T\mathbf{A}\mathbf{A}\mathbf{v}}{(1 + \mathbf{v}^T\mathbf{A}\mathbf{v})} &(7)\end{aligned}$$

Note that since $Tr[\mathbf{A}] = Tr[(\mathbf{X}^T\mathbf{X})^{-1}]$ is the mean squared error before adding the new example, any choice of $\mathbf{v}$, i.e., any additional example $\mathbf{x}$ will reduce the mean squared error. We are interested in finding the one that reduces the error the most. This is the example that maximizes

$$\frac{\mathbf{v}^T\mathbf{A}\mathbf{A}\mathbf{v}}{(1 + \mathbf{v}^T\mathbf{A}\mathbf{v})} \tag{8}$$

How much can we possibly reduce the squared error? The above term is bounded by the largest eigenvalue of $\mathbf{A}$. In other words, with each new example we can at most remove one degree of freedom from the parameter space. If we assume no constraints on the choice of $\mathbf{v}$, the maximizing vector would be of infinite length and proportional to the eigenvector of $\mathbf{A}$ with the largest eigenvalue (all the eigenvalues of $\mathbf{A}$ are positive as it is an inverse of a positive definite matrix $\mathbf{X}^T\mathbf{X}$). It is indeed advantageous in linear regression to have the input points as far from each other as possible (see Figure 1). If we constrain $\|\mathbf{v}\| \leq c$, then the maximizing $\mathbf{v}$ is the normalized eigenvector of $\mathbf{A}$ with the largest eigenvalue,

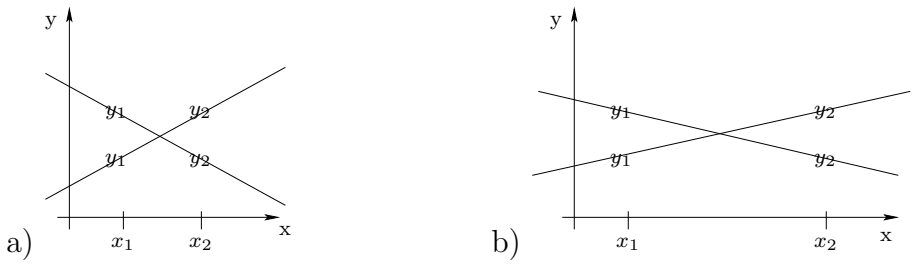


Figure 1: a) The effect of noise in the responses has a large effect on the parameters of the linear model when the corresponding inputs are close to each other; b) the effect is smaller when the inputs are further away.

normalized such that $\|\mathbf{v}\| = c$. Note that it may not be possible to select this eigenvector since $\mathbf{v} = [\mathbf{x}^T, 1]^T$. Other constraints on $\mathbf{x}$ will further restrict $\mathbf{v}$.

Let's take a simple example to illustrate the criterion. Suppose we have a 1-dimensional regression model $y = \theta x + \theta_0 + \epsilon$ where x is constrained to lie within $[-1, 1]$. Assume we have already observed responses for $x_1 = 1$ and $x_2 = -1$. Thus

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}, \; \mathbf{X}^T\mathbf{X} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \; \mathbf{A} = (\mathbf{X}^T\mathbf{X})^{-1} = \frac{1}{2}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{9}$$

$\mathbf{v} = [x, 1]^T$ and therefore $\mathbf{v}^T\mathbf{A}\mathbf{v} = (x^2 + 1)/2$ and $\mathbf{v}^T\mathbf{A}\mathbf{A}\mathbf{v} = (x^2 + 1)/4$. The criterion to be maximized becomes

$$\frac{\mathbf{v}^T\mathbf{A}\mathbf{A}\mathbf{v}}{(1 + \mathbf{v}^T\mathbf{A}\mathbf{v})} = \frac{(x^2 + 1)/4}{1 + (x^2 + 1)/2} \tag{10}$$

Since $z/(1 + z)$ is an increasing function of z, it follows that the criterion is maximized when $(x^2 + 1)/2$ is maximized. Given the constraints, the maximizing point is $x = 1$ or $x = -1$. Either choice would do but, after selecting one, the other one would be preferred at the next step. The result is consistent with the intuition that for linear models the inputs should be as far from each other as possible (cf. Figure 1).

We have so far used the mean squared error in the parameters as the selection criterion. What about the variance in the predictions? Let's try to find the point $\mathbf{x}$ whose response

we are the most uncertain about. We again write $\mathbf{v} = [\mathbf{x}^T, 1]^T$ so that

$$
\begin{aligned}
Var\{y|\mathbf{x}, \mathbf{X}\} &= E\left\{ \left(\hat{\theta}^T \mathbf{x} + \hat{\theta}_0 - \theta^{*T}\mathbf{x} - {\theta^*}_0 \right)^2 |\mathbf{x}, \mathbf{X} \right\} && (11)\\
&= E\left\{ \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}^T \left(\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta^*_0 \end{bmatrix} \right) \left(\begin{bmatrix} \hat{\theta} \\ \hat{\theta}_0 \end{bmatrix} - \begin{bmatrix} \theta^* \\ \theta^*_0 \end{bmatrix} \right)^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} |\mathbf{x}, \mathbf{X} \right\} && (12)\\
&= \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}^T \sigma^{*2} (\mathbf{X}^T\mathbf{X})^{-1} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} && (13)\\
&= \sigma^{*2} \cdot \mathbf{v}^T \mathbf{A} \mathbf{v} && (14)
\end{aligned}
$$

where the expectation is over responses for existing training examples, again assuming that there is a correct underlying linear model. So the largest variance corresponds to the input $\mathbf{x}$ that maximizes $\mathbf{v}^T\mathbf{A}\mathbf{v}$ where $\mathbf{v} = [\mathbf{x}^T, 1]^T$. In the unconstrained case where there are few or no restrictions on $\mathbf{v}$, this maximizing point is exactly the one we would query according the previous selection criterion.

Non-linear predictions, kernels

Essentially all of what we have discussed can be extended to non-linear models, models that remain linear in the parameters as before but perform non-linear operations in the original input space. This is achieved by mapping the input examples to a higher dimensional feature space where the dimensions in the new feature vectors include non-linear functions of the inputs. The simplest setting for demonstrating this is linear regression in one dimension. Consider therefore the linear model $y = \theta x + \theta_0 + \epsilon$ where $\epsilon \sim N(0, \sigma^2)$. We can obtain a quadratic model by simply mapping the input x to a longer feature vector that includes a term quadratic in x. A third order model can be constructed by including all terms up to degree three, and so on. Explicitly, we would make linear predictions using feature vectors

$$
\begin{aligned}
x &\xrightarrow{\phi} [1, \sqrt{2}x, x^2]^T = \phi(x) && (15)\\
x &\xrightarrow{\phi} [1, \sqrt{3}x, \sqrt{3}x^2, x^3]^T && (16)\\
&\cdots && (17)
\end{aligned}
$$

The role of $\sqrt{2}$ and other constants will become clear shortly. The new polynomial regression model is then given by

$$y = \theta^T \phi(x) + \theta_0 + \epsilon, \quad \epsilon \sim N(0, \sigma^2) \tag{18}$$

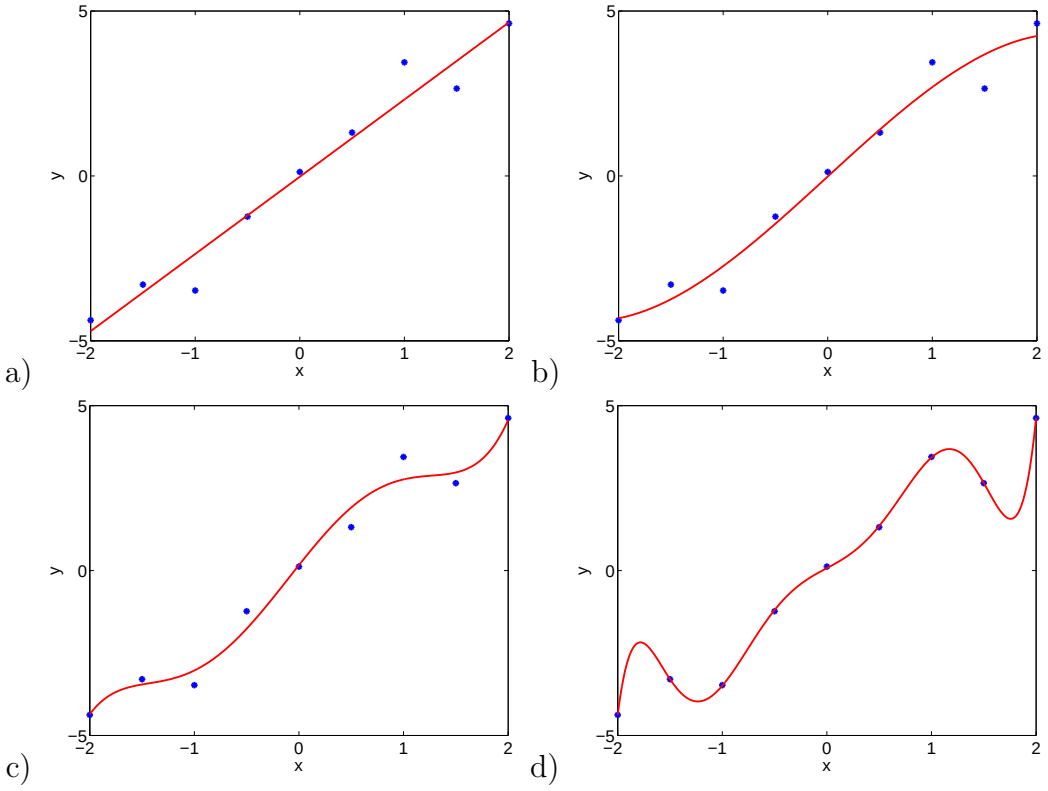


Figure 2: a) a linear model fit; b) a third order polynomial model fit to the same data; c) a fifth order polynomial model; d) a seventh order polynomial model.

where the dimensionality of $\phi(x)$ (and therefore also θ) depends on the order of the polynomial expansion. Regularization of the parameters is almost always used in conjunction with higher dimensional feature vectors. This is necessary since otherwise the higher order models could seriously *overfit* to the data. Examples of fitted polynomial regression models without regularization are shown in Figure 2a-d (the data were generated from a linear model). Note that all these models are linear in the parameters but non-linear in x, save the standard linear regression model in Figure 2a.

The polynomial expansion of input vectors works the same in higher dimensions, e.g.,

$$\mathbf{x} = [x_1, x_2]^T \stackrel{\phi}{\rightarrow} [1, x_1, x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2]^T = \phi(\mathbf{x}) \tag{19}$$

One limitation of explicating the feature vectors is that the dimensionality can increase rapidly with the degree of polynomial expansion, especially when the dimension of the

input vector is already high. The table below gives some indicating of this though the effect is more dramatic with higher input dimensions. Can we somehow avoid explicating the dimensions of the feature vectors? In the context of models we have discussed thus far, yes, we can. We can define the feature vectors implicitly by focusing on specifying the values of their inner products or *kernel* instead. To get a sense of the power of this approach, let's evaluate the inner product between two feature vectors corresponding to the specific cubic expansions of 1-dimensional inputs shown before:

$$\phi(x) = [1, \sqrt{3}x, \sqrt{3}x^2, x^3]^T, \quad (20)$$

$$\phi(x') = [1, \sqrt{3}x', \sqrt{3}x'^2, x'^3]^T, \quad (21)$$

$$\phi(x)^T\phi(x') = 1 + 3xx' + 3(xx')^2 + (xx')^3 = (1 + xx')^3 \quad (22)$$

So it seems we can compactly evaluate the inner products between polynomial feature vectors. The effect is more striking with higher dimensional inputs and higher polynomial degrees. (We did have to specify the constants appropriately in the feature vectors to make this work). To shift the modeling from explicit feature vectors to inner products (kernels) we obviously have to first turn the estimation problem into a form that involves only inner products between feature vectors.

$dim(\mathbf{x}) = 2$

degree p	# of features
2	6
3	10
4	15
5	21

$dim(\mathbf{x}) = 3$

degree p	# of features
2	10
3	20
4	35
5	56

Linear regression and kernels

Let's simplify the model slightly by omitting the offset parameter θ_0, reducing the model to $y = \theta^T\phi(\mathbf{x}) + \epsilon$ where $\phi(\mathbf{x})$ is a particular feature expansion (e.g., polynomial). Our goal here is to turn both the estimation problem and the subsequent prediction task into forms that involve only inner products between the feature vectors.

We have already emphasized that regularization is necessary in conjunction with mapping examples to higher dimensional feature vectors. The regularized least squares objective to be minimized, with parameter λ, is given by

$$J(\theta) = \sum_{t=1}^{n} \left(y_t - \theta^T\phi(\mathbf{x}_t)\right)^2 + \lambda\|\theta\|^2 \quad (23)$$

This form can be derived from penalized log-likelihood estimation (see previous lecture notes). The effect of the regularization penalty is to pull all the parameters towards zero. So any linear dimensions in the parameters that the training feature vectors do not pertain to are set explicitly to zero. We would therefore expect the optimal parameters to lie in the span of the feature vectors corresponding to the training examples. This is indeed the case.

We will continue with the derivation of the kernel (inner product) form of the linear regression model next time.

Lecture topics:

- Kernel form of linear regression
- Kernels, examples, construction, properties

Linear regression and kernels

Consider a slightly simpler model where we omit the offset parameter θ_0, reducing the model to $y = \theta^T \phi(\mathbf{x}) + \epsilon$ where $\phi(\mathbf{x})$ is a particular feature expansion (e.g., polynomial). Our goal here is to turn both the estimation problem and the subsequent prediction task into forms that involve only inner products between the feature vectors.

We have already emphasized that regularization is necessary in conjunction with mapping examples to higher dimensional feature vectors. The regularized least squares objective to be minimized, with parameter λ, is given by

$$J(\theta) = \sum_{t=1}^{n} \left(y_t - \theta^T \phi(\mathbf{x}_t)\right)^2 + \lambda \|\theta\|^2 \tag{1}$$

This form can be derived from penalized log-likelihood estimation (see previous lecture notes). The effect of the regularization penalty is to pull all the parameters towards zero. So any linear dimensions in the parameters that the training feature vectors do not pertain to are set explicitly to zero. We would therefore expect the optimal parameters to lie in the span of the feature vectors corresponding to the training examples. This is indeed the case.

As before, the optimality condition for θ follows from setting the gradient to zero:

$$\frac{dJ(\theta)}{d\theta} = -2 \sum_{t=1}^{n} \overbrace{\left(y_t - \theta^T \phi(\mathbf{x}_t)\right)}^{\alpha_t} \phi(\mathbf{x}_t) + 2\lambda\theta = 0 \tag{2}$$

We can therefore construct the optimal θ in terms of prediction differences α_t and the feature vectors:

$$\theta = \frac{1}{\lambda} \sum_{t=1}^{n} \alpha_t \phi(\mathbf{x}_t) \tag{3}$$

The implication is that the optimal θ (however high dimensional) will lie in the span of the feature vectors corresponding to the training examples. This is due to the regularization

penalty we added. But how do we set α_t? The values for α_t can be found by insisting that they indeed can be interpreted as prediction differences:

$$\alpha_t = y_t - \theta^T \phi(\mathbf{x}_t) = y_t - \frac{1}{\lambda} \sum_{t'=1}^{n} \alpha_{t'} \phi(\mathbf{x}_{t'})^T \phi(\mathbf{x}_t) \tag{4}$$

Thus α_t depends only on the actual responses y_t and the inner products between the training examples, the *Gram matrix*:

$$\mathbf{K} = \begin{bmatrix} \phi(\mathbf{x}_1)^T \phi(\mathbf{x}_1) & \cdots & \phi(\mathbf{x}_1)^T \phi(\mathbf{x}_n) \\ \cdots & \cdots & \cdots \\ \phi(\mathbf{x}_n)^T \phi(\mathbf{x}_1) & \ldots & \phi(\mathbf{x}_n)^T \phi(\mathbf{x}_n) \end{bmatrix} \tag{5}$$

In a vector form,

$$\mathbf{a} = [\alpha_1, \ldots, \alpha_n]^T, \tag{6}$$

$$\mathbf{y} = [y_1, \ldots, y_n]^T, \tag{7}$$

$$\mathbf{a} = \mathbf{y} - \frac{1}{\lambda} \mathbf{K}\mathbf{a} \tag{8}$$

the solution is

$$\hat{\mathbf{a}} = \lambda \Big(\lambda \mathbf{I} + \mathbf{K} \Big)^{-1} \mathbf{y} \tag{9}$$

Note that finding the estimates $\hat{\alpha}_t$ requires inverting a $n \times n$ matrix. This is the cost of dealing with inner products as opposed to handing feature vectors directly. In some cases, the benefit is substantial since the feature vectors in the inner products may be infinite dimensional but never needed explicitly.

As a result of finding $\hat{\alpha}_t$ we can cast the predictions for new examples also in terms of inner products:

$$y = \hat{\theta}^T \phi(\mathbf{x}) = \sum_{t=1}^{n} (\hat{\alpha}_t / \lambda) \phi(\mathbf{x}_{t'})^T \phi(\mathbf{x}) = \sum_{t=1}^{n} \hat{\alpha}_t K(\mathbf{x}_{t'}, \mathbf{x}) \tag{10}$$

where we view $K(\mathbf{x}_{t'}, \mathbf{x})$ as a *kernel function*, a function of two arguments $\mathbf{x}_{t'}$ and $\mathbf{x}$.

Kernels

So we have now successfully turned a regularized linear regression problem into a kernel form. This means that we can simply substitute different kernel functions $K(\mathbf{x}, \mathbf{x}')$ into the estimation/prediction equations. This gives us an easy access to a wide range of possible regression functions. Here are a couple of standard examples of kernels:

- *Polynomial kernel*

$$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T \mathbf{x}')^p, \quad p = 1, 2, \ldots \tag{11}$$

- *Radial basis kernel*

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\beta}{2}\|\mathbf{x} - \mathbf{x}'\|^2\right), \quad \beta > 0 \tag{12}$$

We have already discussed the feature vectors corresponding to the polynomial kernel. The components of these feature vectors were polynomial terms up to degree p with specifically chosen coefficients. The restricted choice of coefficients was necessary in order to collapse the inner product calculations.

The feature "vectors" corresponding to the radial basis kernel are infinite dimensional! The components of these "vectors" are indexed by $\mathbf{z} \in \mathcal{R}^d$ where d is the dimension of the original input $\mathbf{x}$. More precisely, the feature vectors are functions:

$$\phi_{\mathbf{z}}(\mathbf{x}) = c(\beta, d)\, N(\mathbf{z};\, \mathbf{x},\, 1/2\beta) \tag{13}$$

where $N(\mathbf{z}; \mathbf{x}, (1/\beta))$ is a normal pdf over $\mathbf{z}$ and $c(\beta, d)$ is a constant. Roughly speaking, the radial basis kernel measures the probability that you would get the same sample $\mathbf{z}$ (in the same small region) from two normal distributions with means $\mathbf{x}$ and $\mathbf{x}'$ and a common variance $1/2\beta$. This is a reasonable measure of "similarity" between $\mathbf{x}$ and $\mathbf{x}'$ and kernels are often defined from this perspective. The inner product giving rise to the radial basis kernel is defined through integration

$$K(\mathbf{x}, \mathbf{x}') = \int \phi_{\mathbf{z}}(\mathbf{x})\phi_{\mathbf{z}}(\mathbf{x}')d\mathbf{z} \tag{14}$$

We can also construct various types of kernels from simpler ones. Here are a few rules to guide us. Assume $K_1(\mathbf{x}, \mathbf{x}')$ and $K_2(\mathbf{x}, \mathbf{x}')$ are valid kernels (correspond to inner products of some feature vectors), then

1. $K(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})K_1(\mathbf{x}, \mathbf{x}')f(\mathbf{x}')$ for any function $f(\mathbf{x})$,
2. $K(\mathbf{x}, \mathbf{x}') = K_1(\mathbf{x}, \mathbf{x}') + K_2(\mathbf{x}, \mathbf{x}')$,
3. $K(\mathbf{x}, \mathbf{x}') = K_1(\mathbf{x}, \mathbf{x}')K_2(\mathbf{x}, \mathbf{x}')$

are all valid kernels. While simple, these rules are quite powerful. Let's first understand these rules from the point of view of the implicit feature vectors. For each rule, let $\phi(\mathbf{x})$ be the feature vector corresponding to K and $\phi^{(1)}(\mathbf{x})$ and $\phi^{(2)}(\mathbf{x})$ the feature vectors associated with K_1 and K_2, respectively. The feature mapping for the first rule is given simply by multiplying with the scalar function $f(\mathbf{x})$:

$$\phi(\mathbf{x}) = f(\mathbf{x})\phi^{(1)}(\mathbf{x}) \tag{15}$$

so that $\phi(\mathbf{x})^T\phi(\mathbf{x}') = f(\mathbf{x})\phi^{(1)}(\mathbf{x})^T\phi^{(1)}(\mathbf{x}')f(\mathbf{x}') = f(\mathbf{x})K_1(\mathbf{x},\mathbf{x}')f(\mathbf{x}')$. The second rule, adding kernels, corresponds to just concatenating the feature vectors

$$\phi(\mathbf{x}) = \left[\begin{array}{c} \phi^{(1)}(\mathbf{x}) \\ \phi^{(2)}(\mathbf{x}) \end{array} \right] \tag{16}$$

The third and the last rule is a little more complicated but not much. Suppose we use a double index i, j to index the components of $\phi(\mathbf{x})$ where i ranges over the components of $\phi^{(1)}(\mathbf{x})$ and j refers to the components of $\phi^{(2)}(\mathbf{x})$. Then

$$\phi_{i,j}(\mathbf{x}) = \phi^{(1)}_i(\mathbf{x})\phi^{(2)}_j(\mathbf{x}) \tag{17}$$

It is now easy to see that

$$\begin{aligned} K(\mathbf{x},\mathbf{x}') &= \phi(\mathbf{x})^T\phi(\mathbf{x}') && (18)\\ &= \sum_{i,j} \phi_{i,j}(\mathbf{x})\phi_{i,j}(\mathbf{x}') && (19)\\ &= \sum_{i,j} \phi^{(1)}_i(\mathbf{x})\phi^{(2)}_j(\mathbf{x})\phi^{(1)}_i(\mathbf{x}')\phi^{(2)}_j(\mathbf{x}') && (20)\\ &= [\sum_i \phi^{(1)}_i(\mathbf{x})\phi^{(1)}_i(\mathbf{x}')][\sum_j \phi^{(2)}_j(\mathbf{x})\phi^{(2)}_j(\mathbf{x}')] && (21)\\ &= [\phi^{(1)}(\mathbf{x})^T\phi^{(1)}(\mathbf{x}')][\phi^{(2)}(\mathbf{x})^T\phi^{(2)}(\mathbf{x}')] && (22)\\ &= K_1(\mathbf{x},\mathbf{x}')K_2(\mathbf{x},\mathbf{x}') && (23) \end{aligned}$$

These construction rules can also be used to verify that something is a valid kernel. As an example, let's figure out why a radial basis kernel

$$K(\mathbf{x},\mathbf{x}') = \exp\{-\frac{1}{2}\|\mathbf{x}-\mathbf{x}'\|^2\} \tag{24}$$

is a valid kernel.

$$\exp\{-\frac{1}{2}\|\mathbf{x}-\mathbf{x}'\|^2\} = \exp\{-\frac{1}{2}\mathbf{x}^T\mathbf{x} + \mathbf{x}^T\mathbf{x}' - \frac{1}{2}\mathbf{x}'^T\mathbf{x}'\} \quad (25)$$

$$= \overbrace{\exp\{-\frac{1}{2}\mathbf{x}^T\mathbf{x}\}}^{f(\mathbf{x})} \cdot \exp\{\mathbf{x}^T\mathbf{x}'\} \cdot \overbrace{\exp\{-\frac{1}{2}\mathbf{x}'^T\mathbf{x}'\}}^{f(\mathbf{x}')} \quad (26)$$

Here $\exp\{\mathbf{x}^T\mathbf{x}'\}$ is a sum of simple products $\mathbf{x}^T\mathbf{x}'$ and is therefore a kernel based on the second and third rules; the first rule allows us to incorporate $f(\mathbf{x})$ and $f(\mathbf{x}')$.

String kernels. It is often necessary to make predictions (classify, assess risk, determine user ratings) on the basis of more complex objects such as variable length sequences or graphs that do not necessarily permit a simple description as points in $\mathcal{R}^d$. The idea of kernels extends to such objects as well. Consider, for example, the case where the inputs $\mathbf{x}$ are variable length sequences (e.g., documents or biosequences) with elements from some common alphabet $\mathcal{A}$ (e.g., letters or protein residues). One way to compare such sequences is to consider subsequences that they may share. Let $\mathbf{u} \in \mathcal{A}^k$ denote a length k sequence from this alphabet and $\mathbf{i}$ a sequence of k indexes. So, for example, we can say that $\mathbf{u} = \mathbf{x}[\mathbf{i}]$ if $u_1 = x_{i_1}$, $u_2 = x_{i_2}$, ..., $u_k = x_{i_k}$. In other words, $\mathbf{x}$ contains the elements of $\mathbf{u}$ in positions $i_1 < i_2 < \cdots < i_k$. If the elements of $\mathbf{u}$ are found in successive positions in $\mathbf{x}$, then $i_k - i_1 = k - 1$. A simple string kernel corresponds to feature vectors with counts of occurences of length k subsequences:

$$\phi_{\mathbf{u}}(\mathbf{x}) = \sum_{\mathbf{i}:\mathbf{u}=\mathbf{x}[\mathbf{i}]} \delta(i_k - i_1, k-1) \quad (27)$$

In other words, the components are indexed by subsequences $\mathbf{u}$ and the value of $\mathbf{u}$-component is the number of times $\mathbf{x}$ contains $\mathbf{u}$ as a contiguous subsequence. For example,

$$\phi_{\texttt{on}}(\texttt{the common construct}) = 2 \quad (28)$$

The number of components in such feature vectors is very large (exponential in k). Yet, the inner product

$$\sum_{\mathbf{u}\in\mathcal{A}^k} \phi_{\mathbf{u}}(\mathbf{x})\phi_{\mathbf{u}}(\mathbf{x}') \quad (29)$$

can be computed efficiently (there are only a limited number of possible contiguous subsequences in $\mathbf{x}$ and $\mathbf{x}'$). The reason for this difference, and the argument in favor of kernels

more generally, is that the feature vectors have to aggregate the information necessary to compare any two sequences while the inner product is evaluated for two *specific* sequences.

We can also relax the requirement that matches must be contiguous. To this end, we define the length of the window of $\mathbf{x}$ where $\mathbf{u}$ appears as $l(\mathbf{i}) = i_k - i_1$. The feature vectors in a *weighted gapped substring kernel* are given by

$$\phi_{\mathbf{u}}(\mathbf{x}) = \sum_{\mathbf{i}:\mathbf{u}=\mathbf{x}[\mathbf{i}]} \lambda^{l(\mathbf{i})} \tag{30}$$

where the parameter $\lambda \in (0, 1)$ specifies the penalty for non-contiguous matches to $\mathbf{u}$. The resulting kernel

$$K(\mathbf{x}, \mathbf{x}') = \sum_{\mathbf{u}\in\mathcal{A}^k} \phi_{\mathbf{u}}(\mathbf{x})\phi_{\mathbf{u}}(\mathbf{x}') = \sum_{\mathbf{u}\in\mathcal{A}^k} \left(\sum_{\mathbf{i}:\mathbf{u}=\mathbf{x}[\mathbf{i}]} \lambda^{l(\mathbf{i})} \right) \left(\sum_{\mathbf{i}:\mathbf{u}=\mathbf{x}'[\mathbf{i}]} \lambda^{l(\mathbf{i})} \right) \tag{31}$$

can be computed recursively. It is often useful to normalize such a kernel so as to remove any immediate effect from the sequence length:

$$\tilde{K}(\mathbf{x}, \mathbf{x}') = \frac{K(\mathbf{x}, \mathbf{x}')}{\sqrt{K(\mathbf{x}, \mathbf{x})}\sqrt{K(\mathbf{x}', \mathbf{x}')}} \tag{32}$$

Appendix (optional): Kernel linear regression with offset

Given a feature expansion specified by $\phi(\mathbf{x})$ we try to minimize

$$J(\theta, \theta_0) = \sum_{t=1}^{n} \left(y_t - \theta^T \phi(\mathbf{x}_t) - \theta_0 \right)^2 + \lambda \|\theta\|^2 \tag{33}$$

where we have chosen *not* to regularize θ_0 to preserve the similarity to classification discussed later on. Not regularizing θ_0 means, e.g., that we do not care whether all the responses have a constant added to them; the value of the objective, after optimizing θ_0, would remain the same with or without such constant.

Setting the derivatives with respect to θ_0 and θ to zero gives the following optimality conditions:

$$\frac{dJ(\theta, \theta_0)}{d\theta_0} = -2 \sum_{t=1}^{n} \left(y_t - \theta^T \phi(\mathbf{x}_t) - \theta_0 \right) = 0 \tag{34}$$

$$\frac{dJ(\theta, \theta_0)}{d\theta} = 2\lambda\theta - 2 \sum_{t=1}^{n} \overbrace{\left(y_t - \theta^T \phi(\mathbf{x}_t) - \theta_0 \right)}^{\alpha_t} \phi(\mathbf{x}_t) = 0 \tag{35}$$

We can therefore construct the optimal θ in terms of prediction differences α_t and the feature vectors as before:

$$\theta = \frac{1}{\lambda}\sum_{t=1}^{n}\alpha_t\phi(\mathbf{x}_t) \tag{36}$$

Using this form of the solution for θ and Eq.(34) we can also express the optimal θ_0 as a function of the prediction differences α_t:

$$\theta_0 = \frac{1}{n}\sum_{t=1}^{n}\left(y_t - \theta^T\phi(\mathbf{x}_t)\right) = \frac{1}{n}\sum_{t=1}^{n}\left(y_t - \frac{1}{\lambda}\sum_{t'=1}^{n}\alpha_{t'}\phi(\mathbf{x}_{t'})^T\phi(\mathbf{x}_t)\right) \tag{37}$$

We can now constrain α_t to take on values that can indeed be interpreted as prediction differences:

$$\begin{aligned}
\alpha_i &= y_i - \theta^T\phi(\mathbf{x}_i) - \theta_0 && (38)\\
&= y_i - \frac{1}{\lambda}\sum_{t'=1}^{n}\alpha_{t'}\phi(\mathbf{x}_{t'})^T\phi(\mathbf{x}_i) - \theta_0 && (39)\\
&= y_i - \frac{1}{\lambda}\sum_{t'=1}^{n}\alpha_{t'}\phi(\mathbf{x}_{t'})^T\phi(\mathbf{x}_i) - \frac{1}{n}\sum_{t=1}^{n}\left(y_t - \frac{1}{\lambda}\sum_{t'=1}^{n}\alpha_{t'}\phi(\mathbf{x}_{t'})^T\phi(\mathbf{x}_t)\right) && (40)\\
&= y_i - \frac{1}{n}\sum_{t=1}^{n}y_t - \frac{1}{\lambda}\sum_{t'=1}^{n}\alpha_{t'}\left(\phi(\mathbf{x}_{t'})^T\phi(\mathbf{x}_i) - \frac{1}{n}\sum_{t=1}^{n}\phi(\mathbf{x}_{t'})^T\phi(\mathbf{x}_t)\right) && (41)
\end{aligned}$$

With the same matrix notation as before, and letting $\mathbf{1} = [1, \ldots, 1]^T$, we can rewrite the above condition as

$$\mathbf{a} = \overbrace{(\mathbf{I} - \mathbf{1}\mathbf{1}^T/n)}^{C}\mathbf{y} - \frac{1}{\lambda}(\mathbf{I} - \mathbf{1}\mathbf{1}^T/n)\mathbf{K}\mathbf{a} \tag{42}$$

where $C = \mathbf{I} - \mathbf{1}\mathbf{1}^T/n$ is a *centering* matrix. Any solution to the above equation has to satisfy $\mathbf{1}^T\mathbf{a} = 0$ (just left multiply the equation with $\mathbf{1}^T$). Note that this is exactly the optimality condition for θ_0 in Eq.(34). Using this "summing to zero" property of the solution we can rewrite the above equation as

$$\mathbf{a} = C\mathbf{y} - \frac{1}{\lambda}C\mathbf{K}C\mathbf{a} \tag{43}$$

where we have introduced an additional centering operation on the right hand side. This cannot change the solution since $C\mathbf{a} = \mathbf{a}$ whenever $\mathbf{1}^T\mathbf{a} = 0$. The solution $\hat{\mathbf{a}}$ is then

$$\hat{\mathbf{a}} = \lambda \left(\lambda \mathbf{I} + C\mathbf{K}C\right)^{-1} C\mathbf{y} \tag{44}$$

Once we have $\hat{\mathbf{a}}$ we can reconstruct $\hat{\theta}_0$ from Eq.(37). $\hat{\theta}^T\phi(\mathbf{x})$ reduces to the kernel form as before.

Lecture topics:

- Support vector machine and kernels
- Kernel optimization, selection

Support vector machine revisited

Our task here is to first turn the support vector machine into its *dual form* where the examples only appear in inner products. To this end, assume we have mapped the examples into feature vectors $\phi(\mathbf{x})$ of dimension d and that the resulting training set $(\phi(\mathbf{x}_1), y_1), \ldots, (\phi(\mathbf{x}_n), y_n)$ is linearly separable. Finding the maximum margin linear separator in the feature space now corresponds to solving

$$\text{minimize } \|\theta\|^2/2 \text{ subject to } y_t(\theta^T\phi(\mathbf{x}_t) + \theta_0) \geq 1, \quad t = 1, \ldots, n \tag{1}$$

We will discuss later on how slack variables affect the resulting kernel (dual) form. They merely complicate the derivation without changing the procedure. Optimization problems of the above type (convex, linear constraints) can be turned into their *dual* form by means of Lagrange multipliers. Specifically, we introduce a non-negative scalar parameter α_t for each inequality constraint and cast the estimation problem in terms of θ and $\alpha = \{\alpha_1, \ldots, \alpha_n\}$:

$$J(\theta, \theta_0; \alpha) = \|\theta\|^2/2 - \sum_{t=1}^{n} \alpha_t \Big[y_t(\theta^T\phi(\mathbf{x}_t) + \theta_0) - 1 \Big] \tag{2}$$

The original minimization problem for θ and θ_0 is recovered by *maximizing* $J(\theta, \theta_0; \alpha)$ with respect to α. In other words,

$$J(\theta, \theta_0) = \max_{\alpha \geq 0} J(\theta, \theta_0; \alpha) \tag{3}$$

where $\alpha \geq 0$ means that all the components α_t are non-negative. Let's try to see first that $J(\theta, \theta_0)$ really is equivalent to the original problem. Suppose we set θ and θ_0 such that at least one of the constraints, say the one corresponding to $(\mathbf{x}_i, y_i)$, is violated. In that case

$$-\alpha_i \Big[y_i(\theta^T\phi(\mathbf{x}_i) + \theta_0) - 1 \Big] > 0 \tag{4}$$

for any $\alpha_i > 0$. We can then set $\alpha_i = \infty$ to obtain $J(\theta, \theta_0) = \infty$. You can think of the Lagrange multipliers playing an adversarial role to enforce the margin constraints. More

formally,

$$J(\theta, \theta_0) = \begin{cases} \|\theta\|^2/2 \;\text{ if }\; y_t(\theta^T \phi(\mathbf{x}_t) + \theta_0) \geq 1, \quad t = 1, \ldots, n \\ \infty, \;\text{ otherwise} \end{cases} \tag{5}$$

So the minimizing θ and θ_0 are therefore those that satisfy the constraints. On the basis of a general set of criteria governing the optimality when dealing with Lagrange multipliers, criteria known as *Slater conditions*, we can actually switch the maximizing over α and the minimization over $\{\theta, \theta_0\}$ and get the same answer:

$$\min_{\theta,\theta_0} \max_{\alpha \geq 0} J(\theta, \theta_0; \alpha) = \max_{\alpha \geq 0} \min_{\theta,\theta_0} J(\theta, \theta_0; \alpha) \tag{6}$$

The left hand side, equivalent to minimizing Eq.(5), is known as the *primal form*, while the right hand side is the *dual form*. Let's solve the right hand side by first obtaining θ and θ_0 as a function of the Lagrange multipliers (and the data). To this end

$$\frac{d}{d\theta_0} J(\theta, \theta_0; \alpha) = -\sum_{t=1}^{n} \alpha_t y_t = 0 \tag{7}$$

$$\frac{d}{d\theta} J(\theta, \theta_0; \alpha) = \theta - \sum_{t=1}^{n} \alpha_t y_t \phi(\mathbf{x}_t) = 0 \tag{8}$$

So, again the solution for θ is in the span of the feature vectors corresponding to the training examples. Substituting this form of the solution for θ back into the objective, and taking into account the constraint corresponding to the optimal θ_0, we get

$$J(\alpha) = \min_{\theta,\theta_0} J(\theta, \theta_0; \alpha) \tag{9}$$

$$= \begin{cases} \sum_{t=1}^{n} \alpha_t - (1/2) \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j [\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)], \;\text{ if }\; \sum_{t=1}^{n} \alpha_t y_t = 0 \\ -\infty, \quad \text{otherwise} \end{cases} \tag{10}$$

The dual form of the solution is therefore obtained by *maximizing*

$$\sum_{t=1}^{n} \alpha_t - (1/2) \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j [\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)], \tag{11}$$

$$\text{subject to} \quad \alpha_t \geq 0, \;\; \sum_{t=1}^{n} \alpha_t y_t = 0 \tag{12}$$

This is the dual or kernel form of the support vector machine, and is also a *quadratic optimization problem*. The constraints are simpler, however. Moreover, the dimension of

the input vectors does not appear explicitly as part of the optimization problem. It is formulated solely on the basis of the Gram matrix:

$$\mathbf{K} = \begin{bmatrix} \phi(\mathbf{x}_1)^T\phi(\mathbf{x}_1) & \cdots & \phi(\mathbf{x}_1)^T\phi(\mathbf{x}_n) \\ \cdots & \cdots & \cdots \\ \phi(\mathbf{x}_n)^T\phi(\mathbf{x}_1) & \ldots & \phi(\mathbf{x}_n)^T\phi(\mathbf{x}_n) \end{bmatrix} \tag{13}$$

We have already seen that the maximum margin hyperplane can be constructed on the basis of only a subset of the training examples. This should also also in terms of the feature vectors. How will this be manifested in the $\hat{\alpha}_t$'s? Many of them will be exactly zero due to the optimization. In fact, they are *non-zero* only for examples (feature vectors) that are *support vectors*.

Once we have solved for $\hat{\alpha}_t$, we can classify any new example according to the discriminant function

$$\hat{y}(\mathbf{x}) = \hat{\theta}^T\phi(\mathbf{x}) + \hat{\theta}_0 \tag{14}$$

$$= \sum_{t=1}^{n} \hat{\alpha}_t y_t [\phi(\mathbf{x}_t)^T\phi(\mathbf{x})] + \hat{\theta}_0 \tag{15}$$

$$= \sum_{t\in \mathrm{SV}} \hat{\alpha}_t y_t [\phi(\mathbf{x}_t)^T\phi(\mathbf{x})] + \hat{\theta}_0 \tag{16}$$

where SV is the set of support vectors corresponding to non-zero values of α_t. We don't know which examples (feature vectors) become as support vectors until we have solved the optimization problem. Moreover, the identity of the support vectors will depend on the feature mapping or the kernel function.

But what is $\hat{\theta}_0$? It appeared to drop out of the optimization problem. We can set θ_0 after solving for $\hat{\alpha}_t$ by looking at the support vectors. Indeed, for all $i \in \mathrm{SV}$ we should have

$$y_i(\hat{\theta}^T\phi(\mathbf{x}_i) + \hat{\theta}_0) = y_i \sum_{t\in \mathrm{SV}} \hat{\alpha}_t [\phi(\mathbf{x}_t)^T\phi(\mathbf{x}_i)] + y_i\hat{\theta}_0 = 1 \tag{17}$$

from which we can easily solve for $\hat{\theta}_0$. In principle, selecting any support vector would suffice but since we typically solve the quadratic program over α_t's only up to some resolution, these constraints may not be satisfied with equality. It is therefore advisable to construct $\hat{\theta}_0$ as the median value of the solutions implied by the support vectors.

What is the geometric margin we attain with some kernel function $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T\phi(\mathbf{x}')$?

It is still $1/\|\hat{\theta}\|$. In a kernel form

$$\hat{\gamma}_{geom} = \left(\sum_{i=1}^{n} \sum_{j=1}^{n} \hat{\alpha}_i \hat{\alpha}_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \right)^{-1/2} \tag{18}$$

Would it make sense to compare geometric margins we attain with different kernels? We could perhaps use it as a criterion for selecting the best kernel function. Unfortunately this won't work without some care. For example, if we multiply all the feature vectors by 2, then the resulting geometric margin will also be twice as large (we just expanded the space; the relations between the points remain the same). It is necessary to perform some normalization before any comparison makes sense.

We have so far assumed that the examples in their feature representations are linearly separable. We'd also like to have the kernel form of the relaxed support vector machine formulation

$$\text{minimize } \|\theta\|^2/2 + C \sum_{t=1}^{n} \xi_t \tag{19}$$

$$\text{subject to } \; y_t(\theta^T \phi(\mathbf{x}_t) + \theta_0) \geq 1 - \xi_t, \;\; t = 1, \ldots, n \tag{20}$$

The resulting dual form is very similar to the simple one we derived above. In fact, the only difference is that the Lagrange multipliers α_t are now also bounded from above by C (the same C as in the above primal formulation). Intuitively, the Lagrange multipliers α_t serve to enforce the classification constraints and adopt larger values for constraints that are harder to satisfy. Without any upper limit, they would simply reach ∞ for any constraint that cannot be satisfied. The limit C specifies the point when we should stop from trying to satisfy such constraints. More formally, the dual form is

$$\sum_{t=1}^{n} \alpha_t - (1/2) \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j [\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)], \tag{21}$$

$$\text{subject to } \;\; 0 \leq \alpha_t \leq C, \;\; \sum_{t=1}^{n} \alpha_t y_t = 0 \tag{22}$$

The resulting discriminant function has the same form except that the $\hat{\alpha}_t$ values can be different. What about $\hat{\theta}_0$? To solve for $\hat{\theta}_0$ we need to identify classification constraints that are satisfied with equality. These are no longer simply the ones for which $\hat{\alpha}_t > 0$ but those corresponding to $0 < \hat{\alpha}_t < C$. In other words, we have to exclude points that violate the margin constraints. These are the ones for which $\hat{\alpha}_t = C$.

Kernel optimization

Whether we are interested in (linear) classification or regression we are faced with the problem of selecting an appropriate kernel function. A step in this direction might be to tailor a particular kernel a bit better to the available data. We could, for example, introduce additional parameters in the kernel and optimize those parameters so as to improve the performance. These parameters could be simple as the β parameter in the radial basis kernel, weight each dimension of the input vectors, or more flexible as finding the best convex combination of basic (fixed) kernels. Key to such an approach is the measure we would optimize. Ideally, this measure would be the generalization error but we obviously have to settle for a surrogate measure. The surrogate measure could be cross-validation or an alternative criterion related to the generalization error (e.g., margin).

Kernel selection

We can also explicitly select among possible kernels and cast the problem as a *model selection* problem. By choosing a kernel we specify the feature vectors on the basis of which linear predictions are made. Each model[1] (class) refers to a set of linear functions (classifiers) based on the chosen feature representation. In many cases the models are nested in the sense that the more "complex" model contains the "simpler" one. We will continue from this further at the next lecture.

[1] In statistics, a model is a family/set of distributions or a family/set of linear separators.

Lecture topics:

- Kernel optimization
- Model (kernel) selection

Kernel optimization

Whether we are interested in (linear) classification or regression we are faced with the problem of selecting an appropriate kernel function. A step in this direction might be to tailor a particular kernel a bit better to the available data. We could, for example, introduce additional parameters in the kernel and optimize those parameters so as to improve the performance. These parameters could be simple as the β parameter in the radial basis kernel, weight each dimension of the input vectors, or more flexible as finding the best convex combination of basic (fixed) kernels. Key to such an approach is the measure we would optimize. Ideally, this measure would be the generalization error but we obviously have to settle for a surrogate measure. The surrogate measure could be cross-validation or an alternative criterion related to the generalization error such as the geometric margin.

We need additional safeguards if we are to use the geometric margin. For example, simply multiplying the feature vectors by two would double the geometric margin. So, without normalization, the margin cannot serve as an appropriate criterion. The simplest way to normalize the feature vectors prior to estimation would be to require that $\|\phi(\mathbf{x})\| = 1$ for all $\mathbf{x}$ regardless of the kernel. This normalization can be done directly in the kernel representation as follows

$$\tilde{K}(\mathbf{x}, \mathbf{x}') = \frac{K(\mathbf{x}, \mathbf{x}')}{\sqrt{K(\mathbf{x}, \mathbf{x}) K(\mathbf{x}', \mathbf{x}')}} \tag{1}$$

Another approach to optimizing the kernel function is *kernel alignment*. In other words, we would adjust the kernel parameters so as to make it, or its Gram matrix, more towards an ideal target kernel. For example, in a classification setting, we could use

$$K^*_{ij} = y_i y_j \tag{2}$$

as the Gram matrix of the target kernel. One argument for selecting this as the target is that if we set $\alpha_j = 1/n$ then

$$\sum_{j=1}^{n} \alpha_j y_j K^*_{ij} = y_i \tag{3}$$

and all the training examples are classified correctly with the same margin. (You could argue that another target should be used instead). Let's see how we can align the kernel towards this target. Suppose our parameterized kernel is a convex combination of kernels (e.g., constructed on the basis of different sources of input data)

$$K(\mathbf{x}, \mathbf{x}'; \theta) = \sum_{i=1}^{m} \theta_i K_i(\mathbf{x}, \mathbf{x}') \tag{4}$$

where $\theta_i \geq 0$ and $\sum_{i=1}^{m} \theta_i = 1$. These are the parameters we can adjust. We can now set θ so as to make the Gram matrix of this kernel, $K_{ij}(\theta)$, more similar to the Gram matrix of the target kernel, K^*_{ij}. To do this we view the Gram matrices as vectors and define their inner product in the usual way

$$\langle K^*, K_\theta \rangle = \sum_{i,j=1}^{n} K^*_{ij} K_{ij}(\theta) \tag{5}$$

The parameters θ can be now set so as to maximize the cosine of the angle between the Gram matrices:

$$\frac{\langle K^*, K_\theta \rangle}{\sqrt{\langle K^*, K^* \rangle \langle K_\theta, K_\theta \rangle}} \tag{6}$$

Model (kernel) selection

Optimizing the kernel in a parameterized form involved little consideration for the complexity of the set of classifiers we are fitting to finite data. It therefore did not address the problem of *over-fitting* or fitting too complex a model to too few data points. In many cases it makes sense to explicitly cast the problem of selecting a kernel as a *model selection* problem.

By choosing a kernel we specify the feature vectors on the basis of which linear predictions are made. Each model[1] (class) refers to a set of linear functions (classifiers) based on the chosen feature representation. In many cases the models are nested in the sense that the more "complex" model contains the "simpler" one. Consider, for example, solving a classification problem with either

$$K_1(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T \mathbf{x}') \text{ or} \tag{7}$$

$$K_2(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T \mathbf{x}')^2 \tag{8}$$

[1] In statistics, a model is a family/set of distributions or a family/set of linear separators.

Classifiers making use of the quadratic polynomial kernel can in principle reproduce the classifiers based on the linear kernel. As a model, i.e., as a set of linear classifiers based on the quadratic kernel, it therefore contains the simpler linear one. We can state this a bit more formally in terms of discriminant functions. For example, based on the linear kernel K_1, our discriminant functions are of the form

$$f_1(\mathbf{x}; \theta, \theta_0) \quad = \quad \theta^T \phi^{(1)}(\mathbf{x}) + \theta_0 \tag{9}$$

where $\phi^{(1)}(\mathbf{x})$ is the feature representation corresponding to K_1 such that $K_1(\mathbf{x}, \mathbf{x}') = \phi^{(1)}(\mathbf{x})^T \phi^{(1)}(\mathbf{x}')$. By varying the parameters θ and θ_0 we can generate the set of possible discriminant functions corresponding to this kernel:

$$\mathcal{F}_1 = \{ f_1(\cdot; \theta, \theta_0) : \theta \in \mathcal{R}^{d_1}, \theta_0 \in \mathcal{R} \} \tag{10}$$

$\mathcal{F}_2$ is defined analogously for the quadratic kernel. The fact that the two models are nested means that $\mathcal{F}_1 \subseteq \mathcal{F}_2$. For purposes of classification, we wouldn't actually have to assert that the families of discriminant functions are nested, only that the discriminant functions in $\mathcal{F}_2$ can produce the signs of those in $\mathcal{F}_1$.

The formal problem for us to solve is then to select a kernel K_i from a set of possible kernels $K_1, K_2, \ldots$, where the models associated with the kernels are nested $\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \ldots$. This is a model selection problem in a standard nested form.

From here on we will be referring to discriminant functions rather than kernels so as to emphasize the point that the discussion applies to other types of classifiers as well.

Model selection preliminaries

Before getting into specific selection criteria let's understand a bit better what exactly we are doing here. Recall that our goal is to accurately classify new test examples. Model selection is intended to facilitate this process. In other words, we switch from one model (kernel) to another so as to generalize better. The model we select will define how we will respond to any training data, i.e., which classifier we choose to make predictions on new examples. Model selection cannot therefore be decoupled from how we find the "best fitting" classifier from a given model. After all, it is that best fitting classifier that will determine how well we generalize.

Let $S_n = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ denote a training set of n examples and labels. If we chose model $\mathcal{F}_i$ then we would find the best fitting discriminant function $\hat{f}_i \in \mathcal{F}_i$ by minimizing

$$J(\theta, \theta_0) = \sum_{t=1}^{n} \text{Loss}\Big(y_t, f(\mathbf{x}_t; \theta, \theta_0) \Big) + \lambda_n \|\theta\|^2 \tag{11}$$

where the loss could be the hinge loss (SVM), logistic, or other. The regularization parameter λ_n would in general depend on the number of training examples. We are interested in how the classifier $\hat{f}_i(\mathbf{x}) = f(\mathbf{x}; \hat{\theta}, \hat{\theta}_0)$ resulting from our estimation procedure generalizes to new examples.

Each parameter setting (θ, θ_0), i.e., each discriminant function in our set, has an associated expected loss or *risk*

$$R(\theta, \theta_0) = E_{(\mathbf{x},y)\sim P}\left\{ \text{Loss}^*\Big(y, f(\mathbf{x}; \theta, \theta_0)\Big) \right\} \tag{12}$$

where the new test example and label, $(\mathbf{x}, y)$, is sampled from an underlying distribution P which is typically unknown to us. This is the generalization error we would like to minimize. Note that we have used $\text{Loss}^*(\cdot, \cdot)$ above rather than the loss used in training. These need not be the same and often they are not. For example, our goal may be to minimize classification error so that $\text{Loss}^*(y, f(\mathbf{x})) = 1 - \delta(y, \text{sign}(f(\mathbf{x})))$, i.e., the zero-one loss. We could still estimate the SVM classifier from the training set in the usual way, optimizing the hinge loss. The hinge loss can be viewed as a convex surrogate for the zero-one loss and it behaves much better in terms of the resulting optimization problem we have to solve during training (quadratic rather than integer programming problem).

The quantity of interest to us is the generalization error $R(\hat{\theta}, \hat{\theta}_0)$, or $R(\hat{f}_i)$ for short, corresponding to the classifier or discriminant function we would choose from $\mathcal{F}_i$ in response to the training data S_n. Ideally, we would then select the model $\mathcal{F}_i$ that leads to the smallest generalization error, minimizing

$$R(\hat{f}_i) = E_{(\mathbf{x},y)\sim P}\Big\{ \text{Loss}^*\Big(y, \hat{f}_i(\mathbf{x})\Big) \Big\} \tag{13}$$

Note that the risk $R(\hat{f}_i)$ is still a random variable as $\hat{f}_i$ depends on the training data that we assume was also sampled from the same underlying distribution P. If the training data were sampled from a different distribution, how could we expect to generalize? Actually, the only thing we really need is that the relationship between the labels and examples is the same for the training and test samples, along with some guarantee that the training examples cover the areas of input space that we will be tested on. In theoretical analysis it is nevertheless much more convenient to assume that the distributions are the same.

Now, we clearly do not have access to the underlying distribution and therefore cannot evaluate $R(\hat{f}_i)$. In fact, the whole model selection problem would go away if had access to the underlying distribution $P(\mathbf{x}, y)$. To classify new instances, we would simply forget about the training set and use the minimum probability of error classifier $\hat{y}(\mathbf{x}) = \arg\max_y P(y|\mathbf{x})$ (see the appendix). No classifier could lead to a lower probability of error. Our task is

much more difficult since we have to select $\hat{f}_i \in \mathcal{F}_i$ as well as the model $\mathcal{F}_i$ on the basis of the training data alone, without access to P.

Let's try to understand first intuitively what the model selection criterion has to be able to do. To make this a bit more concrete, consider just choosing between $\mathcal{F}_1$ and $\mathcal{F}_2$ corresponding to linear or quadratic feature vectors. Since the models are nested, $\mathcal{F}_1 \subseteq \mathcal{F}_2$, we can always achieve lower classification error on the training set by adopting $\mathcal{F}_2$. This is regardless of whether the true underlying model is linear. So, by choosing $\mathcal{F}_2$, we may be *over-fitting*. If the true relationship between the labels and examples were linear (the minimum probability of error classifier is linear), then the quadratic nature of the resulting decision boundary would simply be due to noise and couldn't generalize very well. So we should be able to see an increasing gap between the training and test errors as a function of the model complexity as in Figure 1 below. Clearly, all things being equal, we should select $\mathcal{F}_1$ as it is a simpler model. The real question is how to balance the "complexity" of the model, some measure of size or power of $\mathcal{F}_i$, against their fit to the training data. There are a number of answers to this question depending on your perspective. We will briefly go over a few possibilities and return to them later on.

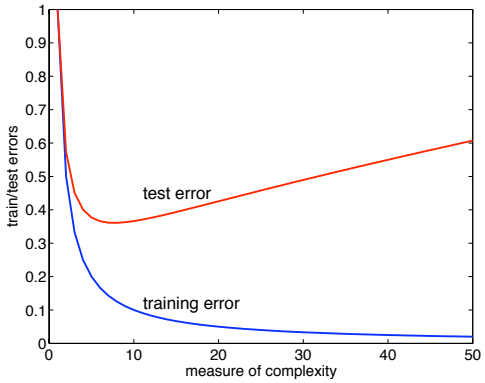


Figure 1: Training and test errors as a function of model order (e.g., degree of polynomial kernel).

Model selection criteria: structural risk minimization

One approach to model selection is to try to directly relate the (expected) risk $R(\hat{f}_i)$

$$R(\hat{f}_i) \quad = \quad E_{(\mathbf{x},y)\sim P}\Big\{ \text{Loss}^*\Big(y, \hat{f}_i(\mathbf{x})\Big) \Big\} \tag{14}$$

that we would like to have and the *empirical risk* $R_n(\hat{f}_i)$

$$R_n(\hat{f}_i) \quad = \quad \frac{1}{n}\sum_{t=1}^{n} \text{Loss}^*\Big(y_t, \hat{f}_i(\mathbf{x}_t)\Big) \tag{15}$$

that we can compute. If we can do this, then we have a partial access to $R(\hat{f}_i)$ through its empirical counterpart $R_n(\hat{f}_i)$. Note that the empirical risk here is computed on the basis of the available training set $S_n = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ and $\text{Loss}^*(\cdot, \cdot)$ rather than say the hinge loss. For our purposes here, $\hat{f}_i \in \mathcal{F}_i$ could be any estimate derived from the training set that approximately tries to minimizing the empirical risk.

We are interested in quantifying how much $R(\hat{f}_i)$ can deviate from $R_n(\hat{f}_i)$. The larger the deviation the less representative the training error is about the generalization error. This happens with more complex models $\mathcal{F}_i$. Indeed, we aim to show that

$$R(\hat{f}_i) \leq R_n(\hat{f}_i) + C(n, \mathcal{F}_i, \delta) \tag{16}$$

where the *complexity penalty* $C(n, \mathcal{F}_i)$ only depends on the model $\mathcal{F}_i$, the number of training instances, and a parameter δ. The peanalty does *not* depend on the actual training data. We will discuss the parameter δ below in more detail. For now, it suffices to say that $1 - \delta$ specifies the probability that the bound holds. We can only give a probabilistic guarantee in this sense since the empirical risk (training error) is a random quantity that depends on the specific instantiation of the data.

For nested models, $\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \ldots$, the penalty is necessarily an increasing function of i, the model order (e.g., the degree of polynomial kernel). Moreover, the penalty should go down as a function n. In other words, the more data we have, the more complex models we expect to be able to fit and still have the training error close to the generalization error.

The type of result in Eq.(16) gives us an *upper bound guarantee of generalization error*. We can then select the model with the best guarantee, i.e., the one with the lowest bound. Figure 2 shows how we would expect the upper bound to behave as a function of increasingly complex models in our nested "hierarchy" of models.

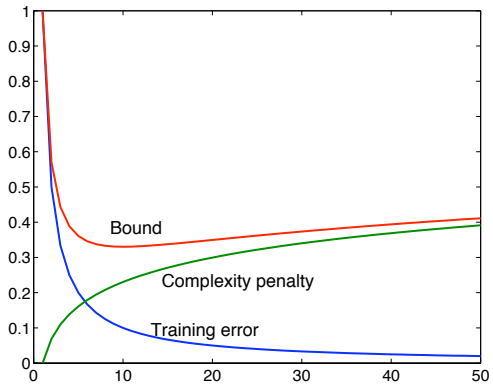


Figure 2: Bound on the generalization error as a function of model order (e.g., degree of polynomial kernel).

Lecture topics: model selection criteria

- Structural risk minimization, example derivation
- Bayesian score, Bayesian Information Criterion (BIC)

Model selection criteria: structural risk minimization

One perspective to model selection is to find the model (set of discriminant functions) that has the best *guarantee of generalization*. To obtain such guarantees we have to relate the *empirical risk* $R_n(\hat{f}_i)$

$$R_n(\hat{f}_i) \quad = \quad \frac{1}{n}\sum_{t=1}^{n} \text{Loss}^*\Big(y_t, \hat{f}_i(\mathbf{x}_t)\Big) \tag{1}$$

that we can compute to the (expected) risk $R(\hat{f}_i)$

$$R(\hat{f}_i) \quad = \quad E_{(\mathbf{x},y)\sim P}\Big\{ \text{Loss}^*\Big(y, \hat{f}_i(\mathbf{x})\Big) \Big\} \tag{2}$$

that we would like to have. In fact, we would like to keep these somewhat close so that the empirical risk (training error) still reflects how well the method will generalize. The empirical risk is computed on the basis of the available training set $S_n = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ and the loss function $\text{Loss}^*(\cdot, \cdot)$ rather than say the hinge loss. For our purposes $\hat{f}_i \in \mathcal{F}_i$ could be any estimate derived from the training set that approximately tries to minimizing the empirical risk. In our analysis we will assume that $\text{Loss}^*(\cdot, \cdot)$ is the zero-one loss (classification error).

We'd like to quantify how much $R(\hat{f}_i)$ can deviate from $R_n(\hat{f}_i)$. The more powerful our set of classifiers is the more we would expect them to deviate from one another. In other words, the more choices we have in terms of discriminant functions, the less representative the training error of the minimizing classifier is about its generalization error. So, our goal is to show that

$$R(\hat{f}_i) \leq R_n(\hat{f}_i) + C(n, \mathcal{F}_i, \delta) \tag{3}$$

where the *complexity penalty* $C(n, \mathcal{F}_i)$ only depends on the model $\mathcal{F}_i$, the number of training instances, and a parameter δ. The peanalty does *not* depend on the actual training data. We will discuss the parameter δ below in more detail. For now, it suffices to say that $1 - \delta$

specifies the probability that the bound holds. We can only give a probabilistic guarantee in this sense since the empirical risk (training error) is a random quantity that depends on the specific instantiation of the data.

For nested models, $\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \ldots$, the penalty is necessarily an increasing function of i, the model order (e.g., the degree of polynomial kernel). Moreover, the penalty should go down as a function n. In other words, the more data we have, the more complex models we expect to be able to fit and still have the training error close to the generalization error.

The type of result in Eq.(3) gives us an *upper bound guarantee of generalization error*. We can then select the model with the best guarantee, i.e., the one with the lowest bound. Figure 1 shows how we would expect the upper bound to behave as a function of increasingly complex models in our nested "hierarchy" of models.

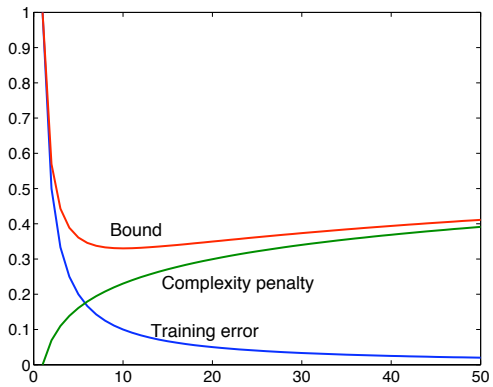


Figure 1: Bound on the generalization error as a function of model order (e.g., degree of polynomial kernel).

Let's derive a result of this type in the simple context where $\mathcal{F}_i$ only contains a finite number of classifiers $|\mathcal{F}_i| < \infty$. We will get to the general theory later on but this simple setting is helpful in understanding how such results come about. To avoid the question of how exactly we estimate $\hat{f}_i$, we will require a stronger guarantee: the bound should hold for *all* the classifiers in our set. Specifically, we try to find a tight upper bound on

$$P\left(\max_{f \in \mathcal{F}_i} |R(f) - R_n(f)| > \epsilon\right) \leq \delta \tag{4}$$

This is the probability that at least one classifier in our set deviates by more than ϵ from its training error. The probability is taken over the choice of the training data. So, if we

used ϵ to claim that

$$R(f) \leq R_n(f) + \epsilon \quad \text{for all } f \in \mathcal{F}_i \tag{5}$$

then this expression would fail with probability

$$\delta = P\left(\max_{f \in \mathcal{F}_i} |R(f) - R_n(f)| > \epsilon \right) \tag{6}$$

or, put another way, it would hold with probability $1 - \delta$ over the choice of the training data. If we fix δ, then the smallest $\epsilon = \epsilon(n, \mathcal{F}_i, \delta)$ that satisfies Eq.(6) is the complexity penalty we are after. Note that since the expression holds for all $f \in \mathcal{F}_i$ it necessarily also holds for $\hat{f}_i$.

In most cases we cannot compute δ exactly from Eq.(6) but we can derive an upper bound. This upper bound will lead to a larger than necessary complexity penalty but at least we will get a closed form expression (the utility of the model selection criterion will indeed depend on how tight a bound we can obtain). We will proceed as follows:

$$\begin{aligned} P\left(\max_{f \in \mathcal{F}_i} |R(f) - R_n(f)| > \epsilon \right) &= P\left(\exists f \in \mathcal{F}_i \text{ s.t. } |R(f) - R_n(f)| > \epsilon \right) & (7) \\ &\leq \sum_{f \in \mathcal{F}_i} P\left(|R(f) - R_n(f)| > \epsilon \right) & (8) \end{aligned}$$

where we have used the *union bound* $P(A_1 \cup A_2 \cup \ldots) \leq P(A_1) + P(A_2) + \ldots$ for any set of events $A_1, A_2, \ldots$ (not necessarily disjoint). In other words, we bound the probability that there are functions in our set with larger than ϵ deviation by a sum that each function individually has more than ϵ deviation between training and generalization errors.

Now, the discriminant function is fixed in any individual term in the sum

$$P\left(|R(f) - R_n(f)| > \epsilon \right) \tag{9}$$

It won't change as a function of the training data. We can then associate with each *i.i.d.* training sample $(\mathbf{x}_t, y_t)$, an indicator $s_t \in \{0, 1\}$ of whether the sample disagrees with f: $s_t = 1$ iff $y_t f(\mathbf{x}_t) \leq 0$. The empirical error $R_n(f)$ is therefore just an average of independent random variables (indicators) s_t:

$$R_n(f) = \frac{1}{n} \sum_{t=1}^{n} s_t \tag{10}$$

What is the expected value of each s_t when the expectation is taken over the choice of the training data? It's just $R(f)$, the expected risk. So, we can rewrite

$$P\left(|R(f) - R_n(f)| > \epsilon\right) \tag{11}$$

as

$$P\left(|q - \frac{1}{n}\sum_{t=1}^{n} s_t| > \epsilon\right) \tag{12}$$

where q equals $R(f)$ and the probability is now over n independent binary random variables $s_1, \ldots, s_n$ for which $P(s_t = 1) = q$. There are now standard results for evaluating a bound on how much an average of binary random variables deviates from its expectation (Hoeffding's inequality):

$$P\left(|q - \frac{1}{n}\sum_{t=1}^{n} s_t| > \epsilon\right) \leq 2\exp(-2n\epsilon^2) \tag{13}$$

Note that the bound does not depend on q (or $R(f)$) and therefore not on which f we chose. Using this result in Eq.(8), gives

$$P\left(\max_{f\in\mathcal{F}_i} |R(f) - R_n(f)| > \epsilon\right) \quad \leq \quad 2|\mathcal{F}_i|\exp(-2n\epsilon^2) = \delta \tag{14}$$

The last equality relates δ, $|\mathcal{F}_i|$, n, and ϵ, as desired. By solving for ϵ we get

$$\epsilon = \epsilon(n, \mathcal{F}_i, \delta) = \sqrt{\frac{\log|\mathcal{F}_i| + \log(2/\delta)}{2n}} \tag{15}$$

This is the complexity penalty we were after in this simple case with only a finite number of classifiers in our set.

We have now showed that with probability at least $1-\delta$ over the choice of the training set,

$$R(f) \leq R_n(f) + \sqrt{\frac{\log|\mathcal{F}_i| + \log(2/\delta)}{2n}}, \quad \text{uniformly for all } f \in \mathcal{F}_i \tag{16}$$

So, for model selection, we would then estimate $\hat{f}_i \in \mathcal{F}_i$ for each model, plug the resulting $\hat{f}_i$ and $|\mathcal{F}_i|$ on the right hand side of the above equation, and choose the model with the lowest bound. n and δ would be the same for all models under consideration.

As an example of another way of using the result, suppose we set $\delta = 0.05$ and would like any classifier that achieves zero training error to have at most 10% generalization error. Let's solve for the number of training examples we would need for such a guarantee within model $\mathcal{F}_i$. We want

$$R(f) \leq 0 + \sqrt{\frac{\log |\mathcal{F}_i| + \log(2/0.05)}{2n}} \leq 0.10 \tag{17}$$

Solving for n gives

$$n = \frac{\log |\mathcal{F}_i| + \log(2/0.05)}{2(0.10)^2} \tag{18}$$

training examples.

Model selection criteria: Bayesian score, Bayesian information criterion

It is perhaps the easiest to explain the Bayesian score with an example. We will start by providing a Bayesian analysis of a simple linear regression problem. So, suppose our model $\mathcal{F}$ takes a $d-$dimensional input $\mathbf{x}$ and maps it to a real valued output y (a distribution over y) according to:

$$P(y|\mathbf{x}, \theta, \sigma^2) = N(y;\, \theta^T\mathbf{x}, \sigma^2) \tag{19}$$

where $N(y; \theta^T\mathbf{x}, \sigma^2)$ is a normal distribution with mean $\theta^T\mathbf{x}$ and variance σ^2. To keep our calculations simpler, we will keep σ^2 fixed and only try to estimate θ. Now, given any set of observed data $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, we can define the likelihood function

$$L(D; \theta) = \prod_{t=1}^{n} N(y_t;\, \theta^T\mathbf{x}_t,\, \sigma^2) = \prod_{t=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_t - \theta^T\mathbf{x}_t)^2 \right) \tag{20}$$

We have previously used only the maximizing parameters $\hat{\theta}$ as estimates of the underlying parameter value (if any). In Bayesian analysis we are no longer satisfied with selecting a single linear regression function but would like to keep all of them, just weighted by their ability to explain the data, i.e., weighted by the corresponding likelihood $L(D; \theta)$. From this perspective, our knowledge about the parameter θ *after seeing the data* is defined by the *posterior distribution* $P(\theta|D)$ proportional to the likelihood

$$P(\theta|D) \propto L(D; \theta) \tag{21}$$

In many cases we cannot normalize this distribution, however. Suppose, as an extreme example, that we have no data. The likelihood function in this case is just one for all the parameter values. As a result the "posterior" after seeing no data is not well defined as a distribution (we cannot normalize the distribution by $\int 1\, d\theta = \infty$). To correct this problem it is advantageous to also put our prior belief about the parameter values in a form of a distribution, the *prior distribution* $P(\theta)$. This distribution captures what we believe about the parameter values before seeing any data. Similarly to the regularization penalty, we will typically choose the prior to prefer small parameter values, e.g.,

$$P(\theta) = N(\theta;\, 0,\, \sigma_p^2 \cdot I) \tag{22}$$

which is a zero mean spherical Gaussian (same variance in all directions). The smaller σ_p^2 is, the smaller values of θ we prefer prior to seeing the data. The posterior distribution, now well-defined as a distribution regardless of how much data we see, is proportional to the prior distribution $P(\theta)$ times the likelihood:

$$P(\theta|D) \propto L(D;\theta)P(\theta) \tag{23}$$

The normalization constant for the posterior, also known as the *marginal likelihood*, is given by

$$P(D|\mathcal{F}) = \int L(D;\theta)P(\theta)d\theta \tag{24}$$

and depends on the model $\mathcal{F}$ and the data but not specific parameter values. In our regression context, we can actually evaluate this marginal likelihood in closed form:

$$\log P(D|\mathcal{F}) = -\frac{n}{2}\log(2\pi\sigma^2) + \frac{d}{2}\log\lambda - \frac{1}{2}\log|\mathbf{X}^T\mathbf{X} + \lambda I| \tag{25}$$

$$-\frac{1}{2\sigma^2}\left(\|\mathbf{y}\|^2 - \mathbf{y}^T\mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda I)^{-1}\mathbf{X}^T\mathbf{y}\right) \tag{26}$$

where $\lambda = \sigma^2/\sigma_p^2$ (ratio of noise to prior variance), $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n]^T$, and $\mathbf{y} = [y_1, \ldots, y_n]^T$. These definitions are identical to the regularized least squares regression discussed earlier.

The posterior distribution over the parameters is simply normalized by the marginal likelihood:

$$P(\theta|D) = \frac{L(D;\theta)P(\theta)}{P(D|\mathcal{F})} \tag{27}$$

In our context the posterior is also Gaussian $P(\theta|D) = N(\theta; \mu, \Sigma)$ with mean μ and covariance Σ given by

$$\mu = (\mathbf{X}^T\mathbf{X} + \lambda I)^{-1}\mathbf{X}^T\mathbf{y} \quad (28)$$

$$\Sigma = \sigma^2(\mathbf{X}^T\mathbf{X} + \lambda I)^{-1} \quad (29)$$

Note that the posterior mean of the parameters is exactly the parameter estimate we derived earlier using penalized log-likelihood with the same prior. This is not an accident when all the distributions involved are indeed Gaussians. It is also worth pointing out that $P(\theta|D)$ is *very* different from the normal distribution over $\hat{\theta}$ we derived earlier when assuming that the responses $\mathbf{y}$ came from a linear model of the same type. We have made no such assumption here and the distribution $P(\theta|D)$ is defined on the basis of the single observed $\mathbf{y}$.

In Bayesian analysis the prediction of y in response to a new $\mathbf{x}$ would be given by weighting predictions based on individual θ's by the posterior distribution:

$$P(y|\mathbf{x}, D) = \int P(y|\mathbf{x}, \theta)P(\theta|D)d\theta \quad (30)$$

So what is the model selection problem in this context? A true Bayesian would refrain from selecting a single model but include all of them in proportion to their ability to explain the data (just as with parameters). We will not go that far, however, but instead try to select different regression models, specified by different feature mappings $\mathbf{x} \to \phi(\mathbf{x})$. Let's consider then two regression models specified by linear $\phi^{(1)}(\mathbf{x})$ and quadratic $\phi^{(2)}(\mathbf{x})$ feature mappings. The models we compare are therefore

$$\mathcal{F}_1 : \quad P(y|\mathbf{x}, \theta, \sigma^2) = N(y; \theta^T\phi^{(1)}(\mathbf{x}), \sigma^2), \quad \theta \in \mathcal{R}^{d_1}, \quad P(\theta|\mathcal{F}_1) \quad (31)$$

$$\mathcal{F}_2 : \quad P(y|\mathbf{x}, \theta, \sigma^2) = N(y; \theta^T\phi^{(2)}(\mathbf{x}), \sigma^2), \quad \theta \in \mathcal{R}^{d_2}, \quad P(\theta|\mathcal{F}_2) \quad (32)$$

Note that θ is of different dimension in the two models and thus the prior distributions over the parameters, $P(\theta|\mathcal{F}_1)$ and $P(\theta|\mathcal{F}_2)$, will have to be different. You might be wondering that since we are including the specification of the prior distribution as part of the model, the result will depend on how we selected the priors. Indeed, but not strongly so. This dependence on the prior is both an advantage and a disadvantage from the model selection point of view. We will discuss this further later on.

So, how do we select between the two competing models? We simply select the one whose marginal likelihood (Bayesian score[1]) is larger. In other words, after seeing data D we

[1]The definition of the Bayesian score often includes a prior over the models as well, e.g., how much we

would select model $\mathcal{F}_1$ if

$$P(D|\mathcal{F}_1) > P(D|\mathcal{F}_2) \tag{33}$$

Model selection criteria: Bayesian information criterion

Bayesian Information Criterion or BIC for short is an asymptotic approximation to the Bayesian score. It is frequently used for its simplicity. The criterion is simply

$$BIC = l(D; \hat{\theta}) - \frac{d}{2}\log(n) \tag{34}$$

where $l(D;\theta)$ is the log-likelihood of the data, $\hat{\theta}$ is the maximum likelihood estimate of the parameters, and d is the number of independent parameters in the model; n is the number of training examples as before. BIC is what the Bayesian score will converge to in the limit of large n. The Bayesian score is typically difficult to evaluate in practice and BIC serves as a simple tractable alternative. Similarly to the Bayesian score (marginal likelihood), we would select the model with the largest BIC score.

would prefer the simpler model before seeing any data. We have no reason to prefer one over another and therefore has used the same prior probability for both. As a result, the selection is carried out entirely on the basis of the marginal likelihood.

Lecture topics: model selection criteria

- Minimum description length (MDL)
- Feature (subset) selection

Model selection criteria: Minimum description length (MDL)

The minimum description length criterion (MDL) turns the model selection problem into a communication problem. The basic idea is that the best model should lead to the best way of compressing the available data. This view equates learning with the ability to compress data. This is indeed a sensible view since learning has to do with the ability to make predictions and effective compression is precisely based on such ability. So, the more we have learned, the better we can compress the observed data.

In a classification context the communication problem is defined as follows. We have a sender and a receiver. The sender has access to the training data, both examples $\mathbf{x}_1, \ldots, \mathbf{x}_n$ and labels $y_1, \ldots, y_n$, and needs to communicate the labels to the receiver. The receiver already has the examples but not the labels.

Any classifier that perfectly classifies the training examples would permit the receiver to reproduce the labels for the training examples. But the sender (say we) would have to communicate the classifier to the receiver before they can run it on the training examples. It is this part that pertains to the complexity of the set of classifiers we are fitting to the data. The more choices we have, the more bits it takes to communicate the specific choice that perfectly classified the training examples. We can also consider classifiers that are not perfectly accurate on the training examples by communicating the errors that they make. As a result, the communication cost, the number of bits we have to send to the receiver, depends on two things: the cost of errors on the training examples (description length of the data given the model), and the cost of describing the selected classifier (description length of the model), both measured in bits.

Simple two-part MDL. Let's start by defining the basic concepts in a simpler context where we just have a sequence of labels. You could view the examples $\mathbf{x}_i$ in this case just specifying the indexes of the corresponding labels. Consider the following sequence of mostly 1's:

$$\overbrace{1 \quad 1 \quad 1 \quad -1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad -1 \quad 1 \ldots}^{n \text{ labels}} \tag{1}$$

If we assume that each label in the sequence is an independent sample from the same distribution, then each $y_t \in \{-1, 1\}$ is a sample from

$$P(y_t|\theta) = \theta^{\delta(y_t,1)}(1-\theta)^{\delta(y_t,-1)} \tag{2}$$

for some θ. Here $\delta(y_1, 1)$ is an indicator such that $\delta(y_t, 1) = 1$ if $y_t = 1$ and zero otherwise. In our model $P(y_t = 1|\theta) = \theta$. We don't necessarily know that this is the correct model for the data. But whatever model (here a set of distributions) we choose, we can evaluate the resulting communication cost.

Now, the question is: given this model for the data, how many bits do we need to send the observed sequence of n labels to the receiver? This can be answered generally. Any distribution over the data can be used as a basis for a coding sceme. Moreover, the optimal coding scheme, assuming the data really did follow our distribution, would require $-\log_2 P(y_1, \ldots, y_n|\theta)$ bits (base two logarithm) or $-\log P(y_1, \ldots, y_n|\theta)$ nats (natural logarithm). We will use the natural logarithm for consistency with other material. You can always convert the answer to bits by multiplying nats by $\log(2)$.

Since our model assumes that the labels are independent of each other, we get

$$-\log P(y_1, \ldots, y_n|\theta) = \sum_{t=1}^{n} -\log P(y_t|\theta) \tag{3}$$

It is now tempting to minimize this encoding cost with respect to θ (maximizing the log-likelihood):

$$\min_\theta \left\{ -\sum_{t=1}^{n} \log P(y_t|\theta) \right\} = -\sum_{t=1}^{n} \log P(y_t|\hat{\theta}) \tag{4}$$

$$= -l(y_1, \ldots, y_n; \hat{\theta}) \tag{5}$$

where $l(D; \theta)$ is again the log-likelihood of data D and $\hat{\theta}$ is the maximum likelihood setting of the parameters. This corresponds to finding the distribution in our model (one corresponding to $\hat{\theta}$) that minimizes the encoding length of the data. The problem is that the receiver needs to be able to *decode* what we send. This is only possible if they have access to the same distribution (be able to recreate the code we use). So the receiver would have to know $\hat{\theta}$ (in addition to the model we are considering but we will omit this part, assuming that the receiver already knows the type of distributions we are have).

The solution is to encode the choice of $\hat{\theta}$ as well. This way the receiver can recreate our steps to decode the bits we have sent. So how do we encode $\hat{\theta}$? It is a continuous parameter

and it might seem that we need an infinite number of bits to encode a real number. But not all θ can arise as ML estimates of the parameters. In our case, the ML estimates of the parameters θ have the following form

$$\hat{\theta} = \frac{\hat{n}(1)}{n} \tag{6}$$

where $\hat{n}(1)$ is the number of 1's in the observed sequence. There are thus only $n+1$ possible values of $\hat{\theta}$ corresponding to $\hat{n}(1) \in \{0, \ldots, n\}$. Since the receiver already knows that we are encoding a sequence of length n, then we can just send θ by defining a distribution over its possible discrete values, say $Q(\theta = k/n) = \frac{1}{n+1}$ for simplicity (i.e., a uniform distribution over possible discrete values). As before the cost in bits (nats) is $-\log Q(\hat{\theta}) = \log(n+1)$.

The total cost we have to send to the receiver is therefore

$$\text{DL} = \overbrace{-l(y_1, \ldots, y_n; \hat{\theta})}^{\text{DL of data given model}} + \overbrace{\log(n+1)}^{\text{DL of model}} \tag{7}$$

This is known as *the description length* of the sequence. The minimum description length criterion simple finds the model that leads to the shortest description length. Asymptotically, the simple two-part MDL principle is essentially the same as BIC.

Universal coding, normalized maximum likelihood. The problem with the simple two part coding is that the description length we associate with the second part, the model, may seem a bit arbitrary. We can correct this by finding a distribution that is in some sense universally closest to just encoding the data with the best fitting distribution in our model:

$$-\log P_{NML}(y_1, \ldots, y_n) \approx \min_{\theta} -l(y_1, \ldots, y_n; \theta) = -\max_{\theta} l(y_1, \ldots, y_n; \theta) \tag{8}$$

In other words, we don't wish to define a prior over the parameters in our model so as to encode the parameter values. While it won't be possible to achieve the above approximate equality with equality since the right hand side, if exponentiated, wouldn't define a valid distribution (because of the ML fitting). The distribution that minimizes the maximum deviation in Eq.(8) is given by

$$P_{NML}(y_1, \ldots, y_n) = \frac{\exp\left(\max_\theta l(y_1, \ldots, y_n; \theta)\right)}{\sum_{y'_1, \ldots, y'_n} \exp\left(\max_\theta l(y'_1, \ldots, y'_n; \theta)\right)} \tag{9}$$

and is known as the *normalized maximum likelihood distribution*. Using this distribution

to encode the sequence leads to a minimax optimal coding length

$$-\log P_{NML}(y_1,\ldots,y_n) = -\max_\theta l(y_1,\ldots,y_n;\theta) + \overbrace{\log \sum_{y'_1,\ldots,y'_n} \exp\left(\max_\theta l(y'_1,\ldots,y'_n;\theta)\right)}^{\text{Parametric complexity of model}} \quad (10)$$

Note that the result is again in the same form, the negative log-likelihood with a ML parameter setting and a complexity penalty. The new parametric complexity penalty depends on the length of the sequence as well as the model we use. While the criterion is theoretically nice, it is hard to evaluate the parametric complexity penalty in practice.

Feature subset selection

Feature selection is a problem where we wish to identify components of feature vectors most useful for classification. For example, some components may be very noisy and therefore unreliable. Depending on how such features[1] would be treated in the classification method, it may be best not to include them. Here we assume that we have no prior knowledge of which features might be useful for solving the classification task and instead have to provide an automated solution. This is a type of model selection problem, where the models are identified with subsets of the features we choose to include (or the number of features we wish to include). Note that kernels do not solve nor avoid this problem. By taking an inner product between feature vectors that may contain many noisy or irrelevant features results in an unnecessarily varying kernel value.

Instead of selecting a subset of the features we could also try to weight them according to how useful they are. This is a related *feature weighting* problem and we will get to this later on. This is also the method typically used with kernels (cf. kernel optimization).

Let's address the feature selection problem a bit more concretely. Suppose our input features are $d-$dimensional binary vectors $\mathbf{x} = [x_1, ..., x_d]^d$ where $x_i \in \{-1, 1\}$ and labels are binary $y \in \{-1, 1\}$ as before. We will begin by using a particular type of probability model known as *Naive Bayes* model to solve the classification problem. This will help tie our MDL discussion to the feature selection problem. In the Naive Bayes model, we assume that the features are conditionally independent given the label so that

$$P(\mathbf{x}, y) = \left[\prod_{i=1}^{d} P(x_i|y)\right] P(y) \quad (11)$$

[1] We will refer to the components of the feature/input vectors as "features".

Put another way, in this model, the features are correlated only through the labels. We will contrast this model with a "null model" that assumes that none of the features are relevant for the label:

$$P_0(\mathbf{x}, y) = \left[\prod_{i=1}^{d} P(x_i)\right] P(y) \tag{12}$$

In other words, in this model we assume that the features and the label are all independent of each other.

The Naive Bayes model (as well as the null model) are *generative models* in the sense that they can generate all the data, not just the labels. These models are typically trained using the log-likelihood of all the data as the estimation criterion. Let us follow this here as well. We will discuss later on how this affects the results. So, we aim to maximize

$$\sum_{t=1}^{n} \log P(\mathbf{x}_t, y_t) = \sum_{t=1}^{n} \left[\sum_{i=1}^{d} \log P(x_{ti}|y_t) + \log P(y_t)\right] \tag{13}$$

$$= \sum_{i=1}^{d}\sum_{t=1}^{n} \log P(x_{ti}|y_t) + \sum_{t=1}^{n} \log P(y_t) \tag{14}$$

$$= \sum_{i=1}^{d} \left(\sum_{x_i, y} \hat{n}_{iy}(x_i, y) \log P(x_i|y)\right) + \sum_{y} \hat{n}_y(y) \log P(y_t) \tag{15}$$

where we have used counts $\hat{n}_{iy}(x_i, y)$ and $\hat{n}_y(y)$ to summarize the available data for the purpose of estimating the model parameters. $\hat{n}_{iy}(x_i, y)$ indicates the number of training instances that have value x_i for the i^{th} feature and y for the label. These are called *sufficient statistics* as they are the only things from the data that the model cares about (the only numbers computed from the data that are required to estimate the model parameters). The ML parameter estimates, values that maximize the above expression, are simply fractions of these counts:

$$\hat{P}(y) = \frac{\hat{n}_y(y)}{n} \tag{16}$$

$$\hat{P}(x_i|y) = \frac{\hat{n}_{iy}(x_i, y)}{\hat{n}_y(y)} \tag{17}$$

As a result, we also have $\hat{P}(x_i, y) = \hat{P}(x_i|y)\hat{P}(y) = \hat{n}_{iy}(x_i, y)/n$. If we plug these back into

the expression for the log-likelihood, we get

$$\begin{aligned}
\hat{l}(S_n) &= \sum_{t=1}^{n} \log \hat{P}(\mathbf{x}_t, y_t) && (18)\\
&= n \left[\sum_{i=1}^{d} \left(\sum_{x_i, y} \frac{\hat{n}_{iy}(x_i, y)}{n} \log \hat{P}(x_i|y) \right) + \sum_{y} \frac{\hat{n}_y(y)}{n} \log \hat{P}(y_t) \right] && (19)\\
&= n \left[\sum_{i=1}^{d} \overbrace{\sum_{x_i, y} \hat{P}(x_i, y) \log \hat{P}(x_i|y)}^{-\hat{H}(X_i|Y)} + \overbrace{\sum_{y} \hat{P}(y) \log \hat{P}(y_t)}^{-\hat{H}(Y)} \right] && (20)\\
&= n \left[-\sum_{i=1}^{d} \hat{H}(X_i|Y) - \hat{H}(Y) \right] && (21)
\end{aligned}$$

where $\hat{H}(Y)$ is the *Shannon entropy* (uncertainty) of y relative to distribution $\hat{P}(y)$ and $\hat{H}(X_i|Y)$ is the *conditional entropy* of X_i given Y. Entropy $H(Y)$ measures how many bits (here nats) we would need on average to encode a value y chosen at random from $\hat{P}(y)$. It is zero when y is deterministic and takes the largest value (one bit) when $\hat{P}(y) = 1/2$ in case of binary labels. Similarly, the conditional entropy $\hat{H}(X_i|Y)$ measures how uncertain we would be about X_i if we already knew the value for y and the values for these variables were sampled from $\hat{P}(x_i, y)$. If x_i is perfectly predictable from y according to $\hat{P}(x_i, y)$, then $\hat{H}(X_i|Y) = 0$.

We can perform a similar calculation for the null model and obtain

$$\hat{l}_0(S_n) = \sum_{t=1}^{n} \log \hat{P}_0(\mathbf{x}_t, y_t) = n \left[-\sum_{i=1}^{d} \hat{H}(X_i) - \hat{H}(Y) \right] \quad (22)$$

Note that the conditioning on y is gone since the features were assumed to be independent of the label. By taking a difference of these two, we can gauge how much we gain in terms of the resulting log-likelihood if we assume the features to depend on the label:

$$\hat{l}(S_n) - \hat{l}_0(S_n) = n \sum_{i=1}^{d} \Big(\hat{H}(X_i) - \hat{H}(X_i|Y) \Big) \quad (23)$$

The expression

$$\hat{I}(X_i; Y) = \hat{H}(X_i) - \hat{H}(X_i|Y) \quad (24)$$

is known as the *mutual information* between x_i and y, relative to distribution $\hat{P}(x_i, y)$. Mutual information is *non-negative* and *symmetric*. In other words,

$$\hat{I}(X_i; Y) = \hat{H}(X_i) - \hat{H}(X_i|Y) = \hat{H}(Y) - \hat{H}(Y|X_i) \tag{25}$$

To understand mutual information in more detail, please consult, e.g., Cover and Tomas listed on the course website.

So, as far as feature selection is concerned, our calculation so far suggests that we should select the features to include in the decreasing order of $\hat{I}(X_i; Y)$ (the higher the mutual information, the more we gain in terms of log-likelihood). It may seem a bit odd, however, that we can select the features individually, i.e., not consider them in specific combinations. This is due to the simple Naive Bayes assumption, and our use of the log-likelihood of all the data to derive the selection criterion. The mutual information criterion is nevertheless very common and it is useful to understand how it comes about. We will now turn to improving this a bit with MDL.

Lecture topics: model selection criteria

- Feature subset selection (cont'd)
  - information criterion
  - conditional log-likelihood and description length
  - regularization
- Combination of classifiers, boosting

Feature subset selection (cont'd)

We have already discussed how to carry out feature selection with the Naive Bayes model. This model is easy to specify and estimate when the input vectors $\mathbf{x}$ are discrete (here vectors with binary $\{-1, 1\}$ components). When we elect to use only a subset $\mathcal{J}$ of features for classification, our model can be written as

$$P(\mathbf{x}, y) = \left[\prod_{i \in J} P(x_i|y)\right] \left[\prod_{i \notin J} P(x_i)\right] P(y) \tag{1}$$

Note that the features not used for classification, indexed by $j \notin \mathcal{J}$, are assumed independent of the label. This way we still have a distribution over the original input vectors $\mathbf{x}$ and labels but assert that only some of the components of $\mathbf{x}$ are relevant for classification. The probabilities involved such as $P(x_i|y)$ are obtained directly from empirical counts involving x_i and y (see previous lecture).

The selection criterion we arrived at indicated that, as far as the log-likelihood of all the data is concerned, we should include features (replace $P(x_i)$ with $P(x_i|y)$ in the above model) in the decreasing order of

$$\hat{I}(X_i; Y) = \hat{H}(X_i) - \hat{H}(X_i|Y) \tag{2}$$

where the entropies $\hat{H}(X_i)$ and $\hat{H}(X_i|Y)$ are evaluated on the basis of the estimated probabilities for the Naive Bayes model. The disadvantage of this criterion is that the features are selected individually, i.e., without regard to how effective they might be in specific combinations with each other. This is a direct result of the Naive Bayes model as well as the fact that we opted to find features that maximally help increase the log-likelihood of all the data. This is clearly slightly different from trying to classify examples accurately.

For example, from the point of view of classification, it is not necessary to model the distribution over the feature vectors $\mathbf{x}$. All we care about is the conditional probability $P(y|\mathbf{x})$. Why should our estimation and the feature selection criterion then aim to increase the log-likelihood of generating the feature vectors as well? We will now turn to improving the feature selection part with MDL while still estimating the probabilities $P(x_i|y)$ and $P(y)$ in closed form as before.

Feature subset selection with MDL

Let's assume we have estimated the parameters in the Naive Bayes model, i.e., $\hat{P}(\mathbf{x}_i|y)$ and $\hat{P}(y)$, as before by maximizing the log-likelihood of all the data. Let's see what this model implies in terms of classification:

$$\hat{P}(y=1|\mathbf{x}) = \frac{\hat{P}(\mathbf{x}, y=1)}{\hat{P}(\mathbf{x}, y=1) + \hat{P}(\mathbf{x}, y=-1)} = \frac{1}{1 + \frac{\hat{P}(\mathbf{x}, y=-1)}{\hat{P}(\mathbf{x}, y=1)}} \tag{3}$$

$$= \frac{1}{1 + \exp(-\log \frac{\hat{P}(\mathbf{x}, y=1)}{\hat{P}(\mathbf{x}, y=-1)})} = \frac{1}{1 + \exp(-\hat{f}(\mathbf{x}))} \tag{4}$$

where

$$\hat{f}(\mathbf{x}) = \log \frac{\hat{P}(\mathbf{x}, y=1)}{\hat{P}(\mathbf{x}, y=-1)} \tag{5}$$

is the discriminant function arising from the Naive Bayes model. So, for example, when $\hat{f}(\mathbf{x}) > 0$, the logistic function implies that $\hat{P}(y=1|\mathbf{x}) > 0.5$ and we would classify the example as positive. If $\hat{f}(\mathbf{x})$ is a linear function of the inputs $\mathbf{x}$, then the form of the conditional probability from the Naive Bayes model would be exactly as in a logistic regression model. Let's see if this is indeed so.

$$\hat{f}(\mathbf{x}) = \log \frac{\hat{P}(\mathbf{x}, y=1)}{\hat{P}(\mathbf{x}, y=-1)} = \sum_{i \in \mathcal{J}} \log \frac{\hat{P}(x_i|y=1)}{\hat{P}(x_i|y=-1)} + \log \frac{\hat{P}(y=1)}{\hat{P}(y=-1)} \tag{6}$$

Note that only terms tried to the labels remain. Is this a linear function of the binary features x_i? Yes, it is. We can write each term as a linear function of x_i as follows:

$$\log \frac{\hat{P}(x_i|y=1)}{\hat{P}(x_i|y=-1)} = \delta(x_i, 1) \log \frac{\hat{P}(x_i=1|y=1)}{\hat{P}(x_i=1|y=-1)} + \delta(x_i, -1) \log \frac{\hat{P}(x_i=-1|y=1)}{\hat{P}(x_i=-1|y=-1)} \tag{7}$$

$$= \frac{x_i+1}{2} \log \frac{\hat{P}(x_i=1|y=1)}{\hat{P}(x_i=1|y=-1)} + \frac{1-x_i}{2} \log \frac{\hat{P}(x_i=-1|y=1)}{\hat{P}(x_i=-1|y=-1)} \tag{8}$$

By combining these terms we can write the discriminant function in the usual linear form

$$\hat{f}(\mathbf{x}) = \sum_{i \in \mathcal{J}} \hat{w}_i x_i + \hat{w}_0 \tag{9}$$

where the parameters $\hat{w}_i$ and $\hat{w}_0$ are functions of the Naive Bayes conditional probabilities. Note that while we ended up with a logistic regression model, the parameters of this conditional distribution, $\hat{w}_i$ and $\hat{w}_0$, have been estimated quite differently from the logistic model.

We can now evaluate the conditional probability $\hat{P}(y|\mathbf{x}, \mathcal{J})$ from the Naive Bayes model corresponding to any subset $\mathcal{J}$ of relevant features. Let's go back to the feature selection problem. The subset $\mathcal{J}$ should be optimized to maximize the conditional log-likelihood of labels given examples. Equivalently, we can *minimize* the description length

$$\text{DL-data}(\mathcal{J}) = \sum_{t=1}^{n} - \log \hat{P}(y_t|\mathbf{x}_t, \mathcal{J}) \tag{10}$$

with respect to $\mathcal{J}$. This is a criterion that evaluates how useful the features are for classification and it can be no longer reduced to evaluating features independently of others. This also means that the optimization problem for finding $\mathcal{J}$ is a difficult one. The simplest way of approximately minimizing this criterion would be to start with no features, then include the single best feature, followed by the second feature that works best with the first one, and so on. Note that in this simple setting the classifier parameters associated with different subsets of features are fixed by the Naive Bayes model; we only optimize over the subset of features.

The above sequential optimization of the criterion would yield features that are more useful for classification than ranking them by mutual information (the first feature to include would be the same, however). But we do not yet have a criterion for deciding how many features to include. In the MDL terminology, we need to describe the model as well. The more features we include the more bits we need to describe both the set and the parameters associated with using that set (the Naive Bayes conditional probabilities).

So, first we need to communicate the set (size and the elements). The (any) integer $|\mathcal{J}|$ can be communicated with the cost of

$$\log^* |\mathcal{J}| = \log |\mathcal{J}| + \log \log |\mathcal{J}| + \ldots \tag{11}$$

nats. The elements in the set, assuming (a priori) that they are drawn uniformly at random from d possible features, require

$$\log \left(\begin{array}{c} d \\ |\mathcal{J}| \end{array} \right) \tag{12}$$

nats. Finally, we need to communicate the Naive Bayes parameters associated with the elements in the feature set. $\hat{P}(y)$ can only take $n+1$ possible values and thus requires $\log(n+1)$ nats. Once we have sent this distribution, the receiver will have access to the counts $n_y(1)$ and $n_y(-1)$ in addition to n that they already knew. $\hat{P}(x_i|y=1)$ can only take $n_y(1)+1$ possible values with the cost of $\log(n_y(1)+1)$, and similarly for $\hat{P}(x_i|y=-1)$. Given that there are $|\mathcal{J}|$ of these, the total communication cost for the model is then

$$\text{DL-model}(|\mathcal{J}|) = \log^* |\mathcal{J}| + \log \left(\begin{array}{c} d \\ |\mathcal{J}| \end{array} \right) + \tag{13}$$

$$\log(n+1) + |\mathcal{J}| \Big(\log(n_y(1)+1) + \log(n_y(-1)+1) \Big) \tag{14}$$

Note that the model cost only depends on the size of the feature set, not the elements in the set. We would find the subset of features by minimizing

$$\text{DL}(\mathcal{J}) = \text{DL-data}(\mathcal{J}) + \text{DL-model}(|\mathcal{J}|) \tag{15}$$

sequentially or otherwise.

Feature selection via regularization

An alternative to explicitly searching for the right feature subset is to try to formulate the selection problem as a regularization problem. We will no longer use parameters from a Naive Bayes model but instead work directly with a logistic regression model

$$P(y|\mathbf{x}, \theta) = g\Big(\sum_{i=1}^{d} \theta_i x_i + \theta_0 \Big) \tag{16}$$

The typical regularization problem for estimating the parameters θ would be penalized conditional log-likelihood with the squared norm regularization

$$J(\theta; S_n) = \sum_{t=1}^{n} \log P(y_t|\mathbf{x}_t, \theta) - \lambda \sum_{i=1}^{d} \theta_i^2 \tag{17}$$

The squared norm regularization won't work for our purposes, however. The reason is that none of the parameters would be set exactly to zero as a result of solving the optimization problem (none of the features would be clearly selected or excluded). We will instead use

a *one-norm* regularization penalty:

$$J(\theta; S_n) = \overbrace{\sum_{t=1}^{n} \log P(y_t|\mathbf{x}_t, \theta)}^{l(\theta;S_n)} - \lambda \sum_{i=1}^{d} |\theta_i| \tag{18}$$

If we let $\lambda \to \infty$, then all θ_i (except for θ_0) would go to zero. On the other hand, when $\lambda = 0$, none of θ_i are zero. These are identical to the squared penalty. However, the intermediate values of λ produce more interesting results: some of θ_i will be set exactly to zero while this would never happen with the squared penalty. Let's try to understand why.

Suppose we fix all but a single parameter θ_k to their optimized values (zero or not) and view $l(\theta; S_n)$ as a function of this parameter. Suppose the optimal value of $\theta_k > 0$, then

$$\frac{d}{d\theta_k} l(\theta; S_n) - \lambda = 0 \tag{19}$$

at the optimal θ_k. In other words, the slope of the conditional log-likelihood function relative to this parameter has to be exactly λ. If parameter θ_k is irrelevant then the conditional log-likelihood is unlikely to be affected much by the parameter, keeping the slope very small. In this case, θ_k would be set exactly to zero. The larger the value of λ, the more of the parameters would end up going to zero. Another nice property is that if the optimum value of θ_k is zero, then we would get the same regression model whether or not the corresponding feature were included in the model to begin with. A good value of λ, and therefore a good number of features, can be found via cross-validation.

So why don't we get the same "zeroing" of the parameters with the squared penalty? Consider the same situation, fix all but θ_k. Then at the optimum

$$\frac{d}{d\theta_k} l(\theta; S_n) - 2\lambda\theta_k = 0 \tag{20}$$

However small the slope of the log-likelihood is at $\theta_k = 0$, we can move the parameter just above zero so as to satisfy the above optimality condition. There's no fixed lower bound on how useful the parameter has to be for it to take non-zero values.

Combining classifiers and boosting

We have so far discussed feature selection as a problem of finding a subset of features out of d possible ones. In many cases the possible feature set available to us may not even

be enumerable. As an example, consider real valued input vectors $\mathbf{x} = [x_1, \ldots, x_d]^d$, still $d-$dimensional. We can turn this real valued vector into a feature vector of binary $\{-1, 1\}$ features in a number of different ways. One possibility is to run simple classifiers on the input vector and collect the predicted class labels into a feature vector. One of the simplest classifiers is the *decision stump*:

$$h(\mathbf{x}; \theta) = \text{sign}\Big(s(x_k - \theta_0)\Big) \tag{21}$$

where $\theta = \{s, k, \theta_0\}$ and $s \in \{-1, 1\}$ specifies the label to predict on the positive side of $x_k - \theta_0$. In other words, we simply select a specific component and threshold its value. There are still an uncountable number of such classifiers, even based on the same input component since the threshold θ_0 is real valued. Our "feature selection" problem here is to find a finite set of such stumps.

In order to determine which stumps ("features") would be useful we have to decide how they will be exploited. There are many ways to do this. For example, we could run a linear classifier based on the resulting binary feature vectors

$$\phi(\mathbf{x}; \theta) = [h(\mathbf{x}; \theta_1), \ldots, h(\mathbf{x}; \theta_m)]^T \tag{22}$$

We will instead collect the outputs of the simple stumps into an *ensemble*:

$$h_m(\mathbf{x}) = \sum_{j=1}^{m} \alpha_j h(\mathbf{x}; \theta_j) \tag{23}$$

where $\alpha_j \geq 0$ and $\sum_{j=1}^m \alpha_j = 1$. We can view the ensemble as a voting combination. Given $\mathbf{x}$, each stump votes for a label and it has α_j votes. The ensemble then classifies the example according to which label received the most votes. Note that $h_m(\mathbf{x}) \in [-1, 1]$. $h_m(\mathbf{x}) = 1$ only if all the stumps agree that the label should be $y = 1$. The ensemble is a linear classifier based on $\phi(\mathbf{x}; \theta)$, for a fixed θ, but with constrained parameters. However, our goal to learn both which features to include, i.e., $h(\mathbf{x}; \theta_j)$'s, as well as how they are combined (the α's). This is a difficult problem solve.

We can combine any set of classifiers into an ensemble, not just stumps. For this reason, we will refer to the simple classifiers we are combining as *base learners* or *base classifiers* (also called weak learners or component classifiers). Note that the process of combining simple "weak" classifiers into one "strong" classifier is analogous to the use of kernels to go from a simple linear classifier to a non-linear classifier. The difference is that here we are *learning* a small number of highly *non-linear features* from the inputs rather than using a

fixed process of generating a large number of features from the inputs as in the polynomial kernel.

The ensembles can be useful even if generated through randomization. For example, we can generate random subsets of smaller training sets from the original one, and train a classifier, e.g., an SVM, based on each such set. The outputs of the SVM classifiers, trained with slightly different training sets, can be combined into an ensemble with uniform α's. This process, known as *bagging*, is a method of reducing the *variance* of the resulting classifier. The uniform weighting will not help with the *bias*.

Now, let's figure out how to train ensembles. We will need a loss function and there many possibilities, including the logistic loss. For simplicity, we will use the *exponential loss*:

$$\text{Loss}(y, h(\mathbf{x})) = \exp(-yh(\mathbf{x})) \tag{24}$$

The loss is small if the ensemble classifier agrees with the label y (the smaller the stronger the agreement). It is large if the ensemble strongly disagrees with the label. This is the basic loss function is in an ensemble method called *Boosting*.

The simplest way to optimize the ensemble is to do it in stages. In other words, we will first find a single stump (an ensemble of one), then add another while keeping the first one fixed, and so on, never retraining those already added into the ensemble. To facilitate this type of estimation, we will assume that $\alpha_j \geq 0$ but won't require that they will sum to one (we can always renormalize the votes after having trained the ensemble).

Suppose now that we have already added $m-1$ base learners into the ensemble and call this ensemble $h_{m-1}(\mathbf{x})$. This part will be fixed for the purpose of adding $\alpha_m h(\mathbf{x}; \theta_m)$. We can then try to minimize the training loss corresponding to the ensemble

$$h_m(\mathbf{x}) = \sum_{j=1}^{m-1} \hat{\alpha}_j h(\mathbf{x}; \hat{\theta}_j) + \alpha_m h(\mathbf{x}; \theta_m) \tag{25}$$

$$= h_{m-1}(\mathbf{x}) + \alpha_m h(\mathbf{x}; \theta_m) \tag{26}$$

To this end,

$$J(\alpha_m, \theta_m) = \sum_{t=1}^{n} \text{Loss}(y_t, h_m(\mathbf{x}_t)) \tag{27}$$

$$= \sum_{t=1}^{n} \exp\Big(- y_t h_{m-1}(\mathbf{x}_t) - y_t \alpha_m h(\mathbf{x}_t; \theta_m) \Big) \tag{28}$$

$$= \sum_{t=1}^{n} \overbrace{\exp\Big(- y_t h_{m-1}(\mathbf{x}_t) \Big)}^{W_{m-1}(t)} \exp\Big(- y_t \alpha_m h(\mathbf{x}_t; \theta_m) \Big) \tag{29}$$

$$= \sum_{t=1}^{n} W_{m-1}(t) \exp\Big(- y_t \alpha_m h(\mathbf{x}_t; \theta_m) \Big) \tag{30}$$

In other words, for the purpose of estimating the new base learner, all we need to know from the previous ensemble are the weights $W_{m-1}(t)$ associated with the training examples. These weights are exactly the losses of the $m-1$ ensemble on each of the training example. Thus, the new base learner will be "directed" towards examples that were misclassified by the $m-1$ ensemble $h_{m-1}(\mathbf{x})$.

The estimation problem that couples α_m and θ_m is still a bit difficult. We will simplify this further by figuring out how to estimate θ_m first and then decide the votes α_m that we should assign to the new base learner (i.e., how much we should rely on its predictions). But what is the criterion for θ_m independent of α_m? Consider as a thought experiment that we calculate for all possible θ_m the derivative

$$\frac{d}{d\alpha_m} J(\alpha_m, \theta_m)\Big|_{\alpha_m=0} = -\sum_{t=1}^{m} W_{m-1}(t) y_t h(\mathbf{x}_t; \theta_m) \tag{31}$$

This derivative tells us how much we can reduce the overall loss by increasing the vote (from zero) of the new base learner with parameters θ_m. This derivative is expected to be negative so that the training loss is decreased by adding the new base learner. It makes sense then to find a base learner $h(\mathbf{x}; \theta_m)$, parameters θ_m, so as to minimize this derivative (making it as negative as possible). Such a base learner would be expected to lead to a large reduction of the training loss. Once we have this $\hat{\theta}_m$ we can subsequently optimize the training loss $J(\alpha_m, \hat{\theta}_m)$ with respect to α_m for a fixed $\hat{\theta}_m$.

We have now essentially all the components to define the *Adaboost algorithm*. We will make one modification which is that the weights will be normalized to sum to one. This is

advantageous as they can become rather small in the course of adding the base learners. The normalization won't change the optimization of θ_m nor α_m. We denote the normalized weights with $\tilde{W}_{m-1}(t)$. The boosting algorithm is defined as

(0) Set $W_0(t) = 1/n$ for $t = 1, \ldots, n$.

(1) At stage m, find a base learner $h(\mathbf{x}; \hat{\theta}_m)$ that approximately minimizes

$$-\sum_{t=1}^{m} \tilde{W}_{m-1}(t) y_t h(\mathbf{x}_t; \theta_m) = 2\epsilon_m - 1 \tag{32}$$

where ϵ_m is the weighted classification error on the training examples, weighted by the normalized weights $\tilde{W}_{m-1}(t)$.

(2) Set

$$\hat{\alpha}_m = 0.5 \log \left(\frac{1 - \hat{\epsilon}_m}{\hat{\epsilon}_m} \right) \tag{33}$$

where $\hat{\epsilon}_m$ is the weighted error corresponding to $\hat{\theta}_m$ chosen in step (1). The value $\hat{\alpha}_m$ is the closed form solution for α_m that minimizes $J(\alpha_m, \hat{\theta}_m)$ for fixed $\hat{\theta}_m$.

(3) Update the weights on the training examples

$$\tilde{W}_m(t) = c_m \cdot \tilde{W}_{m-1}(t) \exp \left(-y_t \hat{\alpha}_m h(\mathbf{x}_t; \hat{\theta}_m) \right) \tag{34}$$

where c_m is the normalization constant to ensure that $\tilde{W}_m(t)$ sum to one. The new weights can be interpreted as normalized losses for the new ensemble $h_m(\mathbf{x}_t) = h_{m-1}(\mathbf{x}) + \hat{\alpha}_m h(\mathbf{x}; \hat{\theta}_m)$.

The Adaboost algorithm sequentially adds base learners to the ensemble so as to decrease the training loss.

Lecture topics:

- Boosting, margin, and gradient descent
- complexity of classifiers, generalization

Boosting

Last time we arrived at a boosting algorithm for sequentially creating an ensemble of base classifiers. Our base classifiers were *decision stumps* that are simple linear classifiers relying on a single component of the input vector. The stump classifiers can be written as $h(\mathbf{x};\theta) = \text{sign}(\, s(x_k - \theta_0)\,)$ where $\theta = \{s, k, \theta_0\}$ and $s \in \{-1, 1\}$ specifies the label to predict on the positive side of $x_k - \theta_0$. Figure 1 below shows a possible decision boundary from a stump when the input vectors $\mathbf{x}$ are only two dimensional.

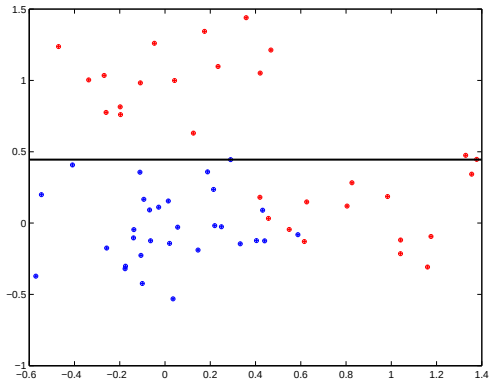


Figure 1: A possible decision boundary from a trained decision stump. The stump in the figure depends only on the vertical x_2-axis.

The boosting algorithm combines the stumps (as base learners) into an ensemble that, after m rounds of boosting, takes the following form

$$h_m(\mathbf{x}) = \sum_{j=1}^{m} \hat{\alpha}_j h(\mathbf{x}; \hat{\theta}_j) \tag{1}$$

where $\hat{\alpha}_j \geq 0$ but they do not necessarily sum to one. We can always normalize the ensemble by $\sum_{j=1}^{m} \hat{\alpha}_j$ after the fact. The simple Adaboost algorithm can be written in the following modular form:

(0) Set $W_0(t) = 1/n$ for $t = 1, \ldots, n$.

(1) At stage m, find a base learner $h(\mathbf{x}; \hat{\theta}_m)$ that approximately minimizes

$$-\sum_{t=1}^{m} \tilde{W}_{m-1}(t) y_t h(\mathbf{x}_t; \theta_m) = 2\epsilon_m - 1 \tag{2}$$

where ϵ_m is the weighted classification error (zero-one loss) on the training examples, weighted by the normalized weights $\tilde{W}_{m-1}(t)$.

(2) Given $\hat{\theta}_m$, set

$$\hat{\alpha}_m = 0.5 \log \left(\frac{1 - \hat{\epsilon}_m}{\hat{\epsilon}_m} \right) \tag{3}$$

where $\hat{\epsilon}_m$ is the weighted error corresponding to $\hat{\theta}_m$ chosen in step (1). For binary $\{-1, 1\}$ base learners, $\hat{\alpha}_m$ exactly minimizes the weighted training loss (loss of the ensemble):

$$J(\alpha_m, \hat{\theta}_m) = \sum_{t=1}^{n} \tilde{W}_{m-1}(t) \exp \Big(-y_t \alpha_m h(\mathbf{x}_t; \hat{\theta}_m) \Big) \tag{4}$$

In cases where the base learners are not binary (e.g., return values in the interval $[-1, 1]$), we would have to minimize Eq.(4) directly.

(3) Update the weights on the training examples based on the new base learner:

$$\tilde{W}_m(t) = c_m \cdot \tilde{W}_{m-1}(t) \exp \Big(-y_t \hat{\alpha}_m h(\mathbf{x}_t; \hat{\theta}_m) \Big) \tag{5}$$

where c_m is the normalization constant to ensure that $\tilde{W}_m(t)$ sum to one after the update. The new weights can be again interpreted as normalized losses for the new ensemble $h_m(\mathbf{x}_t) = h_{m-1}(\mathbf{x}) + \hat{\alpha}_m h(\mathbf{x}; \hat{\theta}_m)$.

Let's try to understand the boosting algorithm from several different perspectives. First of all, there are several different types of errors (errors here refer to zero-one classification losses, not the surrogate exponential losses). We can talk about the weighted error of base learner m, introduced at the m^{th} boosting iteration, relative to the weights $\tilde{W}_{m-1}(t)$ on the training examples. This is the weighted training error $\hat{\epsilon}_m$ in the algorithm. We can also measure the weighted error of the same base classifier relative to the updated weights,

i.e., relative to $\tilde{W}_m(t)$. In other words, we can measure how well the current base learner would do at the next iteration. Finally, in terms of the ensemble, we have the unweighted training error and the corresponding generalization (test) error, as a function of boosting iterations. We will discuss each of these in turn.

Weighted error. The weighted error achieved by a new base learner $h(\mathbf{x}; \hat{\theta}_m)$ relative to $\tilde{W}_{m-1}(t)$ tends to increase with m, i.e., with each boosting iteration (though not monotonically). Figure 2 below shows this weighted error $\hat{\epsilon}_m$ as a function of boosting iterations. The reason for this is that since the weights concentrate on examples that are difficult to classify correctly, subsequent base learners face harder classification tasks.

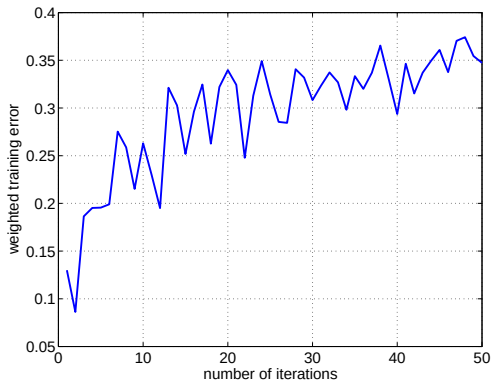


Figure 2: Weighted error $\hat{\epsilon}_m$ as a function of m.

Weighted error relative to updated weights. We claim that the weighted error of the base learner $h(\mathbf{x}; \hat{\theta}_m)$ relative to the updated weights $\tilde{W}_m(t)$ is exactly 0.5. This means that the base learner introduced at the m^{th} boosting iteration will be useless (at chance level) for the next boosting iteration. So the boosting algorithm would never introduce the same base learner twice in a row. The same learner might, however, reappear later on (relative to a different set of weights). One reason for this is that we don't go back and update $\hat{\alpha}_j$'s for base learners already introduced into the ensemble. So the only way to change the previously set coefficients is to reintroduce the base learners. Let's now see that the claim is indeed true. We can equivalently show that the *weighted agreement* relative to $\tilde{W}_m(t)$ is exactly zero:

$$\sum_{t=1}^{m} \tilde{W}_m(t) y_t h(\mathbf{x}_t; \hat{\theta}_m) = 0 \tag{6}$$

Consider the optimization problem for α_m after we have already found $\hat{\theta}_m$:

$$J(\alpha_m, \hat{\theta}_m) = \sum_{t=1}^{n} \tilde{W}_{m-1}(t) \exp\Big(-y_t \alpha_m h(\mathbf{x}_t; \hat{\theta}_m) \Big) \tag{7}$$

The derivative of $J(\alpha_m, \hat{\theta}_m)$ with respect to α_m has to be zero at the optimal value $\hat{\alpha}_m$ so that

$$\frac{d}{d\alpha_m} J(\alpha_m, \hat{\theta}_m)\Big|_{\alpha_m=\hat{\alpha}_m} = -\sum_{t=1}^{n} \tilde{W}_{m-1}(t) \exp\Big(-y_t \hat{\alpha}_m h(\mathbf{x}_t; \hat{\theta}_m) \Big) y_t h(\mathbf{x}_t; \hat{\theta}_m) \tag{8}$$

$$= -c_m \sum_{t=1}^{n} \tilde{W}_m(t) y_t h(\mathbf{x}_t; \hat{\theta}_m) = 0 \tag{9}$$

where we have used Eq.(5) to move from $\tilde{W}_{m-1}(t)$ to $\tilde{W}_m(t)$. So the result is an optimality condition for α_m.

Ensemble training error. The training error of the ensemble does not necessarily decrease monotonically with each boosting iteration. The exponential loss of the ensemble does, however, decrease monotonically. This should be evident since it is exactly the loss we are sequentially minimizing by adding the base learners. We can also quantify, based on the weighted error achieved by each base learner, how much the exponential loss decreases after each iteration. We will need this to relate the training loss to the classification error. In fact, the amount that the training loss decreases after iteration m is exactly c_m, the normalization constant for the updated weights (we have to normalize the weights precisely because the exponential loss over the training examples decreases). Note also that c_m is exactly $J(\hat{\alpha}_m, \hat{\theta}_m)$. Now,

$$J(\hat{\alpha}_m, \hat{\theta}_m) = \sum_{t=1}^{n} \tilde{W}_{m-1}(t) \exp\Big(-y_t \hat{\alpha}_m h(\mathbf{x}_t; \hat{\theta}_m) \Big) \tag{10}$$

$$= \sum_{t:\, y_t = h(\mathbf{x}_t; \hat{\theta}_m)} \tilde{W}_{m-1}(t) \exp\big(-\hat{\alpha}_m \big) + \sum_{t:\, y_t \neq h(\mathbf{x}_t; \hat{\theta}_m)} \tilde{W}_{m-1}(t) \exp\big(\hat{\alpha}_m \big) \tag{11}$$

$$= (1 - \hat{\epsilon}_m) \exp\big(-\hat{\alpha}_m \big) + \hat{\epsilon}_m \exp\big(\hat{\alpha}_m \big) \tag{12}$$

$$= (1 - \hat{\epsilon}_m) \sqrt{\frac{\hat{\epsilon}_m}{1 - \hat{\epsilon}_m}} + \hat{\epsilon}_m \sqrt{\frac{1 - \hat{\epsilon}_m}{\hat{\epsilon}_m}} \tag{13}$$

$$= 2\sqrt{\hat{\epsilon}_m (1 - \hat{\epsilon}_m)} \tag{14}$$

Note that this is always less than one for any $\hat{\epsilon}_m < 1/2$. The training loss of the ensemble after m boosting iterations is exactly the product of these terms (renormalizations). In

other words,

$$\sum_{t=1}^{m} \exp\left(-y_t h_m(\mathbf{x}_t)\right) = \prod_{k=1}^{m} 2\sqrt{\hat{\epsilon}_k(1-\hat{\epsilon}_k)} \tag{15}$$

This and the observation that

$$\text{step}(z) \leq \exp(z) \tag{16}$$

for all z, where the step function $\text{step}(z) = 1$ if $z > 0$ and zero otherwise, suffices for our purposes. A simple upper bound on the training error of the ensemble, $\text{err}_n(h_m)$, follows from

$$\begin{aligned}
\text{err}_n(h_m) &= \frac{1}{n}\sum_{t=1}^{n} \text{step}\left(-y_t h_m(\mathbf{x}_t)\right) && (17)\\
&\leq \frac{1}{n}\sum_{t=1}^{n} \exp\left(-y_t h_m(\mathbf{x}_t)\right) && (18)\\
&= \frac{1}{n}\prod_{k=1}^{m} 2\sqrt{\hat{\epsilon}_k(1-\hat{\epsilon}_k)} && (19)
\end{aligned}$$

Thus the exponential loss over the training examples is an upper bound on the training error and this upper bound goes down monotonically with m provided that the base learners are learning something at each iteration (their weighted errors less than half). Figure 3 shows the training error as well as the upper bound as a function of the boosting iterations.

Ensemble test error. We have so far discussed only training errors. The goal, of course, is to generalize well. What can we say about the generalization error of ensemble generated by the boosting algorithm? We have repeatedly tied the generalization error to some notion of margin. The same is true here. Consider figure 5 below. It shows a typical plot of the ensemble training error and the corresponding generalization error as a function of boosting iterations. Two things are notable in the plot. First, the generalization error seems to decrease (slightly) even after the ensemble has reached zero training error. Why should this be? The second surprising thing seems to be the fact that the generalization error does not increase even after a large number of boosting iterations. In other words, the boosting algorithm appears to be somewhat resistant to overfitting. Let's try to explain these two (related) observations.

The votes $\{\hat{\alpha}_j\}$ generated by the boosting algorithm won't sum to one. We will therefore renormalize ensemble

$$\tilde{h}_m(\mathbf{x}) = \frac{\hat{\alpha}_1 h(\mathbf{x};\hat{\theta}_1) + \ldots \hat{\alpha}_m h(\mathbf{x};\hat{\theta}_m)}{\hat{\alpha}_1 + \ldots + \hat{\alpha}_m} \tag{20}$$

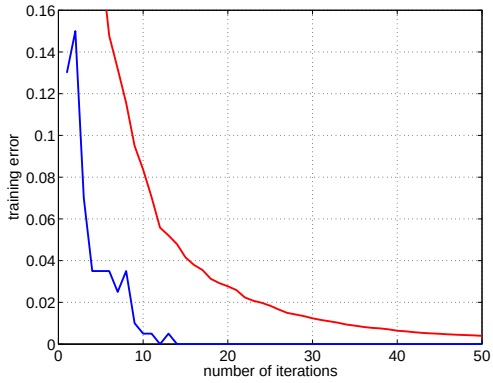


Figure 3: The training error of the ensemble as well as the corresponding exponential loss (upper bound) as a function of the boosting iterations.

so that $\tilde{h}_m(\mathbf{x}) \in [-1, 1]$. As a result, we can define a "voting margin" for training examples as $\text{margin}(t) = y_t \tilde{h}_m(\mathbf{x}_t)$. The margin is positive if the example is classified correctly by the ensemble. It represents the degree to which the base classifiers agree with the correct classification decision (negative value indicates disagreement). Note that $\text{margin}(t) \in [-1, 1]$. It is a very different type of margin (voting margin) than the geometric margin we have discussed in the context linear classifiers. Now, in addition to the training error $\text{err}_n(h_m)$ we can define a margin error $\text{err}_n(h_m; \rho)$ that is the fraction of example margins that are at or below the threshold ρ. Clearly, $\text{err}_n(h_m) = \text{err}_n(h_m; 0)$. We now claim that the boosting algorithm, even after $\text{err}_n(h_m; 0) = 0$ will decrease $\text{err}_n(h_m; \rho)$ for larger values of $\rho > 0$. Figure 4a-b provide an empirical illustration that this is indeed happening. This is perhaps easy to understand as a consequence of the fact that exponential loss, $\exp(-\text{margin}(t))$, decreases as a function of the margin, even after the margin is positive.

The second issue to explain is the apparent resistance to overfitting. One reason is that the complexity of the ensemble does not increase very quickly as a function of the number of base learners. We will make this statement more precise later on. Moreover, the boosting iterations modify the ensemble in sensible ways (increasing the margin) even after the training error is zero. We can also relate the margin, or the margin error $\text{err}_n(h_m; \rho)$ directly to generalization error. Another reason for resistance to overfitting is that the sequential procedure for optimizing the exponential loss is not very effective. We would overfit much more quickly if we reoptimized $\{\alpha_j\}$'s *jointly* rather than through the sequential procedure (see the discussion of boosting as gradient descent below).

Boosting as gradient descent. We can also view the boosting algorithm as a simple

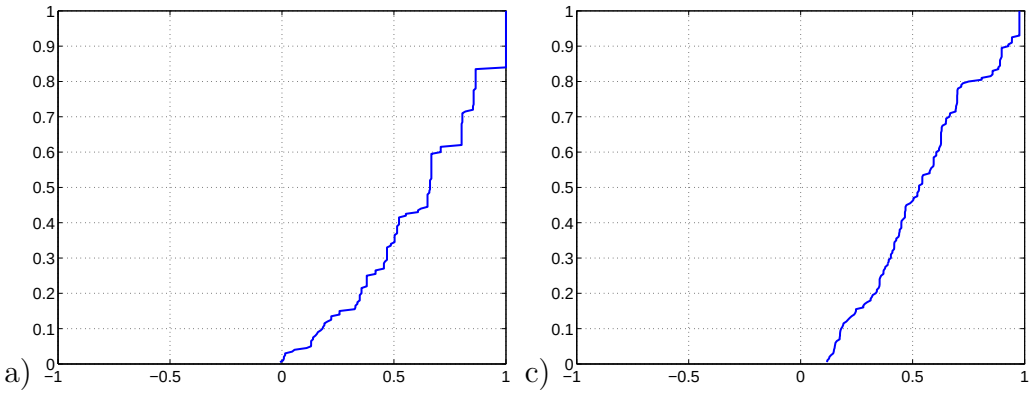


Figure 4: The margin errors $\text{err}_n(h_m; \rho)$ as a function of ρ when a) $m = 10$ b) $m = 50$.

gradient descent procedure (with line search) in the space of discriminant functions. To understand this we can view each base learner $h(\mathbf{x}; \theta)$ as a vector based on evaluating it on each of the training examples:

$$\vec{h}(\theta) = \begin{bmatrix} h(\mathbf{x}_1; \theta) \\ \cdots \\ h(\mathbf{x}_n; \theta) \end{bmatrix} \tag{21}$$

The ensemble vector $\vec{h}_m$, obtained by evaluating $h_m(\mathbf{x})$ at each of the training examples, is a positive combination of the base learner vectors:

$$\vec{h}_m = \sum_{j=1}^{m} \hat{\alpha}_m \vec{h}(\hat{\theta}_m) \tag{22}$$

The exponential loss objective we are trying to minimize is now a function of the ensemble vector $\vec{h}_m$ and the training labels. Suppose we have $\vec{h}_{m-1}$. To minimize the objective, we can select a useful direction, $\vec{h}(\hat{\theta}_m)$, along which the objective seems to decrease. This is exactly how we derived the base learners. We can then find the minimum of the objective by moving in this direction, i.e., evaluating vectors of the form $\vec{h}_{m-1} + \alpha_m \vec{h}(\hat{\theta}_m)$. This is a line search operation. The minimum is attained at $\hat{\alpha}_m$, we obtain $\vec{h}_m = \vec{h}_{m-1} + \hat{\alpha}_m \vec{h}(\hat{\theta}_m)$, and the procedure can be repeated.

Viewing the boosting algorithm as a simple gradient descent procedure also helps us understand why it can overfit if we continue with the boosting iterations.

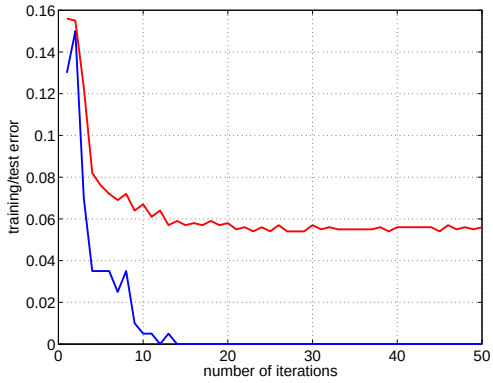


Figure 5: The training error of the ensemble as well as the corresponding generalization error as a function of boosting iterations.

Complexity and generalization

We have approached classification problems using linear classifiers, probabilistic classifiers, as well as ensemble methods. Our goal is to understand what type of performance guarantees we can give for such methods based on finite training data. This is a core theoretical question in machine learning. For this purpose we will need to understand in detail what "complexity" means in terms of classifiers. A single classifier is never complex or simple; complexity is a property of the set of classifiers or the model. Each model selection criterion we have encountered provided a slightly different definition of "model complexity".

Our focus here is on performance guarantees that will eventually relate the margin we can attain to the generalization error, especially for linear classifiers (geometric margin) and ensembles (voting margin). Let's start by motivating the complexity measure we need for this purpose with an example.

Consider a simple decision stump classifier restricted to x_1 coordinate of $2-$dimensional input vectors $\mathbf{x} = [x_1, x_2]^T$. In other words, we consider stumps of the form

$$h(\mathbf{x}; \theta) = \text{sign}\,(\, s(x_1 - \theta_0)\,) \tag{23}$$

where $s \in \{-1, 1\}$ and $\theta_0 \in \mathcal{R}$ and call this set of classifiers $\mathcal{F}_1$. Example decision boundaries are displayed in Figure 6.

Suppose we are getting the data points in a sequence and we are interested in seeing when our predictions for future points become constrained by the labeled points we have already

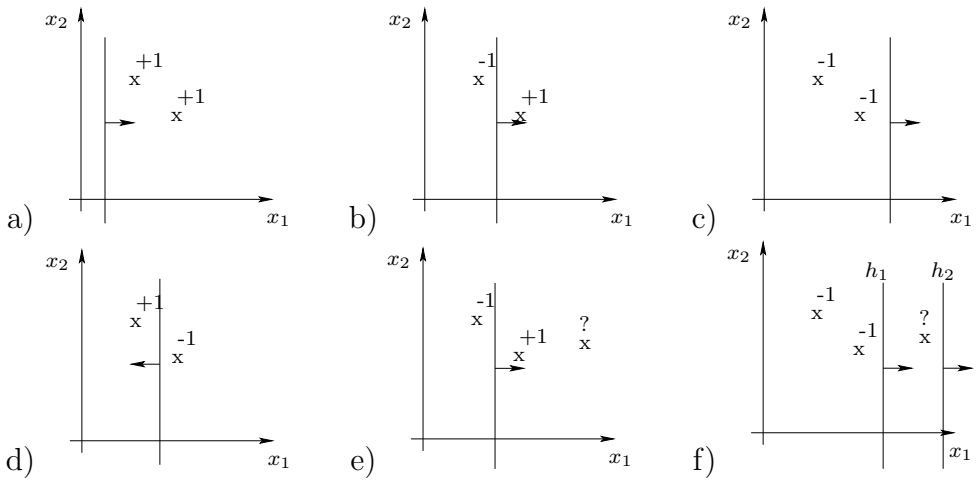


Figure 6: Possible decision boundaries corresponding to decision stumps that rely only on x_1 coordinate. The arrow (normal) to the boundary specifies the positive side.

seen. Such constraints pertain to both the data and the set of classifiers $\mathcal{F}_1$. See Figure 6e. Having seen the labels for the first two points, -1 and $+1$, all classifiers $h \in \mathcal{F}_1$ that are consistent with these two labeled points have to predict $+1$ for the next point. Since the labels we have seen force us to classify the new point in only one way, we can claim to have learned something. We can also understand this as a limitation of (the complexity of) our set of classifiers. Figure 6f illustrates an alternative scenario where we can find two classifiers $h_1 \in \mathcal{F}_1$ and $h_2 \in \mathcal{F}_1$, both consistent with the first two labels in the figure, but make different predictions for the new point. We could therefore classify the new point either way. Recall that this freedom is not available for all label assignments to the first two points. So, the stumps in $\mathcal{F}_1$ can classify any two points (in general position) in all possible ways (Figures 6a-d) but are already partially constrained in how they assign labels to three points (Figure 6e). In more technical terms we say that $\mathcal{F}_1$ *shatters* (can generate all possible labels over) two points but not three.

Similarly, for linear classifiers in 2−dimensions, all the eight possible labelings of three points can be obtained with linear classifiers (Figure 7a). Thus linear classifiers in two dimensions shatter three points. However, there are labels over four points that no linear classifier can produce (Figure 7b).

VC-dimension. As we increase the number of data points, the set of classifiers we are considering may no longer be able to label the points in all possible ways. Such emerging constraints are critical to be able to predict labels for new points. This motivates a key

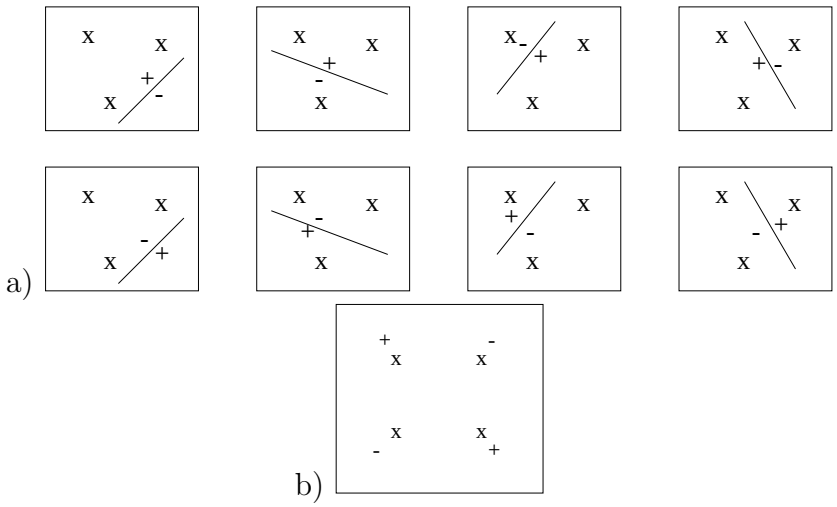


Figure 7: Linear classifiers on the plane can shatter three points a) but not four b).

measure of complexity of the set of classifiers, the *Vapnik-Chervonenkis dimension*. The VC-dimension is defined as the maximum number of points that a classifier can shatter. So, the VC-dimension of $\mathcal{F}_1$ is two, denoted as $d_{VC}(\mathcal{F}_1)$, and the VC-dimension of linear classifiers on the plane is three. Note that the definition involves a maximum over the possible points. In other words, we may find less than d_{VC} points that the set of classifiers cannot shatter (e.g., linear classifiers with points exactly on a line in $2-d$) but there cannot be any set of more than d_{VC} points that the classifier can shatter.

The VC-dimension of the set of linear classifiers in $d-$dimensions is $d+1$, i.e., the number of parameters. This is not a useful result for understanding how kernel methods work. For example, the VC-dimension of linear classifiers using the radial basis kernel is ∞. We can incorporate the notion of margin in VC-dimension, however. This is known as the V_γ dimension. The V_γ dimension of a set of linear classifiers that attain geometric margin γ when examples lie within an enclosing sphere of radius R is bounded by R^2/γ^2. In other words there are not that many points we can label in all possible ways if any valid labeling has to be with margin γ. This result is independent of the dimension of the classifier, and is exactly the mistake bound for the perceptron algorithm!

Lecture topics:

- margin and generalization
  - linear classifiers
  - ensembles
- mixture models

Margin and generalization: linear classifiers

As we increase the number of data points, any set of classifiers we are considering may no longer be able to label the points in all possible ways. Such emerging constraints are critical to be able to predict labels for new points. This motivates a key measure of complexity of the set of classifiers, the *Vapnik-Chervonenkis dimension*. The VC-dimension is defined as the maximum number of points that a classifier can shatter. The VC-dimension of linear classifiers on the plane is three (see previous lecture). Note that the definition involves a maximum over the possible points. In other words, we may find less than d_{VC} points that the set of classifiers cannot shatter (e.g., linear classifiers with points exactly on a line in $2-d$) but there cannot be any set of more than d_{VC} points that the classifier can shatter.

The VC-dimension of the set of linear classifiers in $d-$dimensions is $d+1$, i.e., the number of parameters. This relation to the number of parameters is typical albeit certainly not always true (e.g., the VC-dimension may be infinite for a classifier with a single real parameter!).

The VC-dimension immediately generalizes our previous results for bounding the expected error from a finite number of classifiers. There are a number of technical steps involved that we won't get into, however. Loosely speaking, d_{VC} takes the place of the logarithm of the number of classifiers in our set. In other words, we are counting the number of classifiers on the basis of how they can label points, not based on their identities in the set. More precisely, we have for any set of classifiers $\mathcal{F}$: with probability at least $1-\delta$ over the choice of the training set,

$$R(f) \leq R_n(f) + \epsilon(n, d_{VC}, \delta), \quad \text{uniformly for all } f \in \mathcal{F} \tag{1}$$

where the complexity penalty is now a function of $d_{VC} = d_{VC}(\mathcal{F})$:

$$\epsilon(n, d_{VC}, \delta) = \sqrt{\frac{d_{VC}(\log(2n/d_{VC}) + 1) + \log(1/(4\delta))}{n}} \tag{2}$$

The result is problematic for kernel methods. For example, the VC-dimension of kernel classifiers with the radial basis kernel is ∞. We can, however, incorporate the notion of margin in the classifier "dimension". One such definition is V_γ dimension that measures the VC-dimension with the constraint that distinct labelings have to be obtained with a fixed margin γ. Suppose all the examples fall within an enclosing sphere of radius R. Then, as we increase γ, there will be very few examples we can classify in all possible ways with this constraint (especially when $\gamma \to R$; cf. Figure 1). Put another way, the VC-dimension of a set of linear classifiers required to attain a prescribed margin can be much lower (decreasing as a function of the margin). In fact, V_γ dimension for linear classifiers is bounded by R^2/γ^2, i.e., inversely proportional to the squared margin. Note that this result is *independent of the dimension of input examples*, and is exactly the mistake bound for the perceptron algorithm!

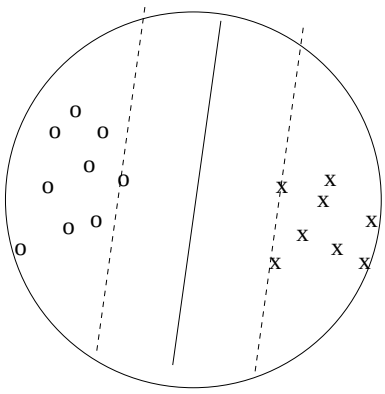


Figure 1: The set of linear classifiers required to obtain a specific geometric margin has a lower VC-dimension when the examples remain within an enclosing sphere.

The previous generalization guarantees can be used with V_γ dimension as well so long as we replace the training error with margin violations, i.e., we count the fraction of examples that cannot be separated with margin at least γ.

Margin and generalization: ensembles

An ensemble classifier can be written as a convex combination of simpler base classifiers

$$h_m(\mathbf{x}) = \sum_{j=1}^{m} \alpha_j h(\mathbf{x}; \theta_j) \tag{3}$$

where $\alpha_j \geq 0$ and $\sum_{j=1}^m \alpha_j = 1$. Boosting generates such ensembles but does not normalized the coefficients α_j to sum to one. The normalization can be easily performed after the fact.

We are interested in understanding the complexity of such ensembles and how generalization guarantees depends on the voting margin achieved, e.g., through boosting algorithm. Note that our discussion here will not refer to how such ensembles are generated.

Let's start by defining what the ensembles are not. They are not decision trees. A decision (classification) tree is a method of recursively partitioning the set of examples into regions such that within each region the examples would have as uniform labels as possible. The partitioning in a decision tree could be based on the same type of decision stumps as we have used for the ensemble. In the ensemble, however, the domain for all the stumps is the whole space. In other words, you cannot restrict the application of the stump within a specific partition. In the ensemble, each stump contributes to the classification of all the examples.

How powerful are ensembles based on the decision stumps? To understand this further let's show how we can shatter any n points with $2n$ stumps even in one dimensions. It suffices to show that we can find an ensemble with $2n$ stumps that reproduces any specific labeling $y_1, \ldots, y_n$ of n points $x_1, \ldots, x_n$ (now real numbers). To do so, we will construct an ensemble of two stumps to reproduce the label y_t for x_t but without affecting the classification of other training examples. If ϵ is less than the smallest distance between any two training examples, then

$$h_{pair}(x; x_t, y_t) = \frac{1}{2}\text{sign}\left(y_t(x - x_t + \epsilon)\right) + \frac{1}{2}\text{sign}\left(-y_t(x - x_t - \epsilon)\right) \tag{4}$$

has value y_t within interval $[x_t - \epsilon, x_t + \epsilon]$ is zero everywhere else. Thus, setting $\alpha_t = 1/n$,

$$h_{2n}(x) = \sum_{t=1} \alpha_t h_{pair}(x; x_t, y_t) \tag{5}$$

has the correct sign for all the training examples. The ensemble of $2n$ components therefore has VC-dimension at least n. Ensembles are powerful as classifiers in this sense and their VC-dimension poorly explains their success in practice.

Each example in the above construction only has a very low voting margin $1/n$, however. Perhaps we can similarly refine the analysis to incorporate the voting margin as we did above with linear classifiers and the geometric margin. The key idea is to reduce an ensemble with many components to a coarse ensemble with few components but one that nevertheless classifies the examples in the same way. When the original ensemble achieves a large voting margin this is indeed possible, and the size of the coarse approximation that

we need decreases with increasing voting margin. In other words, if we achieve a large voting margin, we could have solved the same classification problem with much smaller ensemble insofar as we only pay attention to the classification error.

Based on this and other more technical ideas we can show that with probability at least $1 - \delta$ over the choice of the training data,

$$R_n(\hat{h}) \leq R_n(\hat{h}; \rho) + \tilde{\mathcal{O}}\left(\sqrt{\frac{d_{VC}/\rho^2}{n}}\right), \tag{6}$$

where the $\tilde{\mathcal{O}}(\cdot)$ notation hides constants and logarithmic terms, $R_n(\hat{h}; \rho)$ counts the number of training examples with voting margin less than ρ, and d_{VC} is the VC-dimension of the base classifiers. Note that the result does not depend on the number of base classifiers in the ensemble $\hat{h}$. Note also that the effective dimension d_{VC}/ρ^2 that the number of training examples is compared to has a similar form as before, decreasing with the margin ρ. See Schapire et al. (1998) for details. The paper is available from the course website as optional reading.

Mixture models

There are many problems in machine learning that are not simple classification problems but rather modeling problems (e.g., clustering, diagnosis, combining multiple information sources for sequence annotation, and so on). Moreover, even within classification problems, we often have unobserved variables that would make a difference in terms of classification. For example, if we are interested in classifying tissue samples into specific categories (e.g., tumor type), it would be useful to know the composition of the tissue sample in terms of cells that are present and in what proportions. While such variables are not typically observed, we can still model them and make use of their presence in prediction.

Mixture models are simple probability models that try to capture ambiguities in the available data. They are simple, widely used and useful. As the name suggests, a mixture model "mixes" different predictions on the probability scale. The mixing is based on alternative ways of *generating* the observed data. Let $\mathbf{x}$ be a vector of observations. A mixture model over vectors $\mathbf{x}$ is defined as follows. We assume each $\mathbf{x}$ could be of m possible types. If we knew the type, j, we would model $\mathbf{x}$ with a conditional distribution $P(\mathbf{x}|j)$ (e.g., Gaussian with a specific mean). If the overall frequency of type j in the data is $P(j)$, then the

"mixture distribution" over $\mathbf{x}$ is given by

$$P(\mathbf{x}) = \sum_{j=1}^{m} P(\mathbf{x}|j)P(j) \tag{7}$$

In other words, $\mathbf{x}$ could be generated in m possible ways. We imagine the generative process to be as follows: sample j from the frequencies $P(j)$, then $\mathbf{x}$ from the corresponding conditional distribution $P(\mathbf{x}|j)$. Since we do not observe the particular way the example was generated (assuming the model is correct), we sum over the possibilities, weighted by the overall frequencies.

We have already encountered mixture models. Take, for example, the Naive Bayes model $P(\mathbf{x}|y)P(y)$ over the feature vector $\mathbf{x}$ and label y. If we pool together examples labeled $+1$ and those labeled -1, and throw away the label information, then the Naive Bayes model predicts feature vectors $\mathbf{x}$ according to

$$P(\mathbf{x}) = \sum_{y=\pm 1} P(\mathbf{x}|y)P(y) = \sum_{y=\pm 1} \left[\prod_{i=1}^{d} P(x_i|y) \right] P(y) \tag{8}$$

In other words, the distribution $P(\mathbf{x})$ assumes that the examples come in two different varieties corresponding to their label. This type of unobserved label information is precisely what the mixtures aim to capture.

Let's start with a simple two component mixture of Gaussians model (in two dimensions):

$$P(\mathbf{x}|\theta) = P(1)N(\mathbf{x}; \mu_1, \sigma_1^2 I) + P(2)N(\mathbf{x}; \mu_2, \sigma_2^2 I) \tag{9}$$

The parameters θ defining the mixture model are $P(1)$, $P(2)$, μ_1, μ_2, σ_1^2, and σ_2^2. Figure 2 shows data generated from such a model. Note that the frequencies $P(j)$ (a.k.a. *mixing proportions*) control the size of the resulting clusters in the data in terms of how many examples they involve, μ_j's specify the location of cluster centers, and σ_j^2's control how spread out the clusters are. Note that each example in the figure could in principle have been generated in two possible ways (which mixture component it was sampled from).

There are many ways of using mixtures. Consider, for example, the problem of predicting final exam score vectors for students in machine learning. Each observation is a vector $\mathbf{x}$ with components that specify the points the student received in a particular question. We would expect that different types of students succeed in different types of questions. This "student type" information is not available in the exam score vectors, however, but we can model it. Suppose there are n students taking the course so that we have n vector

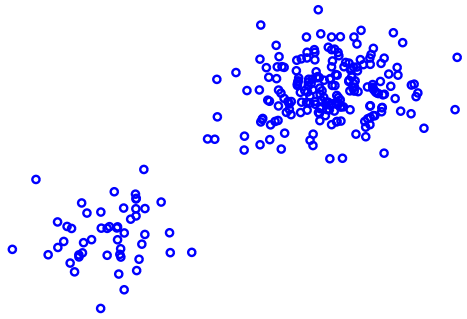


Figure 2: Samples from a mixture of two Gaussians

observations $\mathbf{x}_1, \ldots, \mathbf{x}_n$. Suppose there are m underlying types of students (the number of types can be inferred from the data; this is a model selection problem). We also don't know how many students taking the course are of particular type, i.e., we have to estimate the mixing proportions $P(j)$ as well. The mixture distribution over a single example score vector is now given by

$$P(\mathbf{x}|\theta) = \sum_{j=1}^{m} P(\mathbf{x}|j)P(j) \tag{10}$$

We won't concern ourselves at this point with the problem of deciding how to parameterize the conditional distributions $P(\mathbf{x}|j)$. Suffice it to say that it wouldn't be unreasonable to assume that $P(\mathbf{x}|j) = \prod_{i=1}^{d} P(x_i|j)$ as in the Naive Bayes model but each x_i would take values in the range of scores for the corresponding exam question. Now, our mixture model assumes that each student is of particular type. If someone gave us this information, i.e., j_t for $\mathbf{x}_t$, then we would model the observations with the conditional distribution

$$P(\mathbf{x}_1, \ldots, \mathbf{x}_n|j_1, \ldots, j_n, \theta) = \prod_{t=1}^{n} P(\mathbf{x}_t|j_t) \tag{11}$$

assuming each student obtains their score independently from others. But the type information is not present in the data so we will have to sum over the possible values of j_t for each student, weighted by the prior probabilities of types, $P(j_t)$ (same for all students):

$$P(\mathbf{x}_1, \ldots, \mathbf{x}_n|\theta) = \prod_{t=1}^{n} \left[\sum_{j_t=1}^{m} P(\mathbf{x}_t|j_t)P(j_t) \right] = \prod_{t=1}^{n} \left[\sum_{j=1}^{m} P(\mathbf{x}_t|j)P(j) \right] \tag{12}$$

This is the likelihood of the observed data according to our mixture model. It is important to understand that the model would be very different if we exchanged the product and the sum in the above expression, i.e., define the model as

$$P(\mathbf{x}_1, \ldots, \mathbf{x}_n|\theta) = \sum_{j=1}^{m} \left[\prod_{t=1}^{n} P(\mathbf{x}_t|j) \right] P(j) \tag{13}$$

This is also a mixture model but one that assumes that all students in the class are of specific single type, we just don't know which one, and are summing over the m possibilities (in the previous model there were m^n possible assignments of types over n students).

Lecture topics:

- Different types of mixture models (cont'd)
- Estimating mixtures: the EM algorithm

Mixture models (cont'd)

Basic mixture model

Mixture models try to capture and resolve observable ambiguities in the data. E.g., an $m-$component Gaussian mixture model

$$P(\mathbf{x}; \theta) = \sum_{j=1}^{m} P(j) N(\mathbf{x};\, \mu_j, \Sigma_j) \tag{1}$$

The parameters θ include the mixing proportions (prior distribution) $\{P(j)\}$, means of component Gaussians $\{\mu_j\}$, and covariances $\{\Sigma_j\}$. The notation $\{P(j)\}$ is a shorthand for $\{P(j),\, j = 1, \ldots, m\}$.

To generate a sample $\mathbf{x}$ from such a model we would first sample j from the prior distribution $\{P(j)\}$, then sample $\mathbf{x}$ from the selected component $N(\mathbf{x};\, \mu_j, \Sigma_j)$. If we generated n samples, then we would get m potentially overlapping clusters of points, where each cluster center would correspond to one of the means μ_j and the number of points in the clusters would be approximately $n\, P(j)$. This is the type of structure in the data that the mixture model is trying to capture if estimated on the basis of observed $\mathbf{x}$ samples.

Student exam model: 1-year

We can model vectors of exam scores with mixture models. Each $\mathbf{x}$ is a vector of scores from a particular student and samples correspond to students. We expect that the population of students in a particular year consists of m different types (e.g., due to differences in background). If we expect each type to be present with an overall probability $P(j)$, then each student score is modeled as a mixture

$$P(\mathbf{x}|\theta) = \sum_{j=1}^{m} P(\mathbf{x}|\theta_j) P(j) \tag{2}$$

where we sum over the types (weighted by $P(j)$) since we don't know what type of student t is prior to seeing their exam score $\mathbf{x}_t$.

If there are n students taking the exam a particular year, then the likelihood of all the student score vectors, $D_1 = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, would be

$$L(D_1; \theta) = \prod_{t=1}^{n} P(\mathbf{x}_t|\theta) = \prod_{t=1}^{n} \left(\sum_{j=1}^{m} P(\mathbf{x}|\theta_j) P(j) \right) \tag{3}$$

Student exam model: K-years

Suppose now that we have student data from K years of offering the course. In year k we have n_k students who took the course. Let $\mathbf{x}_{k,t}$ denote the score vector for a student t in year k. Note that t is just an index to identify samples each year and the same index does not imply that the same student took the course multiple years. We can now assume that the number of student types as well as $P(\mathbf{x}|\theta_j)$ remain the same from year to year (the parameters θ_j are the same for all years). However, the population of students may easily change from year to year, and thus the prior probabilities over the types have to be set differently. Let $P(j|k)$ denote the prior probabilities over the types in year k (all of these would have to be estimated of course). Now, according to our mixture distribution, we expect example scores for students in year k be sampled from

$$P(\mathbf{x}|k, \theta) = \sum_{j=1}^{m} P(\mathbf{x}|\theta_j) P(j|k) \tag{4}$$

The likelihood of all the data, across K years, $D = \{D_1, \ldots, D_K\}$, is given by

$$L(D; \theta) = \prod_{k=1}^{K} \prod_{t=1}^{n_k} P(\mathbf{x}_{k,t}|k, \theta) = \prod_{k=1}^{K} \prod_{t=1}^{n_k} \left(\sum_{j=1}^{m} P(\mathbf{x}_{k,t}|\theta_j) P(j|k) \right) \tag{5}$$

The parameters θ here include the mixing portions $\{P(j|k)\}$ that change from year to year in addition to $\{\theta_j\}$.

Collaborative filtering

Mixture models are useful also in recommender systems. Suppose we have n users and m movies and our task is to recommend movies for users. The users have each rated a small fraction of movies and our task is to fill-in the rating matrix, i.e., provide a predicted rating for each user across all m movies. Such a prediction task is known as a collaborative filtering problem (see Figure 1).

Say the possible ratings are $r_{ij} \in \{1, \ldots, 5\}$ (i.e., how many stars you assign to each movie). We will use i to index users and j for movies; a rating r_{ij}, if provided, specifies how user

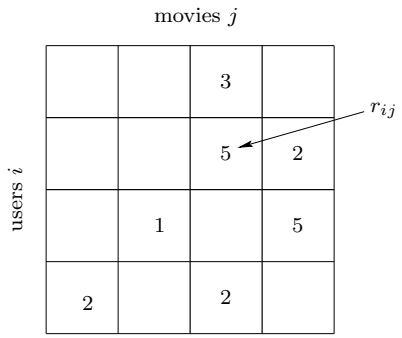


Figure 1: Partially observed rating matrix for a collaborative filtering task.

i rated movie j. Since only a small fraction of movies are rated by each user, we need a way to index these elements of the user/movie matrix: we say $(i, j) \in I_D$ if rating r_{ij} is available (observed). D denotes all the observed ratings.

We can build on the previous discussion on mixture models. We can represent each movie as a distribution over "movie types" $z_m \in \{1, \ldots, K_m\}$. Similarly, a user is represented by a distribution over "user types" $z_u \in \{1, \ldots, K_u\}$. We do not assume that each movie corresponds to a single movie type across all users. Instead, we interpret the distribution over movie types as a bag of features corresponding to the movie and we resample from this bag in the context of each user. This is analogous to predicting exam scores for students in a particular year (we didn't assume that all the students had the same type). We also make the same assumption about user types, i.e., that the type is sampled from the "bag" for each rating, resulting potentially in different types for different movies. Since all the unobserved quantities are summed over, we do not explicitly assign any fixed movie/user type to a rating.

We imagine generating the rating r_{ij} associated with (i, j) element of the rating matrix as follows. Sample a movie type from $P(z_m|j)$, sample a user type from $P(z_u|i)$, then sample a rating r_{ij} with probability $P(r_{ij}|z_u, z_m)$. All the probabilities involved have to estimated from the available data. Note that we will resample movie and user types for each rating. The resulting mixture model for rating r_{ij} is given by

$$P(r_{ij}|i, j, \theta) = \sum_{z_u=1}^{K_u} \sum_{z_m=1}^{K_m} P(r_{ij}|z_u, z_m) P(z_u|i) P(z_m|j) \tag{6}$$

where the parameters θ refer to the mapping from types to ratings $\{P(r|z_u, z_m)\}$ and the user and movie specific distributions $\{P(z_u|i)\}$ and $\{P(z_m|j)\}$, respectively. The likelihood

of the observed data D is

$$L(D;\theta) = \prod_{(i,j)\in I_D} P(r_{ij}|i,j,\theta) \tag{7}$$

Users rate movies differently. For example, some users may only use a part of the scale (e.g., 3, 4 or 5) while others may be bi-modal, rating movies either very bad 1 or very good 5. We can improve the model by assuming that each user has a rating style $s \in \{1,\ldots,K_s\}$. The styles are assumed to be the same for all users, we just don't know how to assign each user to a particular style. The prior probability that any randomly chosen user would have style s is specified by $P(s)$. These parameters are common to all users. We also assume that the rating predictions $P(r_{ij}|z_u,z_m)$ associated with user/movie types now depend on the style s as well: $P(r_{ij}|z_u,z_m,s)$.

We have to be a bit careful in writing the likelihood of the data. All the ratings of one user have to come from one rating style s but we can sum over the possibilities. As a result, the likelihood of the observed ratings is modified to be

$$L'(D;\theta) = \prod_{i=1}^{n}\left[\sum_{s=1}^{K_s} P(s) \overbrace{\prod_{j:(i,j)\in I_D}\left(\sum_{z_u=1}^{K_u}\sum_{z_m=1}^{K_m} P(r_{ij}|z_u,z_m,s)P(z_u|i)P(z_m|j)\right)}^{\text{likelihood of user } i\text{'s ratings with style } s}\right] \tag{8}$$

The model does not actually involve that many parameters to estimate. There are exactly

$$\overbrace{(K_s-1)}^{\{P(s)\}} + \overbrace{(5-1)K_uK_mK_s}^{\{P(r|z_u,z_m,s)\}} + \overbrace{n(K_u-1)}^{\{P(z_u|i)\}} + \overbrace{m(K_m-1)}^{\{P(z_m|j)\}} \tag{9}$$

independent parameters in the model.

A realistic model would include, in addition, a prediction of the "missing elements" in the rating matrix, i.e., a model of why the entry was missing (a user failed to rate a movie they had seen, not seen but could, chosen not to see, etc.).

Estimating mixtures: the EM-algorithm

We have seen a number of different types of mixture models. The advantage of mixture models lies in their ability to incorporate and resolve ambiguities in the data, especially

in terms of unidentified sub-groups. However, we can make use of them only if we can estimate such models easily from the available data.

Complete data. The simplest way to understand how to estimate mixture models is to start by pretending that we knew all the sub-typing (component) assignments for each available data point. This is analogous to knowing the label for each example in a classification context. We don't actually know these (they are unobserved in the data) but solving the estimation problem in this context will help us later on.

Let's begin with the simple Gaussian mixture model in Eq.(1),

$$P(\mathbf{x}; \theta) = \sum_{j=1}^{m} P(j) N(\mathbf{x};\, \mu_j, \Sigma_j) \tag{10}$$

and pretend that each observation $\mathbf{x}_1, \ldots, \mathbf{x}_n$ also had information about the component that was responsible for generating it, i.e., we also observed $j_1, \ldots, j_n$. This additional component information is convenient to include in the form of 0-1 assignments $\delta(j|t)$ where $\delta(j_t|t) = 1$ and $\delta(j|t) = 0$ for all $j \neq j_t$. The log-likelihood of this *complete data* is

$$\begin{aligned} l(\mathbf{x}_1, \ldots, \mathbf{x}_n, j_1, \ldots, j_n; \theta) &= \sum_{t=1}^{n} \log \left[P(j_t) N(\mathbf{x}_t;\, \mu_{j_t}, \Sigma_{j_t}) \right] && (11) \\ &= \sum_{t=1}^{n} \sum_{j=1}^{m} \delta(j|t) \log \left[P(j) N(\mathbf{x}_t;\, \mu_j, \Sigma_j) \right] && (12) \\ &= \sum_{j=1}^{m} \left(\sum_{t=1}^{n} \delta(j|t) \right) \log P(j) \\ &\quad + \sum_{j=1}^{m} \left(\sum_{t=1}^{n} \delta(j|t) \log N(\mathbf{x}_t;\, \mu_j, \Sigma_j) \right) && (13) \end{aligned}$$

It's important to note that in trying to maximize this log-likelihood, all the Gaussians can be estimated separately from each other. Put another way, because our pretend observations are "complete", we can estimate each component from only data pertaining to it; the problem of resolving which component should be responsible for which data points is not

present. As a result, the maximizing solution can be written in closed form:

$$\hat{P}(j) = \frac{\hat{n}(j)}{n}, \text{ where } \hat{n}(j) = \sum_{t=1}^{n} \delta(j|t) \tag{14}$$

$$\hat{\mu}_j = \frac{1}{\hat{n}(j)} \sum_{t=1]}^{n} \delta(j|t)\mathbf{x}_t \tag{15}$$

$$\hat{\Sigma}_j = \frac{1}{\hat{n}(j)} \sum_{t=1]}^{n} \delta(j|t)(\mathbf{x}_t - \hat{\mu}_j)(\mathbf{x}_t - \hat{\mu}_j)^T \tag{16}$$

In other words, the prior probabilities simply recover the empirical fractions from the "observed" $j_1, \ldots, j_n$, and each Gaussian component is estimated in the usual way (evaluating the empirical mean and the covariance) based on data points explicitly assigned to that component. So, the estimation of mixture models would be very easy if we knew the assignments $j_1, \ldots, j_n$.

Incomplete data. What changes if we don't know the assignments? We can always guess what the assignments should be based on the current setting of the parameters. Let $\theta^{(l)}$ denote the current (initial) parameters of the mixture distribution. Using these parameters, we can evaluate for each data point x_t the posterior probability that it was generated from component j:

$$P(j|\mathbf{x}_t, \theta^{(l)}) = \frac{P^{(l)}(j)N(\mathbf{x}_t;\, \mu_j^{(l)}, \Sigma_j^{(l)})}{\sum_{j'=1}^{m} P^{(l)}(j')N(\mathbf{x}_t;\, \mu_{j'}^{(l)}, \Sigma_{j'}^{(l)})} = \frac{P^{(l)}(j)N(\mathbf{x}_t;\, \mu_j^{(l)}, \Sigma_j^{(l)})}{P(\mathbf{x}_t; \theta^{(l)})} \tag{17}$$

Instead of using the 0-1 assignments $\delta(j|t)$ of data points to components we can use "soft assignments" $p^{(l)}(j|t) = P(j|\mathbf{x}_t, \theta^{(l)})$. By substituting these in the above closed form estimating equations we get an iterative algorithm for estimating Gaussian mixtures. The algorithm is iterative since the soft posterior assignments were evaluated based on the current setting of the parameters $\theta^{(l)}$ and may have to revised later on (once we have a better idea of where the clusters are in the data).

The resulting algorithm is known as the *Expectation Maximization algorithm* (EM for short) and applies to all mixture models and beyond. For Gaussian mixtures, the EM-algorithm can be written as

> **(Step 0)** Initialize the Gaussian mixture, i.e., specify $\theta^{(0)}$. A simple initialization consists of setting $P^{(0)}(j) = 1/m$, equating each $\mu_j^{(0)}$ with a randomly chosen data point, and making all $\Sigma_j^{(0)}$ equal to the overall data covariance.

(E-step) Evaluate the posterior assignment probabilities $p^{(l)}(j|t) = P(j|\mathbf{x}_t, \theta^{(l)})$ based on the current setting of the parameters $\theta^{(l)}$.

(M-step) Update the parameters according to

$$P^{(l+1)}(j) = \frac{\hat{n}(j)}{n}, \quad \text{where} \quad \hat{n}(j) = \sum_{t=1}^{n} p^{(l)}(j|t) \tag{18}$$

$$\mu_j^{(l+1)} = \frac{1}{\hat{n}(j)} \sum_{t=1}^{n} p^{(l)}(j|t)\mathbf{x}_t \tag{19}$$

$$\Sigma_j^{(l+1)} = \frac{1}{\hat{n}(j)} \sum_{t=1}^{n} p^{(l)}(j|t)(\mathbf{x}_t - \mu_j^{(l+1)})(\mathbf{x}_t - \mu_j^{(l+1)})^T \tag{20}$$

Perhaps surprisingly, this iterative algorithm is guaranteed to converge and each iteration increases the log-likelihood of the data. In other words, if we write

$$l(D; \theta^{(l)}) = \sum_{t=1}^{n} P(\mathbf{x}_t; \theta^{(l)}) \tag{21}$$

then

$$l(D; \theta^{(0)}) < l(D; \theta^{(1)}) < l(D; \theta^{(2)}) < \ldots \tag{22}$$

until convergence. The main downside of this algorithm is that we are only guaranteed to converge to a locally optimal solution where $d/d\theta\, l(D; \theta) = 0$. In other words, there could be a setting of the parameters for the mixture distribution that leads to a higher log-likelihood of the data[1]. For this reason, the algorithm is typically run multiple times (recall the random initialization of the means) so as to ensure we find a reasonably good solution, albeit perhaps only locally optimal.

Example. Consider a simple mixture of two Gaussians. Figure 2 demonstrates how the EM-algorithm changes the Gaussian components after each iteration. The ellipsoids specify one standard deviation distances from the Gaussian mean. The mixing proportions $P(j)$ are not visible in the figure. Note that it takes many iterations for the algorithm to resolve how to properly assign the data points to mixture components. At convergence, the assignments are still soft (not 0-1) but nevertheless clearly divide the responsibilities of the two Gaussian components across the clusters in the data.

[1] In fact, the highest likelihood for Gaussian mixtures is always ∞. This happens when one of the Gaussians concentrates around a single data point. We do not look for such solutions, however, and they can be removed by constraining the covariance matrices or via regularization. The real comparison is to a non-trivial mixture that achieves the highest log-likelihood.

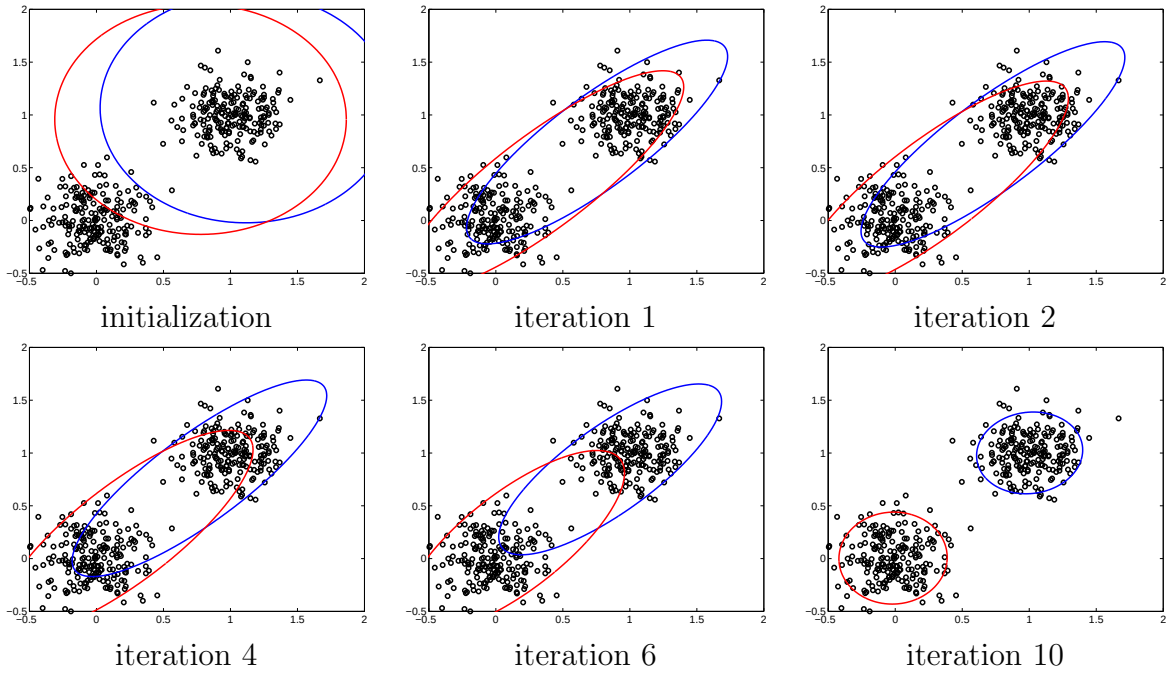


Figure 2: An example of the EM algorithm with a two-component mixture of Gaussians model.

Lecture topics:

- Mixture of Gaussians (cont'd)
- The EM algorithm: some theory
- Additional mixture topics
  - regularization
  - stage-wise mixtures
  - conditional mixtures
- Mixture models and clustering

Mixture of Gaussians (cont'd)

Recall the simple Gaussian mixture model

$$P(\mathbf{x};\theta) = \sum_{j=1}^{m} P(j)N(\mathbf{x};\, \mu_j, \Sigma_j) \tag{1}$$

We have already discussed how to estimate the parameters of such models with the EM algorithm. The E and M steps of the algorithm were:

(E-step) Evaluate the posterior assignment probabilities $p^{(l)}(j|t) = P(j|\mathbf{x}_t, \theta^{(l)})$ based on the current setting of the parameters $\theta^{(l)}$.

(M-step) Update the parameters according to

$$P^{(l+1)}(j) = \frac{\hat{n}(j)}{n}, \text{ where } \hat{n}(j) = \sum_{t=1}^{n} p^{(l)}(j|t) \tag{2}$$

$$\mu_j^{(l+1)} = \frac{1}{\hat{n}(j)} \sum_{t=1}^{n} p^{(l)}(j|t)\mathbf{x}_t \tag{3}$$

$$\Sigma_j^{(l+1)} = \frac{1}{\hat{n}(j)} \sum_{t=1}^{n} p^{(l)}(j|t)(\mathbf{x}_t - \mu_j^{(l+1)})(\mathbf{x}_t - \mu_j^{(l+1)})^T \tag{4}$$

The fact that the EM algorithm is iterative, and can get stuck in locally optimal solutions, means that we have to pay attention to how the parameters are initialized. For example, if we initially set all the components to have identical means and covariances, then they would remain identical even after the EM iterations. In other words, they would be changed in exactly the same way. So, effectively, we would be estimating only a single Gaussian distribution. To understand this, note that the parameter updates (above) are based solely on the posterior assignments. If the parameters of any two components are identical, so will their posterior probabilities. Identical posteriors then lead to identical updates. Note that setting the prior probabilities differently while keeping the component models initialized the same would still lead to this degenerate result.

It is necessary to provide sufficient variation in the initialization step. We can still set the mixing proportions to be uniform, $P^{(0)}(j) = 1/m$, and let all the covariance matrices equal the overall data covariances. But we should randomly position the Gaussian components, e.g., by equating the means with randomly chosen data points. Even with such initialization the algorithm would have to be run multiple times to ensure we will find a good solution. Figure 1 below exemplifies how a particular EM run with four components (with the suggested initialization) can get stuck in a locally optimal solution. The data in the figure came from a four component mixture.

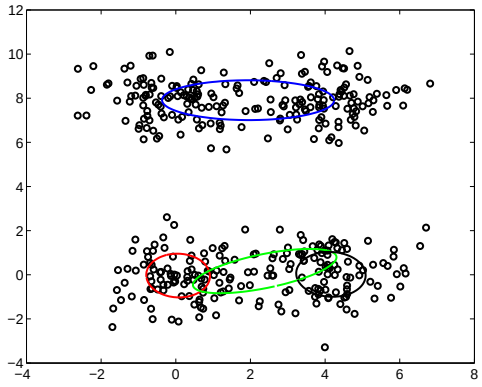


Figure 1: A locally optimal Gaussian mixture with four components

EM theory

Let's understand the EM-algorithm a bit better in a context of a general mixture model of the form

$$P(\mathbf{x};\theta) = \sum_{j=1}^{m} P(j)P(\mathbf{x}|\theta_j) \tag{5}$$

where θ collects together both $\{P(j)\}$ and $\{\theta_j\}$. The goal is the maximize the log-likelihood of data $D = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$

$$l(D;\theta) = \sum_{t=1}^{n} \log P(\mathbf{x}_t;\theta) \tag{6}$$

As before, let's start with a complete data version of the problem and assume that we are given $D = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ as well as the corresponding assignments $\mathcal{J} = \{j_1, \ldots, j_n\}$. The complete log-likelihood of all the observations is given by:

$$l(D, \mathcal{J};\theta) = \sum_{j=1}^{m}\sum_{t=1}^{n} \delta(j|t) \log \Big(P(j)P(\mathbf{x}_t|\theta_j)\Big) \tag{7}$$

where $\delta(j|t) = 1$ if $j = j_t$ and zero otherwise. If $\theta^{(l)}$ denote the current setting of the parameters, then the E-step of the EM-algorithm corresponds to replacing the hard assignments $\delta(j|t)$ with soft posterior assignments $p^{(l)}(j|t) = P(j|\mathbf{x}, \theta^{(l)})$. This replacement step corresponds to evaluating the *expected complete log-likelihood*

$$E\left\{l(D, \mathcal{J};\theta)|D, \theta^{(l)}\right\} = \sum_{j=1}^{m}\sum_{t=1}^{n} p^{(l)}(j|t) \log \Big(P(j)P(\mathbf{x}|\theta_j)\Big) \tag{8}$$

The expectation is over the assignments *given* $D = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ and the current setting of the parameters so that $E\{\delta(j|t)|\mathbf{x}_t, \theta^{(l)}\} = P(j|\mathbf{x}, \theta^{(l)}) = p^{(l)}(j|t)$. The rationale here is that we average over variables whose values we do not know but can guess relative to the current model, completing the incomplete observations. Note that there are two sets of parameters involved in the above expression. The current setting $\theta^{(l)}$ that defined the posterior assignments $p^{(l)}(j|t)$ for completing the data, and θ that we are now free to optimize over. Once we have completed the data, i.e., evaluated the posterior assignments $p^{(l)}(j|t)$, they won't change as a function of θ. The E-step simply casts the incomplete data problem back into a complete data problem. In the M-step of the EM algorithm, we

treat $D = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ and the soft completions $p^{(l)}(j|t)$ as if they were observed data, and maximize the expected log-likelihood with respect to θ while keeping everything else fixed.

Let's relate the EM algorithm a bit more firmly to the goal of maximizing the log-likelihood $l(D;\theta)$. We will show that the EM algorithm is actually optimizing an auxiliary objective that forces the log-likelihood up from below. To this end, let $Q = \{q(j|t)\}$ be any set of distributions over the underlying assignments, not necessarily the posterior assignments $p^{(l)}(j|t)$. The auxiliary objective that the EM algorithm is using is

$$l(D, Q; \theta) = \sum_{j=1}^{m} \sum_{t=1}^{n} q(j|t) \log \Big(P(j) P(\mathbf{x}|\theta_j) \Big) + \sum_{t=1}^{n} H(q(\cdot|t)) \tag{9}$$

where $H(q(\cdot|t)) = -\sum_{j=1}^{m} q(j|t) \log q(j|t)$ is the entropy of the assignment distribution $q(\cdot|t)$. We view $l(D, Q; \theta)$ as a function of Q and θ and have used $q(j|t)$ for the assignment distributions so as to emphasize that they can be set independently of the parameters θ in this objective. The EM algorithm can be seen as an algorithm that alternatingly maximizes $l(D, Q; \theta)$, with respect to Q for fixed θ (E-step) and with respect to θ for a fixed Q (M-step). To make this a bit more precise, we use $\theta^{(l)}$ for the current setting of the parameters and let $Q^{(l)}$ denote the assignment distributions that are really the posteriors, i.e., $q^{(l)}(j|t) = p^{(l)}(j|t) = P(j|\mathbf{x}, \theta^{(l)})$. It is possible to show relatively easily that $l(D, Q; \theta^{(l)})$ attains the maximum with respect to Q exactly when $Q = Q^{(l)}$, i.e., when the assignment distributions are the posteriors $P(j|\mathbf{x}, \theta^{(l)})$. The EM-algorithm is now defined as

(E-step) $Q^{(l)} = \arg\max_Q l(D, Q; \theta^{(l)})$

(M-step) $\theta^{(l+1)} = \arg\max_\theta l(D, Q^{(l)}; \theta)$

The E-step above recovers the posterior assignments, $q^{(l)}(j|t) = p^{(l)}(j|t)$, and the M-step, with these posterior assignments, maximizes

$$l(D, Q^{(l)}; \theta) = \sum_{j=1}^{m} \sum_{t=1}^{n} p^{(l)}(j|t) \log \Big(P(j) P(\mathbf{x}|\theta_j) \Big) + \sum_{t=1}^{n} H(p^{(l)}(\cdot|t)) \tag{10}$$

which is exactly as before since the entropy term is fixed for the purpose of optimizing θ. Since each step in the algorithm is a maximization step, the objective $l(D, Q; \theta)$ has to increase monotonically. This monotone increase is precisely what underlies the monotone increase of the log-likelihood $l(D;\theta)$ in the EM algorithm. Indeed, we claim that our auxiliary objective equals the log-likelihood after any E-step. In other words,

$$l(D, Q^{(l)}; \theta^{(l)}) = l(D; \theta^{(l)}) \tag{11}$$

It is easy (but a bit tedious) to verify this by substituting in the posterior assignments:

$$
\begin{aligned}
l(D, Q^{(l)}; \theta^{(l)}) &= \sum_{j=1}^{m}\sum_{t=1}^{n} p^{(l)}(j|t) \log \left(P^{(l)}(j) P(\mathbf{x}|\theta_j^{(l)}) \right) + \sum_{t=1}^{n} H(p^{(l)}(\cdot|t)) && (12)\\
&= \sum_{j=1}^{m}\sum_{t=1}^{n} p^{(l)}(j|t) \log \left(P^{(l)}(j) P(\mathbf{x}|\theta_j^{(l)}) \right) - \sum_{t=1}^{n}\sum_{j=1}^{m} p^{(l)}(j|t) \log p^{(l)}(j|t) && (13)\\
&= \sum_{j=1}^{m}\sum_{t=1}^{n} p^{(l)}(j|t) \log \frac{P^{(l)}(j) P(\mathbf{x}|\theta_j^{(l)})}{p^{(l)}(j|t)} && (14)\\
&= \sum_{j=1}^{m}\sum_{t=1}^{n} P(j|\mathbf{x}_t, \theta^{(l)}) \log \frac{P^{(l)}(j) P(\mathbf{x}|\theta_j^{(l)})}{P(j|\mathbf{x}_t, \theta^{(l)})} && (15)\\
&= \sum_{j=1}^{m}\sum_{t=1}^{n} P(j|\mathbf{x}_t, \theta^{(l)}) \log P(\mathbf{x}_t; \theta^{(l)}) && (16)\\
&= \sum_{t=1}^{n} \log P(\mathbf{x}_t; \theta^{(l)}) = l(D; \theta^{(l)}) && (17)
\end{aligned}
$$

Note that the entropy term in the auxiliary objective is necessary for this to happen. Now, we are finally ready to state that

$$
\overbrace{l(D, Q^{(l)}; \theta^{(l)})}^{l(D;\theta^{(l)})} \overset{\text{M-step}}{\leq} l(D, Q^{(l)}; \theta^{(l+1)}) \overset{\text{E-step}}{\leq} \overbrace{l(D, Q^{(l+1)}; \theta^{(l+1)})}^{l(D;\theta^{(l+1)})} \overset{\text{M-step}}{\leq} \ldots \qquad (18)
$$

which demonstrates that the EM-algorithm monotonically increases the log-likelihood. The equality above holds only at convergence.

Additional mixture topics: regularization

Regularization plays an important role in the context of any flexible model and this is true for mixtures as well. The number of parameters in an $m-$component mixture of Gaussians model in $d-$dimensions is exactly $m - 1 + md + md(d+1)/2$ and can easily become a problem when d and/or m are large. To include regularization we need to revise the basic EM-algorithm formulated for maximum likelihood estimation. From the point of view of deriving new update equations, the effect of regularization will be the same in the case of complete data (with again the resulting $\delta(j|t)$'s replaced by the posteriors). Let us therefore

simplify the setting a bit and consider regularizing the model when the observations are complete. The key terms to regularize are the covariance matrices.

We can assign a Wishart prior over the covariance matrices. This is a conjugate prior over covariance matrices and will behave nicely in terms of the estimation equations. The prior distribution depends on two parameters, one of which is a common covariance matrix S towards which the estimates of Σ_j's will be pulled. The degree of "pull" is specified by n' known as the *equivalent sample size*. n' specifies the number of data points we would have to observe for the data to influence the solution as much as the prior. More formally, the log-wishart penalty is given by

$$\log P(\Sigma_j|S, n') = \text{const.} - \frac{n'}{2}\text{Trace}(\Sigma_j^{-1}S) - \frac{n'}{2}\log|\Sigma_j| \tag{19}$$

Given data $D = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, and assignments $\mathcal{J} = \{j_1, \ldots, j_n\}$, the penalized complete log-likelihood function is given by

$$\begin{aligned} l(D, \mathcal{J}; \theta) &= \sum_{j=1}^{m}\left(\sum_{t=1}^{n}\delta(j|t)\right)\log P(j) \\ &\quad + \sum_{j=1}^{m}\left(\sum_{t=1}^{n}\delta(j|t)\log N(\mathbf{x}_t;\, \mu_j, \Sigma_j) + \log P(\Sigma_j|S, n')\right) \end{aligned} \tag{20}$$

The solution (steps not provided) changes only slightly and, naturally, only in terms of the covariance matrices[1]:

$$\hat{P}(j) = \frac{\hat{n}(j)}{n}, \quad \text{where} \quad \hat{n}(j) = \sum_{t=1}^{n}\delta(j|t) \tag{21}$$

$$\hat{\mu}_j = \frac{1}{\hat{n}(j)}\sum_{t=1]}^{n}\delta(j|t)\mathbf{x}_t \tag{22}$$

$$\hat{\Sigma}_j = \frac{1}{\hat{n}(j) + n'}\left[\sum_{t=1]}^{n}\delta(j|t)(\mathbf{x}_t - \hat{\mu}_j)(\mathbf{x}_t - \hat{\mu}_j)^T + n'S\right] \tag{23}$$

The resulting covariance updates can be used as part of the EM algorithm simply by replacing $\delta(j|t)$ with the posterior assignments $p^{(l)}(j|t)$, as before. The choice of S depends on the problem but could be, e.g., either the identity matrix or $\hat{\Sigma}$ (the overall data covariance).

[1] Note that the regularization penalty corresponds to having observed n' samples with covariance S from the *same* Gaussian. This update rule is therefore not cast in terms of inverse covariance matrices as it would when combining noisy sources with different noise covariances.

Additional mixture topics: sequential estimation

Mixture models serve two different purposes. They provide efficiently parameterized densities (distributions) over $\mathbf{x}$ but can also help uncover structure in the data, i.e., identify which data points belong to which groups. We will discuss the latter more in the context of clustering and focus here only on predicting $\mathbf{x}$. In this setting, we can estimate mixture models a bit more efficiently in stages, similarly to boosting. In other words, suppose we have already estimated an $m-1$ mixture

$$\hat{P}_{m-1}(\mathbf{x}) = \sum_{j=1}^{m-1} \hat{P}(j) P(\mathbf{x}|\hat{\theta}_j) \tag{24}$$

The component distributions could be Gaussians or other densities or distributions. We can now imagine adding one more component while keeping $\hat{P}_{m-1}(\mathbf{x})$ fixed. The resulting $m-$component mixture can be parameterized as

$$\begin{aligned} P(\mathbf{x}; p_m, \theta_m) &= (1-p_m)\hat{P}_{m-1}(\mathbf{x}) + p_m P(\mathbf{x}|\theta_m) & (25) \\ &= \sum_{j=1}^{m-1} [(1-p_m)\hat{P}(j)] P(\mathbf{x}|\hat{\theta}_j) + p_m P(\mathbf{x}|\theta_m) & (26) \end{aligned}$$

Note that by scaling down $\hat{P}_{m-1}(\mathbf{x})$ with $1-p_m$ we can keep the overall mixture proportions to sum to one regardless of $p_m \in [0,1]$. We can now estimate p_m and θ_m via the EM algorithm. The benefit is that this estimation problem is much simpler than estimating the full mixture; we only have one component to adjust as well as the weight assigned to that component. The EM-algorithm for estimating the component to add is given by

(E-step) Evaluate the posterior assignment probabilities pertaining to the new component:

$$p^{(l)}(m|t) = \frac{p_m^{(l)} P(\mathbf{x}|\theta_m^{(l)})}{(1-p_m^{(l)})\hat{P}_{m-1}(\mathbf{x}) + p_m^{(l)} P(\mathbf{x}|\theta_m^{(l)})} \tag{27}$$

(M-step) Obtain new parameters $p_m^{(l+1)}$ and $\theta_m^{(l+1)}$

$$\begin{aligned} p_m^{(l+1)} &= \frac{\sum_{t=1}^n p^{(l)}(m|t)}{n} & (28) \\ \theta_m^{(l+1)} &= \arg\max_{\theta_m} \left\{ \sum_{t=1}^n p^{(l)}(m|t) \log P(\mathbf{x}_t|\theta_m) \right\} & (29) \end{aligned}$$

Note that in Eq.(29) we maximize the *expected log-likelihood* where the hard assignments $\delta(m|t)$ have been replaced with their expected values, i.e., soft posterior assignments $p^{(l)}(m|t)$. This is exactly how the updates were obtained in the case of Gaussian mixtures as well.

Note that it is often necessary to regularize the mixture components even if they are added sequentially.

Additional mixture topics: conditional mixtures

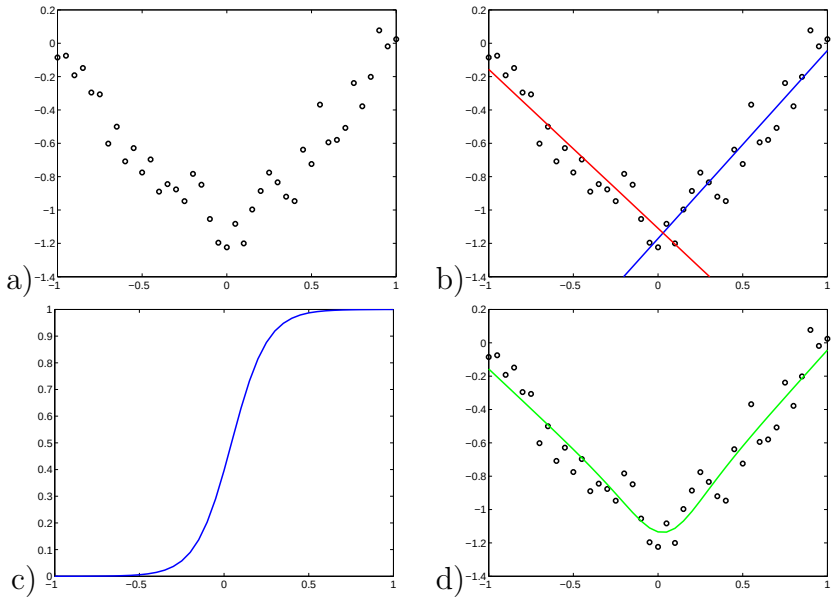


Figure 2: a) A regression problem, b) linear regression models (experts) and their assigned regions, c) gating network output (probability that we select the blue expert on the right), d) the mean output from the conditional mixture distribution as a function of the input.

We can also use mixture models for regression. These are known as *conditional mixtures* or *mixtures of experts models*. Consider a simple linear regression model

$$P(y|\mathbf{x}, \theta, \sigma^2) = N(y; \theta^T \mathbf{x}, \sigma^2) \tag{30}$$

The goal is to predict the responses as a function of the input $\mathbf{x}$. In many cases the responses are not linear functions of the input (see Figure 2a). This leaves us with a few choices. We

could try to find a feature mapping $\phi(\mathbf{x})$ so as to ensure that a linear function in the feature space can capture the non-linearities that relate $\mathbf{x}$ and y. It may not be always clear what the feature vector should be, however (in case of Figure 2a the relevant feature mapping would be $\phi(x) = |x|$). Another approach is to divide the input space into regions such that within each region the relation between $\mathbf{x}$ and y can be captured with a linear function. Such a model requires us to be able to predict how to assign linear functions (experts) to regions. In conditional mixtures, this assignment is carried out by another model, so called *gating network*. The gating network could, for example, be a logistic regression model

$$P(j = 1|\mathbf{x}, \eta, \eta_0) = g(\, \eta^T \mathbf{x} + \eta_0 \,) \tag{31}$$

and $P(j = 2|\mathbf{x}, \eta, \eta_0) = 1 - g(\, \eta^T \mathbf{x} + \eta_0 \,)$ (we would have to use a *softmax* function for dividing the space into m regions). The overall model for the responses y given $\mathbf{x}$ would then be a conditional mixture

$$P(y|\mathbf{x}) = \sum_{j=1}^{2} P(j|\mathbf{x}, \eta, \eta_0) N(y; \theta_j^T \mathbf{x}, \sigma_j^2) \tag{32}$$

Note that the real difference between this and a simple mixture is that the mixing proportions as well as the component models are conditioned on the input. Such models can be again estimated via the EM-algorithm:

(E-step) Evaluate the posterior assignment probabilities:

$$p^{(l)}(j|t) = \frac{P(j|\mathbf{x}_t, \eta^{(l)}, \eta_0^{(l)}) N(y_t; (\theta^{(l)})^T \mathbf{x}_t, (\sigma^{(l)})^2)}{P^{(l)}(y_t|\mathbf{x}_t)} \tag{33}$$

Note that the assignments are based on both $\mathbf{x}_t$ and y_t.

(M-step) Obtain new parameters by maximizing the expected log-likelihoods relative to the posterior assignments:

$$\{\eta^{(l+1)}, \eta_0^{(l+1)}\} = \arg\max_{\eta, \eta_0} \sum_{j=1}^{2} \sum_{t=1}^{n} p^{(l)}(j|t) \log P(j|\mathbf{x}_t, \eta, \eta_0) \tag{34}$$

$$\{\theta_j^{(l+1)}, \sigma_j^{(l+1)}\} = \arg\max_{\theta_j, \sigma_j} \sum_{t=1}^{n} p^{(l)}(j|t) \log N(y_t; \theta_j^T \mathbf{x}_t, \sigma_j^2) \tag{35}$$

These estimation equations are readily obtained from the complete log-likelihood version pertaining to the different conditional probabilities and by replacing $\delta(j|t)$ with the posterior assignments.

Figure 2 illustrates the resulting mixture of experts model in the toy problem.

Mixture models and clustering

We have so far used mixture models as flexible ways of constructing probability models for prediction tasks. The motivation behind the mixture model was that the available data may include unobserved (sub)groups and by incorporating such structure in the model we could obtain more accurate predictions. We are also interested in uncovering that group structure. This is a *clustering* problem. For example, in the case of modeling exam scores it would be useful to understand what types of students there are. In a biological context, it would be useful to uncover which genes are active (expressed) in which cell types when the measurements are from tissue samples involving multiple cell types in unknown quantities. Clustering problems are ubiquitous.

Mixture models as generative models require us to articulate the type of clusters or subgroups we are looking to identify. The simplest type of clusters we could look for are spherical Gaussian clusters, i.e., we would be estimating Gaussian mixtures of the form

$$P(\mathbf{x}; \theta, m) = \sum_{j=1}^{m} P(j) N(\mathbf{x}; \mu_j, \sigma_j^2 I) \tag{36}$$

where the parameters θ include $\{P(j)\}$, $\{\mu_j\}$, and $\{\sigma_j^2\}$. Note that we are estimating the mixture models with a different objective in mind. We are more interested in finding where the clusters are than how good the mixture model is as a generative model.

There are many questions to resolve. For example, how many such spherical Gaussian clusters are there in the data? This is a model selection problem. If such clusters exist, do we have enough data to identify them? If we have enough data, can we hope to find the clusters via the EM algorithm? Is our approach robust, i.e., does our method degrade gracefully when data contain "background samples" or impurities along with spherical Gaussian clusters? Can we make the clustering algorithm more robust?

Lecture topics:

- Mixture models and clustering, k-means
- Distance and clustering

Mixture models and clustering

We have so far used mixture models as flexible ways of constructing probability models for prediction tasks. The motivation behind the mixture model was that the available data may include unobserved (sub)groups and by incorporating such structure in the model we could obtain more accurate predictions. We are also interested in uncovering that group structure. This is a *clustering* problem. For example, in the case of modeling exam scores it would be useful to understand what types of students there are. In a biological context, it would be useful to uncover which genes are active (expressed) in which cell types when the measurements are from tissue samples involving multiple cell types in unknown quantities. Clustering problems are ubiquitous.

There are many different types of clustering algorithms. Some generate a series of nested clusters by merging simple clusters into larger ones (hierarchical agglomerative clustering), while others try to find a pre-specified number of clusters that best capture the data (e.g., k-means). Which algorithm is the most appropriate to use depends in part on what we are looking for. So it is very useful to know more than one clustering method.

Mixture models as generative models require us to articulate the type of clusters or subgroups we are looking to identify. The simplest type of clusters we could look for are spherical Gaussian clusters, i.e., we would be estimating Gaussian mixtures of the form

$$P(\mathbf{x}; \theta, m) = \sum_{j=1}^{m} P(j) N(\mathbf{x}; \mu_j, \sigma_j^2 I) \tag{1}$$

where the parameters θ include $\{P(j)\}$, $\{\mu_j\}$, and $\{\sigma_j^2\}$. Note that we are estimating the mixture models with a different objective in mind. We are more interested in finding where the clusters are than how good the mixture model is as a generative model.

There are many questions to resolve. For example, how many such spherical Gaussian clusters are there in the data? This is a model selection problem. If such clusters exist, do we have enough data to identify them? If we have enough data, can we hope to find the clusters via the EM algorithm? Is our approach robust, i.e., does our method degrade

gracefully when data contain "background samples" or impurities along with spherical Gaussian clusters? Can we make the clustering algorithm more robust? Similar questions apply to other clustering algorithms. We will touch on some of these issues in the context of each algorithm.

Mixtures and K-means

It is often helpful to try to search for simple clusters. For example, consider a mixture of two Gaussians where the variances of the spherical Gaussians may be different:

$$P(\mathbf{x};\theta) = \sum_{j=1}^{2} P(j)N(\mathbf{x};\mu_j,\sigma_j^2 I) \tag{2}$$

Points far away from the two means, in whatever direction, would always be assigned to the Gaussian with the larger variance (tails of that distribution approach zero much slower; the posterior assignment is based on the ratio of probabilities). While this is perfectly fine for modeling the density, it does not quite agree with the clustering goal.

To avoid such issues (and to keep the discussion simpler) we will restrict the covariance matrices to be all identical and equal to $\sigma^2 I$ where σ^2 is common to all clusters. Moreover, we will fix the mixing proportions to be uniform $P(j) = 1/m$. With such restrictions we can more easily understand the type of clusters we will discover. The resulting simplified mixture model has the form

$$P(\mathbf{x};\theta) = \frac{1}{m}\sum_{j=1}^{m} N(\mathbf{x};\mu_j,\sigma^2 I) \tag{3}$$

Let's begin by trying to understand how points are assigned to clusters within the EM algorithm. To this end, we can see how the mixture model partitions the space into regions where within each region a specific component has the highest posterior probability. To start with, consider any two components i and j and the boundary where $P(i|\mathbf{x},\theta) = P(j|\mathbf{x},\theta)$, i.e., the set of points for which the component i and j have the same posterior probability. Since the Gaussians have equal prior probabilities, this happens when $N(\mathbf{x};\mu_j,\sigma^2 I) = N(\mathbf{x};\mu_i,\sigma^2 I)$. Both Gaussians have the same spherical covariance matrix so the probability that they assign to points is based on the Euclidean distances to their mean vectors. The posteriors therefore can be equal only when the component means are equidistant from the points:

$$\|\mathbf{x}-\mu_i\|^2 = \|\mathbf{x}-\mu_j\|^2 \quad \text{or} \quad 2\mathbf{x}^T(\mu_j-\mu_i) = \|\mu_j\|^2 - \|\mu_j\|^2 \tag{4}$$

The boundary is therefore linear[1] in $\mathbf{x}$. We can draw such boundaries between any pair of components as illustrated in Figure 1. The pairwise comparisons induce a *Voronoi* partition of the space where, for example, the region enclosing μ_1 in the figure corresponds to all the points whose closest mean is μ_1. This is also the region where $P(1|\mathbf{x}, \theta)$ takes the highest value among all the posterior probabilities.

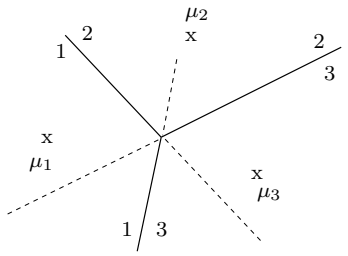


Figure 1: Voronoi regions resulting from pairwise comparison of posterior assignment probabilities. Each line corresponds to a decision between posterior probabilities of two components. The line is highlighted (solid) when the boundary is an active constraint for deciding the highest probability component.

Note that the posterior assignment probabilities $P(j|\mathbf{x}, \theta)$ evaluated in the E-step of the EM algorithm do not vanish across the boundaries. The regions merely highlight when one assignment has a higher probability than the others. The overall variance parameter σ^2 controls how sharply the posterior probabilities change when we cross each boundary. Small σ^2 results in very sharp transitions (e.g., from near zero to nearly one) while the posteriors change smoothly when σ^2 is large.

K-means. We can define a simpler and faster version of the EM algorithm by just assigning each point to the component with the highest posterior probability (its closest mean). Geometrically, the points within each Voronoi region are assigned to one component. The M-step then simply repositions each mean vector to the center of the points within the region. The updated means define new Voronoi regions and so on. The resulting algorithm for finding cluster means is known as the *K-means* algorithm:

E-step: assign each point $\mathbf{x}_t$ to its closest mean, i.e., $j_t = \arg\min_j \|\mathbf{x}_t - \mu_j\|^2$

M-step: recompute μ_j's as means of the assigned points.

[1] The linearity holds when the two Gaussians have the same covariance matrix, spherical or not.

This is a highly popular hard assignment version of the EM-algorithm. We can view the algorithm as optimizing the complete log-likelihood of the data with respect to the assignments $j_1, \ldots, j_n$ (E-step) as well as the means $\mu_1, \ldots, \mu_m$ (M-step). This is equivalent to minimizing the overall squared error (distortion) to the cluster means:

$$J(j_1, \ldots, j_n, \mu_1, \ldots, \mu_m) = \sum_{t=1}^{n} \|\mathbf{x}_t - \mu_{j_t}\|^2 \tag{5}$$

Each step of the algorithm, E or M-step, decreases the objective until convergence. The algorithm can never come back to the same assignments $j_1, \ldots, j_n$ as it would result in the same value of the objective. Since there are only $\mathcal{O}(n^k)$ possible partitions of points with k means, the algorithm has to converge in a finite number of steps. At convergence, the cluster means $\hat{\mu}_1, \ldots, \hat{\mu}_m$ are locally optimal solutions to the minimum distortion objective

$$J(\hat{\mu}_1, \ldots, \hat{\mu}_m) = \sum_{t=1}^{n} \min_j \|\mathbf{x}_t - \hat{\mu}_j\|^2 \tag{6}$$

The speed of the K-means algorithm comes at a cost. It is even more sensitive to proper initialization than a mixture of Gaussians trained with EM. A typical and reasonable initialization corresponds to setting the first mean to be a randomly selected training point, the second as the furthest training point from the first mean, the third as the furthest training point from the two means, and so on. This initialization is a greedy *packing* algorithm.

Identifiability. The k-means or mixture of Gaussians with the EM algorithm can only do as well as the maximum likelihood solution they aim to recover. In other words, even when the underlying clusters are spherical, the number of training examples may be too small for the globally optimal (non-trivial) maximum likelihood solution to provide any reasonable clusters. As the number of points increases, the quality of the maximum likelihood solution improves but may be hard to find with the EM algorithm. A large number of training points also makes it easier to find the maximum likelihood solution. We could therefore roughly speaking divide the clustering problem into three regimes depending on the number of training points: not solvable, solvable but hard, and easy. For a recent discussion on such regimes, see Srebro et al., "An Investigation of Computational and Informational Limits in Gaussian Mixture Clustering", ICML 2006.

Distance and clustering

Gaussian mixture models and the K-means algorithm make use of the Euclidean distance between points. Why should the points be compared in this manner? In many cases the vector representation of objects to be clustered is derived, i.e., comes from some feature transformation. It is not at all clear that the Euclidean distance is an appropriate way of comparing the resulting feature vectors.

Consider, for example, a document clustering problem. We might treat each document $\mathbf{x}$ as a bag of words and map it into a feature vector based on term frequencies:

$$n_w(\mathbf{x}) = \text{number of times word } w \text{ appears in } \mathbf{x} \tag{7}$$

$$f(w|\mathbf{x}) = \frac{n_w(\mathbf{x})}{\sum_{w'} n_{w'}(\mathbf{x})} = \text{term frequency} \tag{8}$$

$$\phi_w(\mathbf{x}) = f(w|\mathbf{x}) \cdot \overbrace{\log\left[\frac{\#\text{ of docs}}{\#\text{ of docs with word } w}\right]}^{\text{IDF}} \tag{9}$$

where the feature vector $\phi(\mathbf{x})$ is known as the *TFIDF* mapping (there are many variations of this). IDF stands for *inverse document frequency* and aims to de-emphasize words that appear in all documents, words that are unlikely to be useful for clustering or classification. We can now interpret the vectors $\phi(\mathbf{x})$ as points in a Euclidean space and apply, e.g., the K-means clustering algorithm. There are, however, many other ways of defining a distance metric for clustering documents.

Distance metric plays a central role in clustering, regardless of the algorithm. For example, a simple hierarchical agglomerative clustering algorithm is defined almost entirely on the basis of the distance function. The algorithm proceeds by successively merging two closest points or closest clusters (average squared distance) and is illustrated in Figure 2.

In solving clustering problems, the focus should be on defining the distance (similarity) metric, perhaps at the expense of a specific algorithm. Most algorithms could be applied with the chosen metric. One avenue for defining a distance metric is through model selection. Let's see how this can be done. The simplest model over the words in a document is a *unigram* model where each word is an independent sample from a multi-nomial distribution $\{\theta_w\}$, $\sum_w \theta_w = 1$. The probability of all the words in document $\mathbf{x}$ is therefore

$$P(\mathbf{x}|\theta) = \prod_{w \in \mathbf{x}} \theta_w = \prod_w \theta_w^{n_w(\mathbf{x})} \tag{10}$$

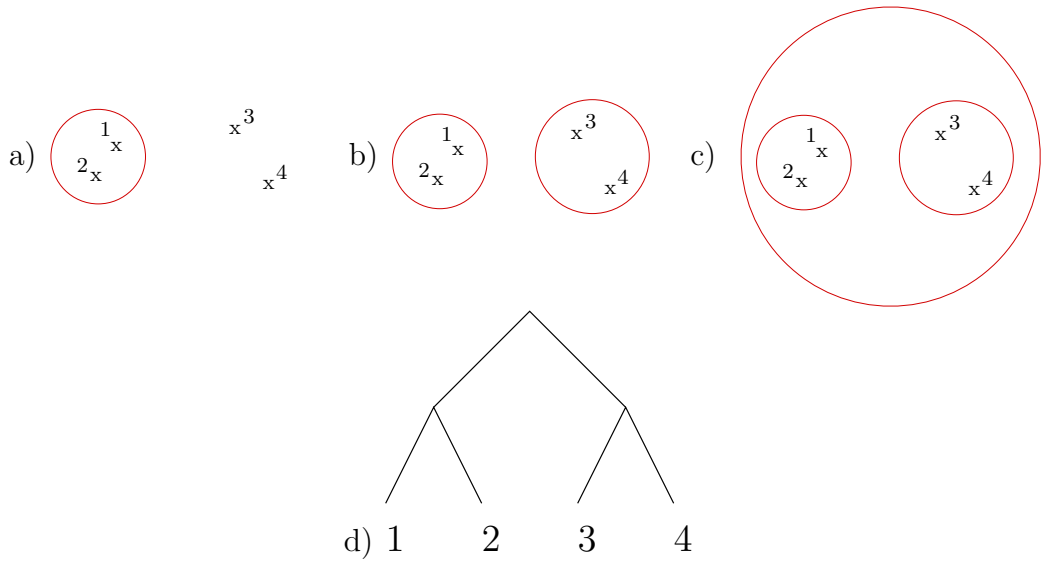


Figure 2: a-c) successive merging of clusters in a hierarchical clustering algorithm and d) the resulting cluster hierarchy.

It is often a good idea to normalize the documents so they have the same length. We could, for example, say that each document is of length one and word "counts" are given by term frequencies $f(w|\mathbf{x})$. Accordingly,

$$P(\tilde{\mathbf{x}}|\theta) = \prod_w \theta_w^{f(w|\mathbf{x})} \tag{11}$$

where $\tilde{\mathbf{x}}$ denotes a normalized version of document $\mathbf{x}$. Consider a cluster of documents C. The unigram model that best describes the normalized documents in the cluster can be obtained by maximizing the log-likelihood

$$l(C;\theta) = \sum_{\mathbf{x}\in C} \log P(\tilde{\mathbf{x}}|\theta) = \sum_{\mathbf{x}\in C}\sum_w f(w|\mathbf{x}) \log \theta_w \tag{12}$$

The maximum likelihood solution is, as before, obtained through empirical counts (represented here by term frequencies):

$$\hat{\theta}_w = \frac{1}{|C|}\sum_{\mathbf{x}\in C} f(w|\mathbf{x}) \tag{13}$$

The resulting log-likelihood value is

$$
\begin{aligned}
l(C;\hat{\theta}) &= \sum_{\mathbf{x}\in C}\sum_{w} f(w|\mathbf{x})\log\hat{\theta}_w && (14)\\
&= |C|\sum_{w}\frac{1}{|C|}\sum_{\mathbf{x}\in C} f(w|\mathbf{x})\log\hat{\theta}_w && (15)\\
&= |C|\sum_{w}\hat{\theta}_w\log\hat{\theta}_w = -|C|H(\hat{\theta}) && (16)
\end{aligned}
$$

where $H(\hat{\theta})$ is the entropy of the unigram distribution.

We can now proceed to define a distance corresponding to the unigram model. Consider any two clusters C_i and C_j and their combination $C = C_i \cup C_j$. When C_i and C_j are treated as distinct clusters, we will use a different unigram model to capture their term frequencies. The combined cluster C, on the other hand, is modeled with a single unigram model. We can now treat the documents in the two clusters as observations and devise a model selection problem for deciding whether the clusters should be viewed as separate or merged. To this end, we will evaluate the resulting log-likelihoods in two ways corresponding to whether the clusters are modeled as separate or combined. When separate:

$$
\begin{aligned}
l(C_i|\hat{\theta}_{\cdot|i}) + l(C_j|\hat{\theta}_{\cdot|j}) &= -|C_i|H(\hat{\theta}_{\cdot|i}) - |C_j|H(\hat{\theta}_{\cdot|j}) && (17)\\
&= -(|C_i| + |C_j|)\sum_{y\in\{i,j\}}\hat{P}(y)H(\hat{\theta}_{\cdot|y}) && (18)
\end{aligned}
$$

where $\hat{P}(y=i) = |C_i|/(|C_i| + |C_j|)$ is the probability that a randomly drawn document from $C = C_i \cup C_j$ is from cluster C_i. The parameter estimates $\hat{\theta}_{w|i}$ and $\hat{\theta}_{w|j}$ are obtained as before. Similarly, when the two clusters are modeled with the same unigram:

$$
l(C_i, C_j|\hat{\theta}) = -(|C_i| + |C_j|)H(\hat{\theta}) \qquad (19)
$$

where the parameter estimate $\hat{\theta}_w$ can be written as $\hat{\theta}_w = \sum_{y\in\{i,j\}}\hat{P}(y)\hat{\theta}_{w|y}$, i.e., as a cluster size weighted combination of the estimates from the individual clusters.

We can now define a (squared) distance between C_i and C_j according to how much better they are modeled as separate clusters (log-likelihood ratio statistic):

$$
\begin{aligned}
d^2(C_i, C_j) &= l(C_i|\hat{\theta}_{\cdot|i}) + l(C_j|\hat{\theta}_{\cdot|j}) - l(C_i, C_j|\hat{\theta}) && (20)\\
&= (|C_i| + |C_j|)\left[H(\hat{\theta}) - \sum_{y\in\{i,j\}}\hat{P}(y)H(\hat{\theta}_{\cdot|y})\right] && (21)\\
&= (|C_i| + |C_j|)\,\hat{I}(w;y) && (22)
\end{aligned}
$$

where $\hat{I}(w; y)$ is the mutual information between words w sampled from the combined cluster C and the identity of the cluster (i or j). (Recall a similar derivation in the feature selection context). More precisely, the mutual information is computed on the basis of $\hat{P}(w, y) = \hat{P}(y)\hat{\theta}_{w|y}$. Note that $d^2(C_i, C_j)$ is symmetric and non-negative but need not satisfy all the properties of a metric. It nevertheless compares the two clusters in a very natural way: we measure how distinguishable they are based on their word distributions. In other words, $\hat{I}(w; y)$ tells us how much a word sampled at random from a document in the combined cluster C tells us about the cluster C_i or C_j that the sample came from. Low information content means that the two clusters have nearly identical distributions over words.

The (squared) metric $d^2(C_i, C_j)$ can be immediately used, e.g., in a hierarchical clustering algorithm (the same algorithm derived from a different perspective is known as an agglomerative information bottleneck method). We could take the model selection idea further and decide when to stop merging clusters rather than simply providing a scale for deciding how different two clusters are.

Lecture topics:

- Spectral clustering, random walks and Markov chains

Spectral clustering

Spectral clustering refers to a class of clustering methods that approximate the problem of partitioning nodes in a weighted graph as eigenvalue problems. The weighted graph represents a similarity matrix between the objects associated with the nodes in the graph. A large positive weight connecting any two nodes (high similarity) biases the clustering algorithm to place the nodes in the same cluster. The graph representation is relational in the sense that it only holds information about the comparison of objects associated with the nodes.

Graph construction

A relational representation can be advantageous even in cases where a vector space representation is readily available. Consider, for example, the set of points in Figure 1a. There appears to be two clusters but neither cluster is well-captured by a small number of spherical Gaussians. By connecting each point to their two nearest neighbors (two closest points) yields a graph in Figure 1b that places the two clusters in different connected components. While typically the weighted graph representation would have edges spanning across the clusters, the example nevertheless highlights the fact that the relational representation can potentially be used to identify clusters whose form would make them otherwise difficult to find. This is particularly the case when the points lie on a lower dimensional surface (manifold). The weighted graph representation can essentially perform the clustering along the surface rather than in the enclosing space.

How exactly do we construct the weighted graph? The problem is analogous to the choice of distance function for hierarchical clustering and there are many possible ways to do this. A typical way, alluded to above, starts with a $k-$nearest neighbor graph, i.e., we construct an undirected graph over the n points such that i and j are connected if either i is among the k nearest neighbors of j or vice versa (nearest neighbor relations are not symmetric). Given the graph, we can then set

$$W_{ij} = \begin{cases} \exp(-\beta \|\mathbf{x}_i - \mathbf{x}_j\|) & \text{if } i \text{ and } j \text{ connected} \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

The resulting weights (similarities) are symmetric in the sense that $W_{ij} = W_{ji}$. All the

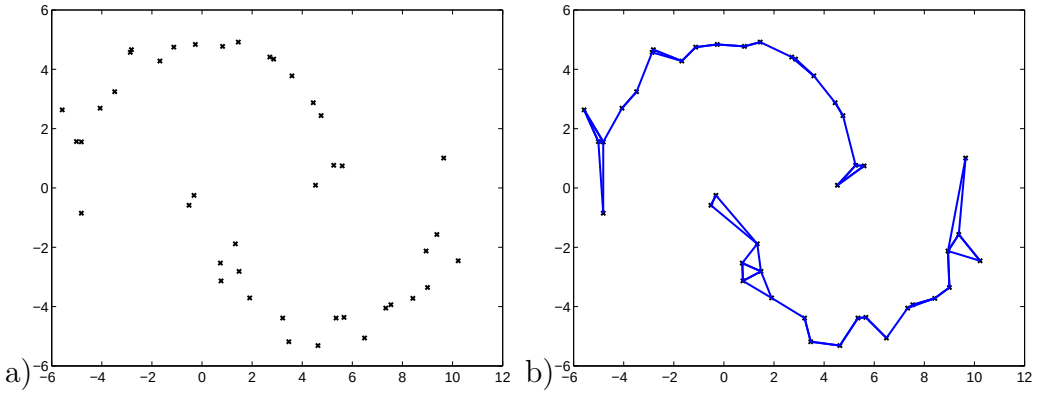


Figure 1: a) a set of points and b) the corresponding 2-nearest neighbor graph.

diagonal entries are set to zero: $W_{ii} = 0$ for $i = 1, \ldots, n$. The $n \times n$ matrix W now represents the weighted graph.

There are two parameters to set: k and β. The choice of k is tied to the dimensionality of the clusters we are trying to find. For example, if we believe that the clusters look like $d-$dimensional surfaces, then k should be at least d. A small k leads to a sparse graph and serves to limit the comparisons between points to those that are close. This is advantageous since the Euclidean distance is unlikely to be reasonable for points far away. For example, consider points on the surface of a unit sphere, and a context where their distance really should be measured along the surface. The simple Euclidean distance nevertheless provides a reasonable approximation for points that are close on the surface. β serves a similar role but, unlike k, is tied to the actual scale of the points (their distances).

Graph partitioning and criteria

Let's now define the clustering problem more formally. Suppose we have n objects to be clustered into two groups (binary partition). A multi-way partition can be obtained through a recursive application of binary partitioning. The objects are represented by a weighted graph with symmetric positive weights $W_{ij} = W_{ji} \geq 0$, W_{ij} is zero when no edge is present between i and j, and $W_{ii} = 0$. The goal is to use the weighted graph as a similarity measure to partition the nodes into two disjoint groups C^+ and C^- such that $C^+ \cup C^- = \{1, \ldots, n\}$. Any such partition corresponds to a labeling of the nodes in the graph with binary labels $y_i \in \{-1, 1\}$ such that, e.g., $y_i = 1$ when $i \in C^+$. The clusters can be therefore equivalently specified by the sets (C^+, C^-) or the labeling y (a binary vector of length n).

It remains to specify an objective function for finding the clusters. Each binary clustering is associated with a *cut* in the graph. The weight of the cut is given by

$$s(C^+, C^-) = \sum_{i \in C^+, j \in C^-} W_{ij} = \frac{1}{4} \sum_{i,j} W_{ij}(y_i - y_j)^2 = J(y) \tag{2}$$

where $(y_i - y_j)^2 = 4$ when y_i and y_j differ and zero otherwise. The cut simply corresponds to adding the weights of the edges connecting nodes that are in different clusters (labeled differently). The value of the cut is obviously zero if all the nodes are labeled the same. If we require both labels to be present we arrive at a *minimum cut* criterion. It is actually efficient to find the labeling or, equivalently, sets C^+ and C^- that minimize the value of the cut under this constraint. The approach does not work well as a clustering algorithm, however, as it tends to simply identify outliers as clusters (individual points weakly connected to others). We will have to modify the objective to find more balanced clusters.

A better criterion is given by so called *normalized cut* (see Shi and Malik 2000):

$$\text{Norm-cut}(C^+, C^-) = \frac{s(C^+, C^-)}{s(C^+, C^+) + s(C^+, C^-)} + \frac{s(C^+, C^-)}{s(C^-, C^-) + s(C^+, C^-)} \tag{3}$$

where, e.g., $s(C^+, C^+) = \sum_{i \in C^+, j \in C^+} W_{ij}$, the sum of weights between nodes in cluster C^+. Each term in the criterion is a ratio of the weight of the cut to the total weight associated with the nodes in the cluster. In other words, it is the fraction of weight tied to the cut. This normalization clearly prohibits us from separating outliers from other nodes. For example, an outlier connected to only one other node with a small weight cannot form a single cluster as the fraction of weight associated with the cut would be 1, the highest possible value of the ratio. So we can expect the criterion to yield more balanced partitions. Unfortunately, we can no longer find the solution efficiently (it is an integer programming problem). An approximate solution can be found by relaxing the optimization problem into an eigenvalue problem.

Spectral clustering, the eigenvalue problem

We begin by extending the "labeling" over the reals $z_i \in \mathcal{R}$. We will still interpret the sign of the real number z_i as the cluster label. This is a relaxation of the binary labeling problem but one that we need in order to arrive at an eigenvalue problem. First, let's

rewrite the cut as

$$J(z) = \frac{1}{4}\sum_{i,j} W_{ij}(z_i - z_j)^2 = \frac{1}{4}\sum_{i,j} W_{ij}(z_i^2 - 2z_i z_j + z_j^2) = \frac{1}{4}\sum_{i,j} W_{ij}(2z_i^2 - 2z_i z_j) \quad (4)$$

$$= \frac{1}{2}\sum_i \overbrace{\left(\sum_j W_{ij}\right)}^{D_{ii}} z_i^2 + \frac{1}{2}\sum_{i,j} W_{ij} z_i z_j = \frac{1}{2}\, z^T(D - W)z \quad (5)$$

where we have used the symmetry of the weights. D is a diagonal matrix with elements $D_{ii} = \sum_j W_{ij}$. The matrix $L = D - W$ is known as the *graph Laplacian* and is guaranteed to be positive semi-definite (all the eigenvalues are non-negative). The smallest eigenvalue of the Laplacian is always exactly zero and corresponds to a constant eigenvector $z = \mathbf{1}$.

We will also have to take into account the normalization terms in the normalized cut objective. A complete derivation is a bit lengthy (described in the Shi and Malik paper available on the website). So we will just motivate here how to get to the relaxed version of the problem. Now, the normalized cut objective tries to balance the overall weight associated with the nodes in the two clusters, i.e., $s(C^+, C^+) + s(C^+, C^-) \approx s(C^-, C^-) + s(C^+, C^-)$. In terms of the labels, the condition for exactly balancing the weights would be $y^T D\mathbf{1} = 0$. We will instead use a relaxed criterion $z^T D\mathbf{1} = 0$. Moreover, now that z_i's are real numbers and not binary labels, we will have to normalize them so as to avoid $z_i \to 0$. We can do this by requiring that $z^T Dz = 1$. As a result, small changes in z_i for nodes that are strongly connected to others ($D_{ii} = \sum_j W_{ij}$ is large) require larger compensating changes at nodes that are only weakly coupled to others. This helps ensure that isolated nodes become "followers".

The resulting relaxed optimization problem is given by

$$\text{minimize} \quad \frac{1}{2} z^T(D - W)z \quad \text{subject to} \quad z^T Dz = 1,\ z^T D\mathbf{1} = 0 \quad (6)$$

The solution can be found easily via Lagrange multipliers and reduces to finding the eigenvector z_2 (components z_{2i}) corresponding to the second smallest eigenvalue from

$$(D - W)z = \lambda Dz \ \text{ or, equivalently, } \ (I - D^{-1}W)z = \lambda z \quad (7)$$

The eigenvector with the smallest ($\lambda = 0$) eigenvalue is always the constant vector $z = \mathbf{1}$. This would not satisfy $z^T D\mathbf{1} = 0$ but the second smallest eigenvector does. Note that since the goal is to minimize $z^T(D - W)z$ we are interested in only the eigenvectors with small eigenvalues. The clusters can now be found by labeling the nodes according to

$\hat{y}_i = \text{sign}(z_{2i})$. If we wish to further balance the number of nodes in each cluster, we could sort the components of z_2 in the ascending order and label nodes as negative in this order. Figure 2 illustrates a possible solution and the corresponding values of the eigenvector.

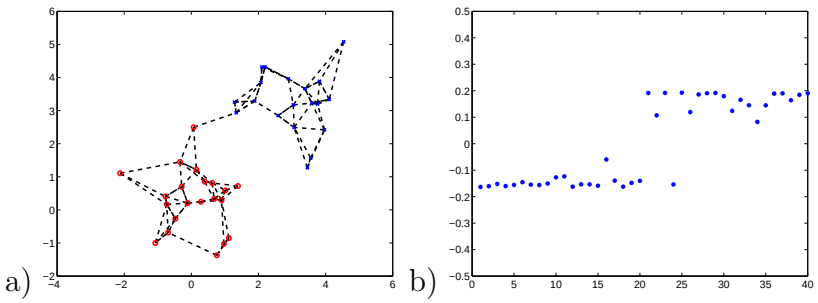


Figure 2: a) spectral clustering solution and b) the values of the second largest eigenvector.

Spectral clustering, random walk

The relaxed optimization problem is an approximate solution to the normalized cut problem. It is therefore not immediately clear that this approximate solution behaves appropriately. We can try to justify it from a very different perspective, that of random walks on the weighted graph. To this end, note that the eigenvectors we get by solving $(I - D^{-1}W)z = \lambda z$ are exactly the same as those obtained from $D^{-1}Wz = \lambda' z$. The resulting eigenvalues are also in one to one correspondence: $\lambda' = 1 - \lambda$. Thus the constant eigenvector $z = \mathbf{1}$ with $\lambda = 0$ should have $\lambda' = 1$ and satisfy $D^{-1}Wz = z$. Let's understand this further. Define

$$P_{ij} = \frac{W_{ij}}{\sum_{j'} W_{ij'}} = \frac{W_{ij}}{D_{ii}} \tag{8}$$

so that $P = D^{-1}W$. Clearly, $\sum_j P_{ij} = 1$ for all i so that $P\mathbf{1} = \mathbf{1}$. We can therefore interpret P as a *transition probability matrix* associated with the nodes in the weighted graph. In other words, P_{ij} defines a random walk where we hop from node i to node j with probability P_{ij}. If $X(t)$ denotes the node we happen to be at time t, then

$$P(X(t+1) = j | X(t) = i) = P_{ij} \tag{9}$$

Our random walk corresponds to a *homogeneous Markov chain* since the transition probabilities remain the same every time we come back to a node (i.e., the transition probabilities are not time dependent). Markov chains are typically defined in terms of states and transitions between them. The states in our case are the nodes in the graph.

In a Markov chain two states i and j are said to be *communicating* if you can get from i to j and from j to i with finite probability. If all the pairs of states (nodes) are communicating, then the Markov chain is *irreducible*. Note that a random walk defined on the basis of the graph in Figure 1b would not be irreducible since the nodes across the two connected components are not communicating.

It is often useful to write a *transition diagram* that specifies all the permissible one-step transitions $i \to j$, those corresponding to $P_{ij} > 0$. This is usually a directed graph. However, in our case, because the weights are symmetric, if you can directly transition from i to j then you can also go directly from j to i. The transition diagram therefore reduces to the undirected graph (or directed graph where each undirected edge is directed both ways). Note that the transition probabilities themselves are not symmetric as the normalization terms D_{ii} vary from node to node. On the other hand, the zeros (prohibited one-step transitions) do appear in symmetric places in the matrix P_{ij}.

We need to understand one additional property of (some) Markov chains – *ergodicity*. To this end, let us consider one-step, two-step, and $m-$step transition probabilities:

$$P(X(t+1) = j|X(t) = i) = P_{ij} \tag{10}$$

$$P(X(t+2) = j|X(t) = i) = \sum_k P_{ik}P_{kj} = [PP]_{ij} = [P^2]_{ij} \tag{11}$$

$$\cdots \tag{12}$$

$$P(X(t+m) = j|X(t) = i) = [P^m]_{ij} \tag{13}$$

where $[P^m]_{ij}$ is the i, j element of the matrix $PP \cdots P$ (m multiplications). A Markov chain is *ergodic* if there is a finite m such that for this m (and all larger values of m)

$$P(X(t+m) = j|X(t) = i) > 0, \text{ for all } i \text{ and } j \tag{14}$$

In other words, we have to be able to get form any state to any other state with finite probability after m transitions. Note that this has to hold for the same m. For example, a Markov chain with three states and possible transitions $1 \to 2 \to 3 \to 1$ is not ergodic even though we can get from any state to any other state. However, for this Markov chain, any m step transition probability matrix would still have prohibited transitions. For example, starting from 1, after three steps we can only be back in 1.

Now, what will happen if we let $m \to \infty$, i.e., follow the random walk for a long time? If the Markov chain is ergodic then

$$\lim_{m\to\infty} P(X(t+m) = j|X(t) = i) = \pi_j \tag{15}$$

for some *stationary distribution* π. Note that π_j does not depend on i at all. In other words, the random walk will forget where it started from. Ergodic Markov chains ensure that there's enough "mixing" so that the information about the initial state is lost. In our case, roughly speaking, any connected graph gives rise to an ergodic Markov chain.

Back to clustering. The fact that a random walk on the graph forgets where it started from is very useful to us in terms of identifying clusters. Consider, for example, two tightly connected clusters that are only weakly coupled across. The random walk started at a node in one of the clusters quickly forgets which state within the cluster it begun. However, the information about which cluster the starting node was in lingers much longer. It is precisely this lingering information about clusters in random walks that helps us identify them. This is also something we can understand based on eigenvalues and eigenvectors.

So, let's try to identify clusters by seeing what information we have about the random walk after a large number of steps. To make our analysis a bit easier, we will rewrite

$$P^m = D^{-1/2}(D^{-1/2}WD^{-1/2})^m D^{1/2} \tag{16}$$

You can easily verify this for $m = 1, 2$. The symmetric matrix $D^{-1/2}WD^{-1/2}$ can be written in terms of its eigenvalues $\lambda'_1 \geq \lambda'_2 \geq \ldots$ and eigenvectors $\tilde{z}_1, \tilde{z}_2, \ldots$

$$(D^{-1/2}WD^{-1/2})^m = (\lambda'_1)^m \tilde{z}_1 \tilde{z}_1^T + (\lambda'_2)^m \tilde{z}_2 \tilde{z}_2^T + \ldots + (\lambda'_n)^m \tilde{z}_n \tilde{z}_n^T \tag{17}$$

The eigenvalues are the same as those of P and any eigenvector $\tilde{z}$ of $D^{-1/2}WD^{-1/2}$ corresponds to an eigenvector $z_2 = D^{-1/2}\tilde{z}_2$ of P. As $m \to \infty$, clearly

$$P^\infty = D^{-1/2}\tilde{z}_1 \tilde{z}_1^T D^{1/2} \tag{18}$$

since $\lambda'_1 = 1$ as before and $\lambda'_2 < 1$ (ergodicity). The goal is to understand which transitions remain strong even for large m. These should be transitions within clusters. So, since the eigenvalues are ordered, for large m

$$P^m \approx D^{-1/2}\left(\tilde{z}_1 \tilde{z}_1^T + (\lambda'_2)^m \tilde{z}_2 \tilde{z}_2^T\right) D^{1/2} \tag{19}$$

where $\tilde{z}_2$ is the eigenvector with the second largest eigenvalue. Note that its components have the same signs as the components of z_2, the second largest eigenvector of P. Let's look at the "correction term" $(\tilde{z}_2 \tilde{z}_2^T)_{ij} = \tilde{z}_{2i}\tilde{z}_{2j}$. In other words, we get lingering stronger transitions between i and j corresponding to nodes where $\tilde{z}_{2i}$ and $\tilde{z}_{2j}$ have the same sign, and decreased transition across. These are the clusters and indeed obtained by reading the cluster assignments from the signs of the components of the relevant eigenvector.

Lecture topics:

- Markov chains (cont'd)
- Hidden Markov Models

Markov chains (cont'd)

In the context of spectral clustering (last lecture) we discussed a random walk over the nodes induced by a weighted graph. Let $W_{ij} \geq 0$ be symmetric weights associated with the edges in the graph; $W_{ij} = 0$ whenever edge doesn't exist. We also assumed that $W_{ii} = 0$ for all i. The graph defines a random walk where the probability of transitioning from node (state) i to node j is given by

$$P(X(t+1) = j | X(t) = i) = P_{ij} = \frac{W_{ij}}{\sum_{j'} W_{ij'}} \tag{1}$$

Note that self-transitions (going from i to i) are disallowed because $W_{ii} = 0$ for all i. We can understand the random work as a *homogeneous Markov chain*: the probability of transitioning from i to j only depends on i, not the path that took the process to i. In other words, the current state summarizes the past as far as the future transitions are concerned. This is a Markov (conditional independence) property:

$$P(X(t+1) = j | X(t) = i, X(t-1) = i_{t-1}, \ldots, X(1) = i_1) = P(X(t+1) = j | X(t) = i) \tag{2}$$

The term "homogeneous" specifies that the transition probabilities are independent of time (the same probabilities are used whenever the random walk returns to i).

We also defined *ergodicity* as follows: Markov chain is ergodic if there exist a finite m such that

$$P(X(t+m) = j | X(t) = i) > 0 \text{ for all } i \text{ and } j \tag{3}$$

Simple weighted graphs need not define ergodic chains. Consider, for example, a weighted graph between two nodes $1 - 2$ where $W_{12} > 0$. The resulting random walk is necessarily periodic, i.e., $121212\ldots$. A Markov chain is ergodic only when all the states are communicating and the chain is *aperiodic* which is clearly not the case here. Similarly, even a graph $1 - 2 - 3$ with positive weights on the edges would not define an ergodic Markov chain. Every other state would necessarily be 2, thus the chain is periodic. The reason here is

that $W_{ii} = 0$. By adding a positive self-transition, we can remove periodicity (random walk would stay in the same state a variable number of steps). Any connected weighted graph with positive weights and positive self-transitions gives rise to an ergodic Markov chain.

Our definition of the random walk so far is a bit incomplete. We did not specify how the process started, i.e., we didn't specify the initial state distribution. Let $q(i)$ be the probability that the random walk is started from state i. We will use q as a vector of probabilities across k states (reserving n for the number training examples as usual).

There are two ways of describing Markov chains: through *state transition diagrams* or as simple *graphical models*. The descriptions are complementary. A transition diagram is a directed graph over the possible states where the arcs between states specify all allowed transitions (those occuring with non-zero probability). See Figure 1 for examples. We could also add the initial state distribution as transitions from a dedicated initial (null) state (not shown in the figure).

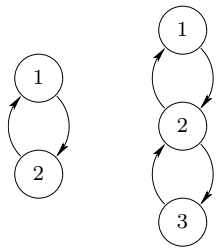


Figure 1: Examples of transition diagrams defining non-ergodic Markov chains.

In graphical models, on the other hand, we focus on explicating variables and their dependencies. At each time point the random walk is in a particular state $X(t)$. This is a random variable. It's value is only affected by the random variable $X(t-1)$ specifying the state of the random walk at the previous time point. Graphically, we can therefore write a sequence of random variables where arcs specify how the values of the variables are influenced by others (dependent on others). More precisely, $X(t-1) \rightarrow X(t)$ means that the value of $X(t)$ depends on $X(t-1)$. Put another way, in simulating the random walk, we would have to know the value of $X(t-1)$ in order to sample a value for $X(t)$. The graphical model is shown in Figure 2.

State prediction

We will cast the problem of calculating the predictive probabilities over states in a form

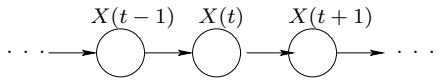


Figure 2: Markov chain as a graphical model.

that will be useful for Hidden Markov Models later on. Since

$$P(X(t+m) = j | X(t) = i) = [P^m]_{ij} \tag{4}$$

we can also write for any n

$$P(X(n) = j) = \sum_{i=1}^{k} q(i) P(X(n) = j | X(1) = i) = \sum_{i=1}^{k} q(i) [P^{n-1}]_{ij} \tag{5}$$

In a vector form $q^T P^{n-1}$ is a row vector whose j^{th} component is $P(X(n) = j)$. Note that the matrix products involve summing over all the intermediate states until $X(n) = j$. More explicitly, let's evaluate the sum over all the states $x_1, \ldots, x_n$ in the matrix form as

$$\sum_{x_1,\ldots,x_n} P(X(1) = x_1) \prod_{t=1}^{n-1} P(X(t+1) = x_{t+1} | X(t) = x_t) = q^T \overbrace{P\, P \cdots P\, P}^{n-1 \text{ times}} \mathbf{1} = 1 \tag{6}$$

This is a sum over k^n possible state configurations (settings of $x_1, \ldots, x_n$) but can be easily performed in terms of matrix products. We can understand this in terms of recursive evaluation of t step probabilities $\alpha_t(i) = P(X(t) = i)$. We will write α_t for the corresponding column vector so that

$$q^T \overbrace{P\, P \cdots P\, P}^{t-1 \text{ times}} = \alpha_t^T \tag{7}$$

Clearly,

$$q^T = \alpha_1^T \tag{8}$$

$$\alpha_{t-1}^T P = \alpha_t^T, \quad t > 1 \tag{9}$$

$$\sum_{i=1}^{k} \alpha_{t-1}(i) P_{ij} = \alpha_t(j) \tag{10}$$

Estimation

Markov models can be estimated easily from observed sequences of states. Given $x_1, \ldots, x_n$ (e.g., 1212221), the log-likelihood of the observed sequence is given by

$$\log P(x_1, \ldots, x_n) = \log \left[P(X(1) = x_1) \prod_{t=1}^{n-1} P(X(t+1) = x_{t+1} | X(t) = x_t) \right] \quad (11)$$

$$= \log q(x_1) + \sum_{t=1}^{n-1} \log P_{x_t, x_{t+1}} \quad (12)$$

$$= \log q(x_1) + \sum_{i,j} \hat{n}(i,j) \log P_{ij} \quad (13)$$

where $\hat{n}(i,j)$ is the number of observed transitions from i to j in the sequence $x_1, \ldots, x_n$. The resulting maximum likelihood setting of P_{ij} is obtained as an empirical fraction

$$\hat{P}_{ij} = \frac{\hat{n}(i,j)}{\sum_{j'} \hat{n}(i,j')} \quad (14)$$

Note that $q(i)$ can only be reliably estimated from multiple observed sequences. For example, based on $x_1, \ldots, x_n$, we would simply set $\hat{q}(i) = \delta(i, x_1)$ which is hardly accurate (sample size one). Regularization is useful here, as before.

Hidden Markov Models

Hidden Markov Models (HMMs) extend Markov models by assuming that the states of the Markov chain are not observed directly, i.e., the Markov chain itself remains hidden. We therefore also model how the states relate to the actual observations. This assumption of a simple Markov model underlying a sequence of observations is very useful in many practical contexts and has made HMMs very popular models of sequence data, from speech recognition to bio-sequences. For example, to a first approximation, we may view words in speech as Markov sequences of phonemes. Phonemes are not observed directly, however, but have to be related to acoustic observations. Similarly, in modeling protein sequences (sequences of amino acid residues), we may, again approximately, describe a protein molecule as a Markov sequence of structural characteristics. The structural features are typically not observable, only the actual residues.

We can understand HMMs by combining mixture models and Markov models. Consider the simple example in Figure 3 over four discrete time points $t = 1, 2, 3, 4$. The figure summarizes multiple sequences of observations $y_1, \ldots, y_4$, where each observation sequence

corresponds to a single value y_t per time point. Let's begin by ignoring the time information and instead collapse the observations across the time points. The observations form two clusters are now well modeled by a two component mixture:

$$P(y) = \sum_{j=1}^{2} P(j)P(y|j) \tag{15}$$

where, e.g., $P(y|j)$ could be a Gaussian $N(y; \mu_j, \sigma_j^2)$. By collapsing the observations we are effectively modeling the data at each time point with the same mixture model. If we generate data form the resulting mixture model we would select a mixture component at random at each time step and generate the observation from the corresponding component (cluster). There's nothing that ties the selection of mixture components in time so that samples from the mixture yield "phantom" clusters at successive time points (we select the wrong component/cluster with equal probability). By omitting the time information, we therefore place half of the probability mass in locations with no data. Figure 4 illustrates the mixture model as a graphical model.

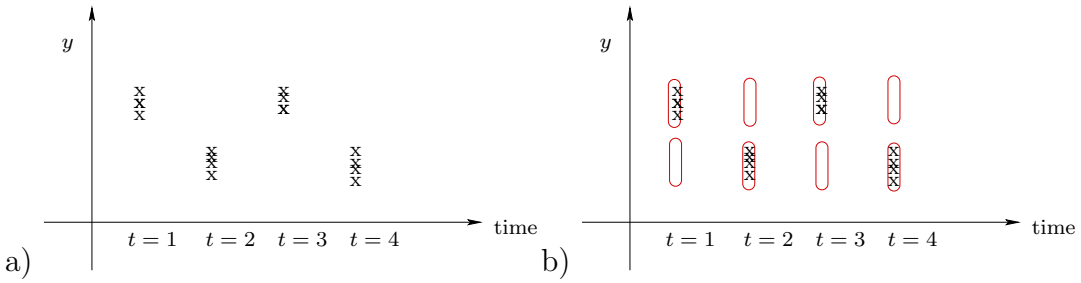


Figure 3: a) Example data over four time points, b) actual data and ranges of samples generated from a mixture model (red ovals) estimated without time information.

The solution is to model the selection of the mixture components as a Markov model, i.e., the component at $t = 2$ is selected on the basis of the component used at $t = 1$. Put another way, each state in the Markov model now uses one of the components in the mixture model to generate the corresponding observation. As a graphical model, the mixture model is a combination of the two as shown in Figure 5.

Probability model

One advantage of representing the HMM as a graphical model is that we can easily write down the joint probability distribution over all the variables. The graph explicates how the

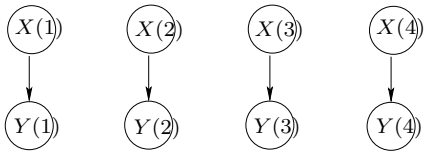


Figure 4: A graphical model view of the mixture model over the four time points. The variables are indexed by time (different samples would be drawn at each time point) but the parameters are shared across the four time points. $X(t)$ refers to the selection of the mixture component while $Y(t)$ refers to the observations.

variables depend on each other (who influences who) and thus highlights which conditional probabilities we need to write down:

$$P(x_1, \ldots, x_n, y_1, \ldots, y_n) = P(x_1)P(y_1|x_1)P(x_2|x_1)P(y_2|x_2)\ldots \quad (16)$$

$$= P(x_1)P(y_1|x_1)\prod_{t=1}^{n-1}[P(x_{t+1}|x_t)P(y_{t+1}|x_{t+1})] \quad (17)$$

$$= q(x_1)P(y_1|x_1)\prod_{t=1}^{n-1}\left[P_{x_t,x_{t+1}}P(y_{t+1}|x_{t+1})\right] \quad (18)$$

where we have used the same notation as before for the Markov chains.

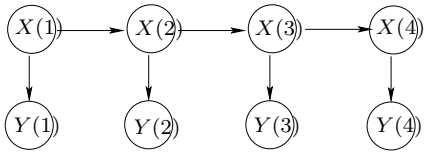


Figure 5: HMM as a graphical model. It is a Markov model where each state is associated with a distribution over observations. Alternatively, we can view it as a mixture model where the mixture components are selected in a time dependent manner.

Three problems to solve

We typically have to be able to solve the following three problems in order to use these models effectively:

1. Evaluate the probability of observed data or

$$P(y_1, \ldots, y_n) = \sum_{x_1,\ldots,x_n} P(x_1, \ldots, x_n, y_1, \ldots, y_n) \tag{19}$$

2. Find the most likely hidden state sequence $x_1^*, \ldots, x_n^*$ given observations $y_1, \ldots, y_n$, i.e.,

$$\{x_1^*, \ldots, x_n^*\} = \arg \max_{x_1,\ldots,x_n} P(x_1, \ldots, x_n, y_1, \ldots, y_n) \tag{20}$$

3. Estimate the parameters of the model from multiple sequences of $y_1^{(l)}, \ldots, y_{n_l}^{(l)}$, $l = 1, \ldots, L$.

Problem 1

As in the context of Markov chains we can efficiently sum over the possible hidden state sequences. Here the summation means evaluating $P(y_1, \ldots, y_n)$. We will perform this in two ways depending on whether the recursion moves forward in time, computing $\alpha_t(j)$, or backward in time, evaluating $\beta_t(i)$. The only change from before is the fact that whatever state we happen to visit at time t, we will also have to generate the observation y_t from that state. This additional requirement of generating the observations can be included via diagonal matrices

$$D_y = \begin{bmatrix} P(y|1) & & 0 \\ & \cdots & \\ 0 & & P(y|k) \end{bmatrix} \tag{21}$$

So, for example,

$$q^T D_{y_1} \mathbf{1} = \sum_{i=1}^{k} q(i) P(y_1|i) = P(y_1) \tag{22}$$

Similarly,

$$q^T D_{y_1} P D_{y_2} \mathbf{1} = \sum_{i=1}^{k} q(i) P(y_1|i) \sum_{j=1}^{k} P_{ij} P(y_2|j) = P(y_1, y_2) \tag{23}$$

We can therefore write the forward and backward algorithms as methods that perform the matrix multiplications in

$$q^T D_{y_1} P D_{y_2} P \cdots P D_{y_n} \mathbf{1} = P(y_1, \ldots, y_n) \tag{24}$$

either in the forward or backward direction. In terms of the forward pass algorithm:

$$q^T D_{y_1} = \alpha_1^T \tag{25}$$

$$\alpha_{t-1}^T P D_{y_t} = \alpha_t^T, \text{ or equivalently} \tag{26}$$

$$\left(\sum_{i=1}^{k} \alpha_{t-1}(i) P_{ij} \right) P(y_t|j) = \alpha_t(j) \tag{27}$$

These values hold exactly $\alpha_t(j) = P(y_1, \ldots, y_t, X(t) = j)$ since we have generated all the observations up to and including y_t and have summed over all the states except for the last one $X(t)$.

The backward pass algorithm is similarly defined as:

$$\beta_n = \mathbf{1} \tag{28}$$

$$\beta_t = P D_{y_{t+1}} \beta_{t+1}, \text{ or equivalently} \tag{29}$$

$$\beta_t(i) = \sum_{j=1}^{k} P_{ij} P(y_{t+1}|j) \beta_{t+1}(j) \tag{30}$$

In this case $\beta_t(i) = P(y_{t+1}, \ldots, y_n | X(t) = i)$ since we have summed over all the possible values of the state variables $X(t+1), \ldots, X(n)$, starting from a fixed $X(t) = i$, and the first observation we have generated in the recursion is y_{t+1}.

By combining the two recursions we can finally evaluate

$$P(y_1, \ldots, y_n) = \alpha_t^T \beta_t = \sum_{i=1}^{k} \alpha_t(i) \beta_t(i) \tag{31}$$

which holds for any $t = 1, \ldots, n$. You can understand this result in two ways: either in terms of performing the remaining matrix multiplication corresponding to the two parts

$$P(y_1, \ldots, y_n) = \overbrace{(q^T D_{y_1} P \cdots P D_{y_t})}^{\alpha_t^T} \overbrace{(P D_{y_{t+1}} \cdots P D_{y_n} \mathbf{1})}^{\beta_t} \tag{32}$$

or as an illustration of the Markov property:

$$P(y_1, \ldots, y_n) = \sum_{i=1}^{k} \overbrace{P(y_1, \ldots, y_t, X(t) = i)}^{\alpha_t(i)} \overbrace{P(y_{t+1}, \ldots, y_n | X(t) = i)}^{\beta_t(i)} \tag{33}$$

Also, since $\beta_n(i) = 1$ for all i, clearly

$$P(y_1, \ldots, y_n) = \sum_{i=1}^{k} \alpha_n(i) = \sum_{i=1}^{k} P(y_1, \ldots, y_t, X(t) = i) \tag{34}$$

Lecture topics:

- Hidden Markov Models (cont'd)

Hidden Markov Models (cont'd)

We will continue here with the three problems outlined previously. Consider having given a set of sequences of observations $y_1, \ldots, y_n$. The observations typically do not contain the hidden state sequence and we are left with the following problems to solve:

1. Evaluate the probability of observed data or
$$P(y_1, \ldots, y_n) = \sum_{x_1,\ldots,x_n} P(x_1, \ldots, x_n, y_1, \ldots, y_n) \quad (1)$$

2. Find the most likely hidden state sequence $x_1^*, \ldots, x_n^*$ given observations $y_1, \ldots, y_n$, i.e.,
$$\{x_1^*, \ldots, x_n^*\} = \arg \max_{x_1,\ldots,x_n} P(x_1, \ldots, x_n, y_1, \ldots, y_n) \quad (2)$$

3. Estimate the parameters of the model from multiple sequences of $y_1^{(l)}, \ldots, y_{n_l}^{(l)}$, $l = 1, \ldots, L$.

We have already solved the first problem. For example, this can be done with the forward algorithm

$$q(j)P(y_1|j) = \alpha_1(j) \quad (3)$$
$$\left(\sum_{i=1}^{k} \alpha_{t-1}(i)P_{ij}\right) P(y_t|j) = \alpha_t(j) \quad (4)$$

where $\alpha_t(j) = P(y_1, \ldots, y_t, X(t) = j)$ so that $P(y_1, \ldots, y_n) = \sum_j \alpha_n(j)$.

Problem 2: most likely hidden state sequence

The most likely hidden state sequence can be found with a small modification to the forward pass algorithm. The goal is to first evaluate "max-probability" of data

$$\max_{x_1,\ldots,x_n} P(y_1, \ldots, y_n, x_1, \ldots, x_n) = P(y_1, \ldots, y_n, x_1^*, \ldots, x_n^*) \quad (5)$$

and subsequently reconstruct the maximizing sequence $x_1^*, \ldots, x_n^*$. The max operation is similar to evaluating

$$\sum_{x_1,\ldots,x_n} P(y_1, \ldots, y_n, x_1, \ldots, x_n) = P(y_1, \ldots, y_n) \tag{6}$$

which we were able to do with just the forward algorithm. In fact, we can obtain the max-probability of data by merely changing the 'sum' in the forward algorithm to a 'max':

$$q(j)P(y_1|j) = d_1(j) \tag{7}$$

$$\left(\max_i d_{t-1}(i)P_{ij}\right) P(y_t|j) = d_t(j) \tag{8}$$

where

$$d_t(j) = \max_{x_1,\ldots,x_{t-1}} P(y_1, \ldots, y_t, x_1, \ldots, x_{t-1}, X(t) = j) \tag{9}$$

In the forward algorithm we finally summed over the last state j in $\alpha_n(j)$ to get the probability of observations. Analogously, here we need to maximize over that last state so as to get the max-probability:

$$\max_{x_1,\ldots,x_n} P(y_1, \ldots, y_n, x_1, \ldots, x_n) = \max_j d_n(j) \tag{10}$$

We now have the maximum value but not yet the maximizing sequence. We can easily reconstruct the sequence by backtracking search, i.e., by sequentially fixing states starting with the last one:

$$x_n^* = \arg\max_j d_n(j) \tag{11}$$

$$x_t = \arg\max_i d_t(i)P_{i,x_{t+1}^*} \tag{12}$$

In other words, the backward iteration simply finds i that attains the maximum in Eq.(8) when j has already been fixed to the maximizing value x_{t+1}^* for the next state. The resulting algorithm that evaluates $d_t(j)$ through the above recursive formula, and follows up with the backtracking search to realize (one of) the most likely hidden state sequences, is known as the *Viterbi algorithm*.

Consider an HMM with two underlying states and transition probabilities as described in Figure 1. Note that the model cannot return to state 1 after it has left it. Each state $j = 1, 2$ is associated with a Gaussian output distribution $P(y|j) = N(y; \mu_j, \sigma^2)$, where

$\mu_1 = 3$, $\mu_2 = 1$, and the variances are assumed to be the same. We are given 8 observations $y_1, \ldots, y_8$ shown in the figure. Now, for any specific value of σ^2, we can find the most likely hidden state sequence $x_1^*, \ldots, x_8^*$ with the Viterbi algorithm.

Let's try to understand how this state sequence behaves as a function of the common variance σ^2. When σ^2 is large, the two output distributions, $P(y|1)$ and $P(y|2)$, assign essentially the same probability to all the observations in the figure. Thus the most likely hidden state sequence is one that is guided solely by the Markov model (no observations). The resulting sequence is all 2's. The probability of this sequence under the Markov model is just 1/2 (there's only one choice, the initial selection). The probability of any other state sequence is at most 1/4. Now, let's consider the other extreme, when the variance σ^2 is very small. In this case, the state sequence is essentially only guided by the observations with the constraint that the you cannot transition out of state 2. The most likely state sequence in this case is five 1's followed by three 2's, i.e., 11111222. The two observations y_4 and y_5 keep the model in state 1 even though y_3 is low. This is because the Markov chain forces us to either capture y_3 or $\{y_4, y_5\}$ but not both. If the model could return to state 1, the most likely state sequence would become 11211222. For intermediate values of σ^2 the most likely state sequence tries to balance the tendency of the Markov chain to choose 2 as soon as possible and the need to assign a reasonable probability to all observations. For example, if $\sigma^2 \approx 1$ then the resulting most likely state sequence is 11222222.

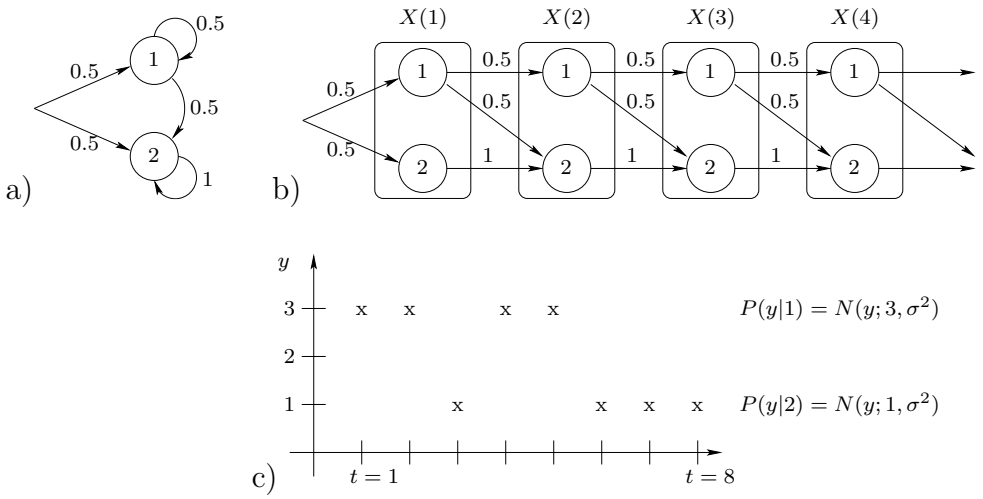


Figure 1: a) A two-state markov chain with transition probabilities, b) the same chain unfolded in time with the corresponding state variables, c) example observations over 8 time points and the output distributions.

Problem 3: estimation

In most cases we have to estimate HMMs only on the basis of output sequences such as $y_1, \ldots, y_n$ without knowing the corresponding states of the Markov chain. This is akin to mixture models discussed earlier. These are incomplete data estimation problems, i.e., we do not have observations for all the variables involved in the model. As before, the estimation can be performed iteratively via the EM algorithm.

A simple way to derive the EM algorithm is to start with complete observations, i.e., we assume we have $x_1, \ldots, x_n$ as well as $y_1, \ldots, y_n$. Note that while typically we would have multiple observation sequences, we will focus here on a single sequence to keep the equations simple. Now, we can encode the complete observations by defining

$$\delta(i|t) = \begin{cases} 1, & \text{if } x_t = i \\ 0, & \text{otherwise} \end{cases} \tag{13}$$

$$\delta(i,j|t) = \begin{cases} 1, & \text{if } x_t = i \text{ and } x_{t+1} = j \\ 0, & \text{otherwise} \end{cases} \tag{14}$$

The complete log-likelihood then becomes

$$\begin{aligned} l(\{x_t\},\{y_t\}) &= \sum_{i=1}^{k} \delta(i|1) \log q(i) + \sum_{i=1}^{k} \left(\sum_{t=1}^{n} \delta(i|t) \log P(y_t|i) \right) \\ &\quad + \sum_{i=1}^{k} \sum_{j=1}^{k} \left(\sum_{t=1}^{n} \delta(i,j|t) \right) \log P_{ij} \end{aligned} \tag{15}$$

where the first term simply picks out $\log q(x_1)$; in the second term, for each i, we consider all the observations that had to be generated from state i; in the last expression, the term in the brackets counts how many times each $i \to j$ transition occurred in the sequence $x_1, \ldots, x_n$. Given the counts $\delta(\cdot)$, we can now solve for the maximizing parameters as in the case of simple Markov chains (the first and the last expression), and as in mixture models (second expression).

The EM algorithm now follows directly from replacing the hard counts, $\delta(i|t)$ and $\delta(i,j|t)$, with the corresponding posterior probabilities $p(i|t)$ and $p(i,j|t)$, evaluated on the basis of the current HMM parameters. The posteriors we need are

$$p(i|t) = P(X(t) = i | y_1, \ldots, y_n) \tag{16}$$

$$p(i,j|t) = P(X(t) = i, X(t+1) = j | y_1, \ldots, y_n) \tag{17}$$

It remains to show how these posterior probabilities can be computed. We can start by writing

$$\begin{aligned} P(y_1, \ldots, y_n, X(t) = i) &= P(y_1, \ldots, y_t, X(t) = i) P(y_{t+1}, \ldots, y_n | X(t) = i) \quad (18) \\ &= \alpha_t(i)\beta_t(i) \quad (19) \end{aligned}$$

which follows directly from the Markov property (future observations do not depend on the past ones provided that we know which state we are in currently). The posterior is now obtained by normalization

$$P(X(t) = i | y_1, \ldots, y_n) = \frac{\alpha_t(i)\beta_t(i)}{\sum_{i'=1}^{k} \alpha_t(i')\beta_t(i')} \quad (20)$$

Similarly,

$$\begin{aligned} &P(y_1, \ldots, y_n, X(t) = i, X(t+1) = j) \\ &= P(y_1, \ldots, y_t, X(t) = i)\, P_{ij}\, P(y_{t+1}|j)\, P(y_{t+2}, \ldots, y_n | X(t+1) = j) \quad (21) \\ &= \alpha_t(i) P_{ij} P(y_{t+1}|j) \beta_{t+1}(j) \quad (22) \end{aligned}$$

The posterior again results from normalizing across i and j:

$$P(X(t) = i, X(t) = j | y_1, \ldots, y_n) = \frac{\alpha_t(i) P_{ij} P(y_{t+1}|j) \beta_{t+1}(j)}{\sum_{i'=1}^{k} \sum_{j'=1}^{k} \alpha_t(i') P_{i'j'} P(y_{t+1}|j') \beta_{t+1}(j')} \quad (23)$$

It is important to understand that $P(X(t) = i, X(t) = j | y_1, \ldots, y_n)$ are not the transition probabilities we have as parameters in the HMM model. These are posterior probabilities that a likely hidden state sequence that had to generate the observation sequence went through i at time t and transitioned into j; they are evaluated on the basis of the model and the observed sequence.

Multiple (partial) alignment

As another example of the use of the Viterbi algorithm as well as the EM algorithm for estimating HMM models, let's consider the problem of multiple alignment of sequences. Here we are interested in finding a pattern, a fairly conserved sequence of observations, embedded in unknown locations in multiple observed sequences. We assume we know very little about the pattern other than that it appeared once in all the sequences (a constraint we could easily relax further). For simplicity, we will assume here that we know the length of the pattern (four time points/positions). The sequences could be speech signals where a particular word was uttered in each but we don't know when the word appeared in

the signal, nor what exactly the word was. The sequences could also be protein or DNA sequences where we are looking for a sequence fragment that appears in all the available sequences (e.g., a binding site).

Perhaps the simplest possible HMM model we could specify is given in Figure 2. The states $m_1, \ldots, m_4$ are "match states" that we envision will be used to generate the pattern we are looking for. States I_1 and I_2 are "insert states" that generate the remaining parts of each observation sequence, before and after the pattern. Each state is associated with an output distribution: $P(y|I_i)$, $i = 1, 2$, $P(y|m_i)$, $i = 1, \ldots, 4$. The parameters p and the output distributions need to be learned from the available data.

Note that this is a model that generates a finite length sequence. We will first enter the insert state, spend there on average $1/(1-p)$ time steps, then generate the pattern, i.e., one observation in succession from each of the match states, and finally spend another $1/(1-p)$ time steps on average to generate flanking observations. We have specifically set the p parameter associated with the first insert state to agree with that of the second. This tying of parameters (balancing the cost of repeating insert states) ensures that, given any specific observation sequence, $y_1, \ldots, y_n$, there is no bias towards finding the pattern in any particular location of the sequence.

This model is useless for finding a pattern in a single observation sequence. However, it becomes more useful when we have multiple observation sequences that can all be assumed to contain the pattern. The stereotypical way that the pattern is generated, one observation from each successive match states, and the freedom to associate any observation with each of the match states, encourages the match states to take over the recurring pattern in the sequences (rather than being generated from the insert states). The output distributions for the insert states cannot become very specific since they will have to be used to generate most of the observations in the sequences.

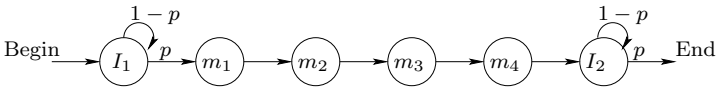


Figure 2: A simple HMM model for the multiple alignment task (only the Markov chain part shown).

Now, given multiple sequences, we can simply train the parameters in our HMM model via the EM algorithm to maximize the log-likelihood that the model assigns to those sequences. At this point we are not concerned about where the patterns actually occur, just interested in finding appropriate parameter values (output distributions). Similarly to mixture models

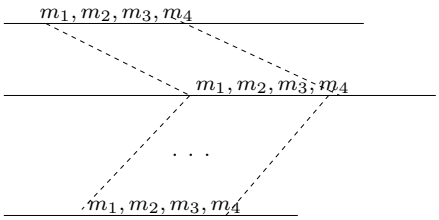
for clustering, the location of the pattern (and what the pattern is), is resolved in a soft manner through the posterior assignments of observations to match or insert states.

Once the parameters are found, we can use the Viterbi algorithm to "label" each observation in a sequence with the corresponding most likely hidden state. So, for example, for a particular observation sequence, $y_1, \ldots, y_n$, we might get

$$\begin{array}{cccccccccccc} I_1 & I_1 & \ldots & I_1 & m_1 & m_2 & m_3 & m_4 & I_2 & I_2 & \ldots & I_2 \\ y_1 & y_2 & \ldots & y_{t-1} & y_t & y_{t+1} & y_{t+2} & y_{t+3} & y_{t+4} & y_{t+5} & \ldots & y_n \end{array} \tag{24}$$

as the most likely hidden state sequence. The states in the most likely state sequence are in one to one correspondence with the observations. So, in this case, the pattern clearly occurs exactly at time/position t, where the sequence of match states begins.

The sequence fragments in all the observation sequences that were identified with the pattern can be subsequently aligned as in the figure below.



Lecture topics:

- Bayesian networks

Bayesian networks

Bayesian networks are useful for representing and using probabilistic information. There are two parts to any Bayesian network model: 1) directed graph over the variables and 2) the associated probability distribution. The graph represents qualitative information about the random variables (conditional independence properties), while the associated probability distribution, consistent with such properties, provides a quantitative description of how the variables relate to each other. If we already have the distribution, why consider the graph? The graph structure serves two important functions. First, it explicates the properties about the underlying distribution that would be otherwise hard to extract from a given distribution. It is therefore useful to maintain the consistency between the graph and the distribution. The graph structure can also be learned from available data, i.e., we can explicitly learn qualitative properties from data. Second, since the graph pertains to independence properties about the random variables, it is very useful for understanding how we can use the probability model efficiently to evaluate various marginal and conditional properties. This is exactly why we were able to carry out efficient computations in HMMs. The forward-backward algorithms relied on simple Markov properties which are independence properties, and these are generalized in Bayesian networks. We can make use of independence properties whenever they are explicit in the model (graph).

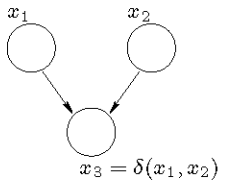


Figure 1: A simple Bayesian network over two independent coin flips x_1 and x_2 and a variable x_3 checking whether the resulting values are the same. All the variables are binary.

Let's start with a simple example Bayesian network over three binary variables illustrated in Figure 1. We imagine that two people are flipping coins independently from each other. The resulting values of their unbiased coin flips are stored in binary (0/1) variables x_1 and x_2. Another person checks whether the coin flips resulted in the same value and the

outcome of the comparison is a binary (0/1) variable $x_3 = \delta(x_1, x_2)$. Based on the problem description we can easily write down a joint probability distribution over the three variables

$$P(x_1, x_2, x_3) = P(x_1)P(x_2)P(x_3|x_1, x_2) \quad (1)$$

where $P(x_1) = 0.5$, $x_1 \in \{0, 1\}$, $P(x_2) = 0.5$, $x_2 \in \{0, 1\}$, and $P(x_3 = 1|x_1, x_2) = 1$ if $x_1 = x_2$ and zero otherwise.

We could have read the structure of the joint distribution from the graph as well. We need a bit of terminology to do so. In the graph, x_1 is a *parent* of x_3 since there's a directed edge from x_1 to x_3 (the value of x_3 depends on x_1). Analogously, we can say that x_3 is a *child* of x_1. Now, x_2 is also a parent of x_3 so that the value of x_3 depends on both x_1 and x_2. We will discuss later what the graph means more formally. For now, we just note that Bayesian networks always define acyclic graphs (no directed cycles) and represent how values of the variables depend on their parents. As a result, any joint distribution consistent with the graph, i.e., any distribution we could imagine associating with the graph, has to be able to be written as a product of conditional probabilities of each variable given its parents. If a variable has no parents (as is the case with x_1) then we just write $P(x_1)$. Eq.(1) is exactly a product of conditional probabilities of variables given their parents.

Marginal independence and induced dependence

Let's analyze the properties of the simple model a bit. For example, what is the marginal probability over x_1 and x_2? This is obtained from the joint simply by summing over the values of x_3

$$P(x_1, x_2) = \sum_{x_3} P(x_1)P(x_2)P(x_3|x_1, x_2) = P(x_1)P(x_2) \sum_{x_3} P(x_3|x_1, x_2) = P(x_1)P(x_2) \quad (2)$$

Thus x_1 and x_2 are *marginally independent* of each other. In other words, if we don't know the value of x_3 then there's nothing that ties the coin flips together (they were, after all, flipped independently in the description). This is also a property we could have extracted directly from the graph. We will provide shortly a formal way of deriving this type of independence properties from the Bayesian network.

Another typical property of probabilistic models is *induced dependence*. Suppose now that the coins x_1 and x_2 were flipped independently but we don't know their outcomes. All we know is the value of x_3, i.e., whether the outcomes where identical or not (say they were identical). What do we know about x_1 and x_2 in this case? We know that either $x_1 = x_1 = 0$ or $x_1 = x_2 = 1$. So their values are clearly *dependent*. The dependence was *induced by additional knowledge*, in this case the value of x_3. This is again a property we

could have read off directly from the graph (explained below). Note that the dependence pertains to our beliefs about the values of x_1 and x_2. The coins were physically flipped independently of each other and our knowledge of the value of x_3 doesn't change this. However, the value of x_3 narrows down the set of possible outcomes of the two coin flips for this *particular sample of* x_1 *and* x_2.

Both marginal independence and induced dependence are typical properties of realistic models. Consider, for example, a factorial Hidden Markov Model in Figure 2c). In this model you have two marginally independent Markov models that conspire to generate the observed output. In other words, the two Markov models are tied only through observations (induced dependence). To sample values for the variables in the model, we would be sampling from the two Markov models independently and just using the two states at each time point to sample a value for the output variables. The joint distribution over the variables for the model in Figure 2c) is again obtained by writing a product of conditional probabilities of each variable given its parents:

$$P(x_1')P(x_1)P(y_1|x_1', x_1)P(x_2'|x_1')P(x_2|x_1)P(y_2|x_2, x_2')P(x_3'|x_2')P(x_3|x_2)P(y_3|x_3, x_3) \quad (3)$$

where, e.g., $P(y_1|x_1', x_1)$ could be defined as $N(y; \mu(x_1') + \mu(x_1), \sigma^2)$. Such a model could, for example, capture how two independent subprocesses in speech production generate the observed acoustic signal, model two speakers observed through a common microphone, or with a different output model, capture how haplotypes generate observed genotypes. Given the model and say an observed speech signal, we would be interested in inferring likely sequences of states for the subprocesses.

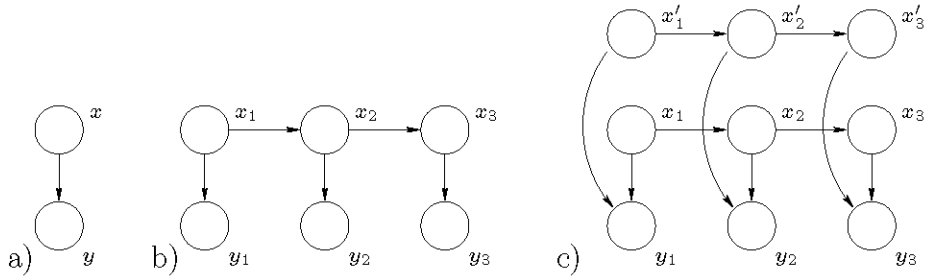


Figure 2: Different models represented as Bayesian networks: a) mixture model, b) HMM, c) factorial HMM.

Explaining away

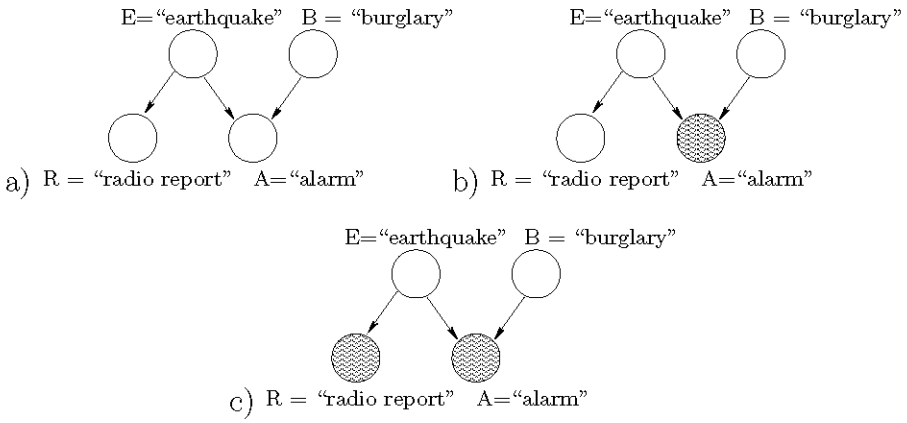


Figure 3: Network structure exhibiting explaining away. a) basic model, b) alarm went off, c) we have also heard a radio report about an earthquake

Another typical phenomenon that probabilistic models can capture is *explaining away*. Consider the following typical example (Pearl 1988) in Figure 3. We have four variables A, B, E, and R capturing possible causes for why a burglary alarm went off. All the variables are binary (0/1) and, for example, $A = 1$ means that the alarm went off (Figure 3b). Shaded nodes indicate that we know something about the values of these variables. In our example here all the observed values are one (property is true). We assume that earthquakes ($E = 1$) and burglaries ($B = 1$) are equally unlikely events $P(E = 1) = P(B = 1) \approx 0$. Alarm is likely to go off only if either $E = 1$ or $B = 1$ or both. Both events are equally likely to trigger the alarm so that $P(A = 1|E, B) \approx A$ or B. An earthquate ($E = 1$) is likely to be followed by a radio report ($R = 1$), $P(R = 1|E = 1) \approx 1$, and we assume that the report never occurs unless an earthquake actually took place: $P(R = 1|E = 0) = 0$.

What do we believe about the values of the variables if we only observe that the alarm went off ($A = 1$)? At least one of the potential causes $E = 1$ or $B = 1$ should have occured. However, since both are unlikely to occur by themselves, we are basically left with either $E = 1$ or $B = 1$ but (most likely) not both. We therefore have two alternative or competing explanations for the observation and both explanations are equally likely. If we know hear, in addition, that there was a radio report about an earthquake, we believe that $E = 1$. This makes $B = 1$ unnecessary for explaining the alarm. In other words, the additional observation about the radio report *explained away* the evidence for $B = 1$. Thus, $P(E = 1|A = 1, R = 1) \approx 1$ whereas $P(B = 1|A = 1, R = 1) \approx 0$.

Note that we have implicitly captured in our calculation here that R and B are *dependent*

given $A = 1$. If they were not, we would not be able to learn anything about the value of B as a result of also observing $R = 1$. Here the effect is drastic and the variables are strongly dependent. We could have, again, derived this property from the graph.

Bayesian networks and conditional independence

We have claimed in several occasions that we could have derived useful properties about the probability model directly from the graph. How is this done exactly? Since the graph encodes independence properties about the variables, we have to define a criterion for extracting independence properties between the variables directly from the graph. For Bayesian networks (acyclic graphs) this is given by so called *D-separation criterion.*

As an example, consider a slightly extended version of the previous model in Figure 4a, where we have added a binary variable L (whether we "leave work" as a result of hearing/learning about the alarm). We will define a procedure for answering questions such as: are R and B independent given A?

The general procedure involves three graph transformation steps that we will illustrate in relation to the graph in Figure 4a.

1. Construct *ancestral graph* of the variables of interest. The variables we care about here are R, B, and A. The ancestral graph includes these variables as well as all the variables (ancestors) you can get to by starting from one of these variables and following the arrows in the reverse direction (their parents, their parents' parents, and so on). The ancestral graph in our case is given in Figure 4b.

   The motivation for this step is that unobserved effects of random variables cannot lead to dependence and can be therefore removed.

2. *Moralize* the resulting ancestral graph. This operation simply adds an *undirected edge* between any two variables in the ancestral graph that have a common child ("marry the parents"). In case of multiple parents, they are connected pairwise, i.e., by adding an edge between any two parents. See Figure 4c.

   Moralization is needed to take into account induced dependences discussed earlier.

3. Change all the direct edges into undirected edges. This gives the resulting *undirected graph* in Figure 4d.

We can now read off the answer to the original question from the resulting undirected graph. R and B are independent given A (they are *D-separated given* A) if they are separated by A in the undirected graph. In other words, if they become disconnected in the undirected

graph by removing the conditioning variable A and its associated edges. They clearly remain connected in our example and thus, from the point of view of the Bayesian network model, would have to be assumed to be dependent.

Let's go back to the previous examples to make sure we can read off the properties we claimed from the graphs. For example, if we are interested in asking whether x_1 and x_2 are marginally independent (i.e., given nothing) in the model in Figure 1, we would create the graph transformations shown in Figure 5. The nodes are clearly separated. Similarly, to establish that x_1 and x_2 become dependent with the observation of x_3, we would ask whether x_1 and x_2 are independent given x_3 and get the transformations in Figure 6. The nodes are not separated by x_3 and therefore not independent.

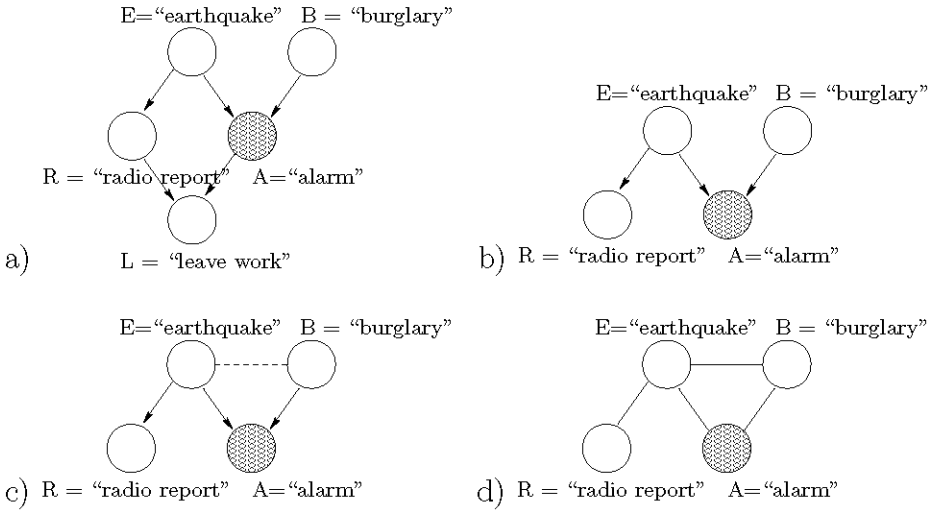


Figure 4: a) Burglary model, extended, b) ancestral graph of R, B, and A, c) moralized ancestral graph, d) resulting undirected graph.

Graph and the probability distribution

The graph and the independence properties we can derive from it are useful to us only if the probability distribution we associate with the graph is consistent with the graph. By consistency we meant that all the independence properties we can derive from the graph should hold for the associated distribution. In other words, if the graph is an explicit representation of such properties, then clearly whatever we can infer from it, should be true. There are actually a large number of possible independence properties that we can derive from any typical graph, even in the context of HMMs. How is it that we can ever hope to find and deal with distributions that are consistent with all such properties? While

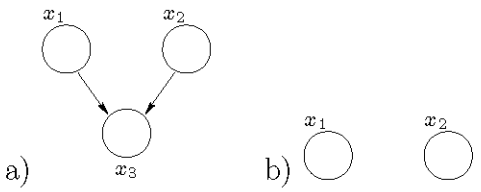


Figure 5: a) Bayesian network model, b) ancestral graph of x_1 and x_2, already moralized and undirected.

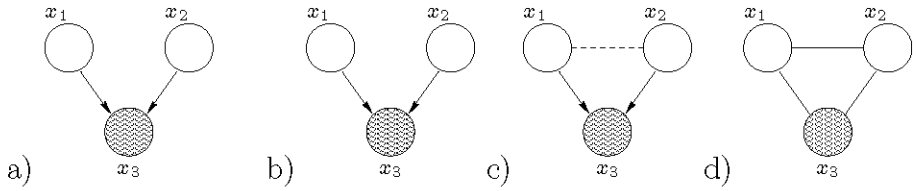


Figure 6: a) Bayesian network model, b) ancestral graph of x_1 and x_2 given x_3, c) moralized ancestral graph, d) resulting undirected graph.

the task is hard, the answer is simple. In fact, given an acyclic graph G over d variables, the most general form of the joint distribution consistent with *all* the properties we can derive from the graph is given by

$$P(x_1, \ldots, x_d) = \prod_{i=1}^{d} P(x_i | x_{pa_i}) \tag{4}$$

where x_{pa_i} refers to the set of variables that are the parents of variable x_i (e.g., $x_{pa_3} = \{x_1, x_2\}$ for x_3 in the above models). So, we can just read off the answer from the graph: look at each variable and include a term in the joint distribution of that variable given its parents (those that directly influence it).

Note that some distributions may satisfy more independence properties that are represented in the graph. For example, a distribution where all the variables are independent of each other is consistent with every acyclic graph. It clearly satisfies all the possible independence properties (edges in the graph only indicate possible dependences; they may actually be

weak or non-existent). We typically would use a graph representation that tries to capture most if not all of the independence properties that hold for the associated distribution. Not all independence properties can be captured (are representable) by our D-separation criterion.

Lecture topics:

- Learning Bayesian networks from data
  - maximum likelihood, BIC
  - Bayesian, marginal likelihood

Learning Bayesian networks

There are two problems we have to solve in order to estimate Bayesian networks from available data. We have to estimate the parameters given a specific structure, and we have to search over possible structures (model selection).

Suppose now that we have d discrete variables, $x_1, \ldots, x_d$, where $x_i \in \{1, \ldots, r_i\}$, and n *complete* observations $D = \{(x_1^t, \ldots, x_d^t), t = 1, \ldots, n\}$. In other words, each observation contains a value assignment to all the variables in the model. This is a simplification and models in practice (e.g., HMMs) have to be estimated from incomplete data. We will also assume that the conditional probabilities in the models are fully parameterized. This means, e.g., that in $P(x_1|x_2)$ we can select the probability distribution over x_1 separately and without constraints for each possible value of the parent x_2. Models used in practice often do have parametric constraints.

Maximum likelihood parameter estimation

Given an acyclic graph G over d variables, we know from previous lecture that we can write down the associated joint distribution as

$$P(x_1, \ldots, x_d) = \prod_{i=1}^{d} P(x_i|x_{pa_i}) \tag{1}$$

The parameters we have to learn are therefore the conditional distributions $P(x_i|x_{pa_i})$ in the product. For later utility we will use $P(x_i|x_{pa_i}) = \theta_{x_i|x_{pa_i}}$ to specify the parameters.

Given the complete data D, the log-likelihood function is

$$\begin{aligned} l(D;\theta,G) &= \log P(D|\theta) && (2)\\ &= \sum_{t=1}^{n} \log P(x_1^t,\ldots,x_d^t|\theta) && (3)\\ &= \sum_{t=1}^{n}\sum_{i=1}^{n} \log \theta_{x_i^t|x_{pa_i}^t} && (4)\\ &= \sum_{i=1}^{n}\sum_{x_i,x_{pa_i}} n(x_i,x_{pa_i}) \log \theta_{x_i|x_{pa_i}} && (5) \end{aligned}$$

where we have again collapsed the available data into counts $n(x_i, x_{pa_i})$, the number of observed instances with a particular setting of the variable and its parents. These are the *sufficient statistics* we need from the data in order to estimate the parameters. This will be true in the Bayesian setting as well (discussed below). Note that the statistics we need depend on the model structure or graph G. The parameters $\hat{\theta}_{x_i|x_{pa_i}}$ that maximize the log-likelihood have simple closed form expressions in terms of empirical fractions:

$$\hat{\theta}_{x_i|x_{pa_i}} = \frac{n(x_i,x_{pa_i})}{\sum_{x_i'=1}^{r_i} n(x_i',x_{pa_i})} \tag{6}$$

This simplicity is due to our assumption that $\theta_{x_i|x_{pa_i}}$ can be chosen freely for each setting of the parents x_{pa_i}. The parameter estimates are likely not going to be particularly good when the number of parents increases. For example, just to provide one observation per a configuration of parent variables would require $\prod_{j\in pa_i} r_j$ instances. Introducing some regularization is clearly important, at least in the fully parameterized case. We will provide a Bayesian treatment of the parameter estimation problem shortly.

BIC and structure estimation

Given the ML parameter estimates $\hat{\theta}_{x_i|x_{pa_i}}$ we can evaluate the resulting maximum value of the log-likelihood $l(D;\hat{\theta},G)$ as well as the corresponding BIC score:

$$BIC(G) = l(D;\hat{\theta},G) - \frac{dim(G)}{2}\log(n) \tag{7}$$

where $dim(G)$ specifies the number of (independent) parameters in the model. In our case this is given by

$$dim(G) = \sum_{i=1}^{d}(r_i-1)\prod_{j\in pa_i} r_j \tag{8}$$

where each term in the sum corresponds to the size of the probability table $P(x_i|x_{pa_i})$ minus the associated normalization constraints $\sum_{x_i'} P(x_i'|x_{pa_i}) = 1$.

BIC and likelihood equivalence

Suppose we have two different graphs G and G' that nevertheless make exactly the same independence assumptions about the variables involved. For example, neither graph in

makes any independence assumptions and are therefore equivalent in this sense. The resulting BIC scores for such graphs are also identical. The principle that equivalent graphs should receive the same score is known as *likelihood equivalence*. How can we determine if two graphs are equivalent? In principle this can be done by deriving all the possible independence statements from the graphs and comparing the resulting lists but there are easier ways. Two graphs are equivalent if they differ only in the direction of arcs and possess the same *v-structures*, i.e., they have the same set of converging arcs (two or more arcs pointing to a single node). This criterion captures most equivalences. Figure 1 provides a list of all equivalence classes of graphs over three variables. Only one representative of each class is shown and the number next to the graph indicates how many graphs there are that are equivalent to the representative.

Equivalence of graphs and the associated scores highlight why we should not interpret the arcs in Bayesian networks as indicating the direction of causal influence. While models are often drawn based on one's causal understanding, when learning them from the available data we can only distinguish between models that make different probabilistic assumptions about the variables involved (different independence properties), not based on which way the arcs are pointing. It is nevertheless possible to estimate causal Bayesian networks, models where we can interpret the arcs as causal influences. The difficulty is that we need *interventional* data to do so, i.e., data that correspond to explicitly setting some of the variables to specific values (controlled experiments) rather than simply observing the values they take.

Bayesian estimation

The idea in Bayesian estimation is to avoid reducing our knowledge about the parameters into point estimates (e.g., ML estimates) but instead retain all the information in a form of a distribution over the possible parameter values. This is advantageous when the available data are limited and the number of parameters is large (e.g., only a few data points per

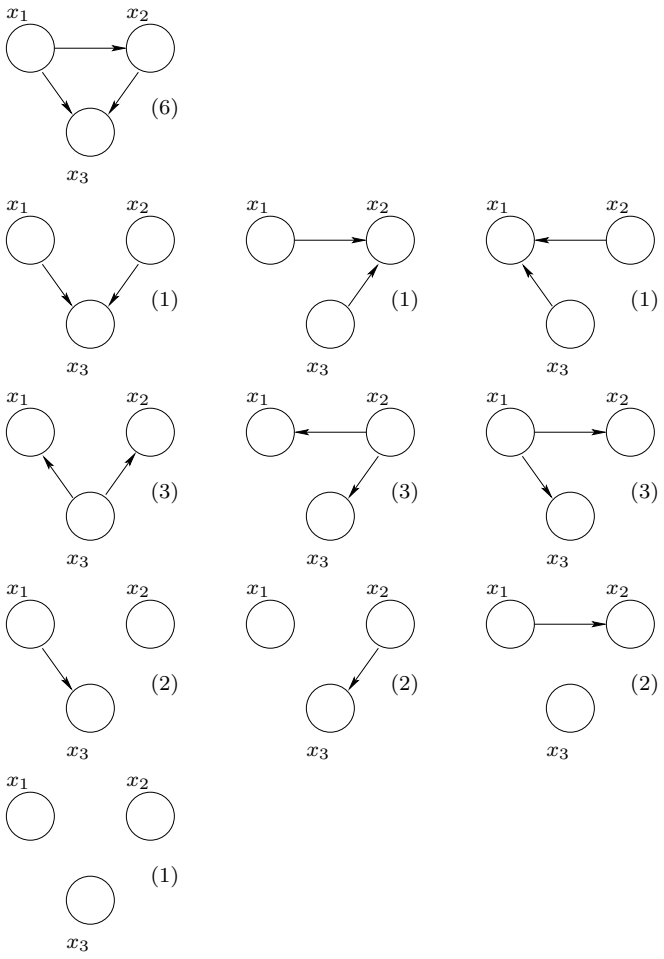
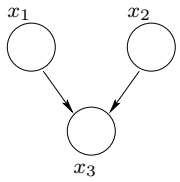


Figure 1: Equivalence classes of graphs over three variables.

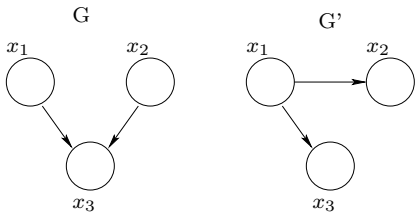
parameter to estimate). The Bayesian framework requires us to also articulate our knowledge about the parameters prior to seeing any data in a form of a distribution, the prior distribution. Consider the following simple graph with three variables



The parameters we have to estimate are $\{\theta_{x_1}\}$, $\{\theta_{x_2}\}$, and $\{\theta_{x_3|x_1,x_2}\}$. We will assume that the parameters are a priori independent for each variable and across different configurations of parents (parameter independence assumption):

$$P(\theta) \;=\; P(\{\theta_{x_1}\}_{x_1=1,\ldots r_1})\, P(\{\theta_{x_2}\}_{x_2=1,\ldots,r_2}) \prod_{x_1,x_2} P(\{\theta_{x_3|x_1,x_2}\}_{x_3=1,\ldots,r_3}) \tag{9}$$

We will also assume that we will use the same prior distribution over the same parameters should they appear in different graphs (parameter modularity). For example, since x_1 has no parents in G and G' given by



we need the same parameter $\{\theta_{x_1}\}$ in both models. The parameter modularity assumption corresponds to using the same prior distribution $P(\{\theta_{x_1}\}_{x_1=1,\ldots,r_1})$ for both models (other parameters would have different prior distributions since, e.g., $\theta_{x_3|x_1,x_2}$ does not appear in graph G'). Finally, we would like the marginal likelihood score to satisfy likelihood equivalence similarly to BIC. In other words, if G and G' are equivalent, then we would like $P(D|G) = P(D|G')$ where, e.g.,

$$P(D|G) = \int P(D|\theta, G)P(\theta)d\theta \tag{10}$$

If we agree to these three assumptions (parameter independence, modularity, and likelihood equivalence), then we can only choose one type of prior distribution over the parameters,

the Dirichlet distribution. To specify and use this prior distribution, it will be helpful to change the notation slightly. We will denote the parameters by θ_{ijk} where i specifies the variable, j the parent configuration (see below), and k the value of the variable x_i. Clearly, $\sum_{k=1}^{r_i} \theta_{ijk} = 1$ for all i and j. The parent configurations are simply indexed from $j = 1, \ldots, q_i$ as in

j	x_1	x_2
1	1	1
2	2	1
$\ldots$	$\ldots$	$\ldots$
q	r_1	r_2

(11)

where $q = r_1 r_2$. When x_i has no parents we say there is only one "parent configuration" so that $P(x_i = k) = \theta_{i1k}$. Note that writing parameters as θ_{ijk} is graph specific; the parents of each variable, and therefore also parent configurations, vary from one graph to another. We will define $\theta_{ij} = \{\theta_{ijk}\}_{k=1,\ldots,r_i}$ so we can talk about all the parameters for x_i given a fixed parent configuration.

Now, the prior distribution of each θ_{ij} has to be a Dirichlet:

$$P(\theta_{ij}) = \frac{\Gamma(\sum_k \alpha_{ijk})}{\prod_k \Gamma(\alpha_{ijk})} \prod_{k=1}^{r_i} \theta_{ijk}^{\alpha_{ijk}-1} = \text{Dirichlet}(\theta_{ij}; \alpha_{ij1}, \ldots, \alpha_{ijr_i}) \tag{12}$$

where, for integers, $\Gamma(z+1) = z!$. The mean of this distribution is

$$\int P(\theta_{ij})\, \theta_{ijk}\, d\theta_{ij} = \frac{\alpha_{ijk}}{\sum_{k'} \alpha_{ijk'}} \tag{13}$$

and it is more concentrated around the mean the larger the value of $\sum_{k'} \alpha_{ijk'}$. We can further write the hyper-parameters $\alpha_{ijk} > 0$ in the form $\alpha_{ijk} = n' p'_{ijk}$ where n' is the equivalent sample size specifying how many observations we need to balance the effect of the data on the estimates in comparison to the prior. There are two subtleties here. First, the number of available observations for estimating θ_{ij} varies with j, i.e., depends on how many times the parent configurations appear in the data. To keep n' as an equivalent sample size across all the parameters, we will have to account for this variation. The parameters p'_{ijk} are therefore not normalized to one across the values of variable x_i but across its values and the parent configurations: $\sum_{j=1}^{q_i} \sum_{k=1}^{r_i} p'_{ijk} = 1$ so we can interpret p'_{ijk} as a distribution over (x_i, x_{pa_j}). In other words, they include the expectation of how many times we would see a particular parent configuration in n' observations.

The second subtlety further constrains the values p'_{ijk}, in addition to the normalization. In order for the likelihood equivalence to hold, p'_{ijk} should be possible to interpret as marginals $P'(x_i, x_{pa_i})$ of some *common distribution* over all the variables $P'(x_1, \ldots, x_d)$ (common to all the graphs we consider). For example, simply normalizing p'_{ijk} across the parent configurations and the values of the variables does not ensure that they can be viewed as marginals from some common joint distribution P'. This subtlety does not often arise in practice. It is typical and easy to set them based on a uniform distribution so that

$$p'_{ijk} = \frac{1}{r_i \prod_{l \in pa_i} r_l} = \frac{1}{r_i q_i} \quad \text{or} \quad \alpha_{ijk} = \frac{n'}{r_i q_i} \tag{14}$$

This leaves us with only one hyper-parameter to set: n', the equivalent sample size.

We can now combine the data and the prior to obtain posterior estimates for the parameters. The prior factors across the variables and across parent configurations. Moreover, we assume that each observation is complete, containing a value assignment for all the variables in the model. As a result, we can evaluate the posterior probability over each θ_{ij} separately from others. Specifically, for each $\theta_{ij} = \{\theta_{ijk}\}_{k=1,\ldots,r_i}$, where i and j are fixed, we get

$$P(\theta_{ij}|D, G) \propto \left[\prod_{t:\, x^t_{pa_i} \to j} P(x^t_i | x^t_{pa_i}, \theta_{ij}) \right] P(\theta_{ij}) \tag{15}$$

$$= \left[\prod_{k=1}^{r_i} \theta_{ijk}^{n_{ijk}} \right] P(\theta_{ij}) \tag{16}$$

$$\propto \left[\prod_{k=1}^{r_i} \theta_{ijk}^{n_{ijk}} \right] \left[\prod_{k=1}^{r_i} \theta_{ijk}^{\alpha_{ijk}-1} \right] \tag{17}$$

$$= \prod_{k=1}^{r_i} \theta_{ijk}^{n_{ijk}+\alpha_{ijk}-1} \tag{18}$$

where the product in the first line picks out only observations where the parent configuration maps to j (otherwise the case would fall under the domain of another parameter vector). n_{ijk} specifies the number of observations where x_i had value k and its parents x_{pa_i} were in configuration j. Clearly, $\sum_{j=1}^{q_i} \sum_{k=1}^{r_i} n_{ijk} = n$. The posterior has the same form as the prior[1] and is therefore also Dirichlet, just with updated hyper-parameters:

$$P(\theta_{ij}|D, G) = \text{Dirichlet}(\theta_{ij}; \alpha_{ij1} + n_{ij1}, \ldots, \alpha_{ijr_i} + n_{ijr_i}) \tag{19}$$

[1] Dirichlet is a *conjugate prior* for the multi-nomial distribution.

The normalization constant for the posterior in Eq.(15) is given by

$$\int \left[\prod_{t:\, x^t_{pa_i} \to j} P(x^t_i | x^t_{pa_i}, \theta_{ij}) \right] P(\theta_{ij}) d\theta_{ij} = \frac{\Gamma(\sum_k \alpha_{ijk})}{\Gamma(\sum_k \alpha_{ijk} + \sum_k n_{ijk})} \prod_{k=1}^{r_i} \frac{\Gamma(\alpha_{ijk} + n_{ijk})}{\Gamma(\alpha_{ijk})} \tag{20}$$

This is also the marginal likelihood of data pertaining to x_i when x_{pa_i} are in configuration j. Since the observations are complete, and the prior is independent for each set of parameters, the marginal likelihood of all the data is simply a product of these local normalization terms. The product is taken across variables and across different parent configurations:

$$P(D|G) = \prod_{i=1}^{n} \prod_{j=1}^{q_i} \frac{\Gamma(\sum_k \alpha_{ijk})}{\Gamma(\sum_k \alpha_{ijk} + \sum_k n_{ijk})} \prod_{k=1}^{r_i} \frac{\Gamma(\alpha_{ijk} + n_{ijk})}{\Gamma(\alpha_{ijk})} \tag{21}$$

We would now find a graph G that maximizes $P(D|G)$. Note that Eq.(21) is easy to evaluate for any particular graph by recomputing some of the counts n_{ijk}. We can further penalize graphs that involve a large number of parameters (or edges) by assigning a prior probability $P(G)$ over the graphs, and maximizing instead $P(D|G)P(G)$. For example, the prior could be some function of the number of parameters in the model or $\sum_{i=1}^{n}(r_i - 1)q_i$ such as

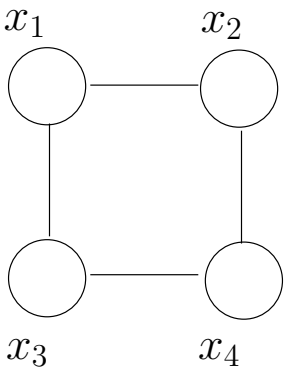
$$P(G) \propto \frac{1}{\sum_{i=1}^{n}(r_i - 1)q_i} \tag{22}$$

Lecture topics:

- Markov Random Fields
- Probabilistic inference

Markov Random Fields

We will briefly go over *undirected graphical models* or *Markov Random Fields* (MRFs) as they will be needed in the context of probabilistic inference discussed below (using the model to calculate various probabilities over the variables). The origin of these models is physics (e.g., spin glass) and they retain some of the terminology from the physics literature. The semantics of MRFs is similar but simpler than Bayesian networks. The graph again represents independence properties between the variables but the properties can be read off from the graph through simple graph separation rather than D-separation criterion. So, for example,



encodes two independence properties. First, x_1 is independent of x_4 given x_2 and x_3. In other words, if we remove x_2 and x_3 from the graph then x_1 and x_4 are no longer connected. The second property is that x_2 is independent of x_3 given x_1 and x_4. Incidentally, we couldn't define a Bayesian network over the same four variables that would explicate both of these properties (you can capture one while failing the other). So, in terms of their ability to explicate independence properties, MRFs and Bayesian networks are not strict subsets of each other.

By *Hammersley-Clifford theorem* we can specify the form that any joint distribution consistent with an undirected graph has to take. A distribution is consistent with the graph

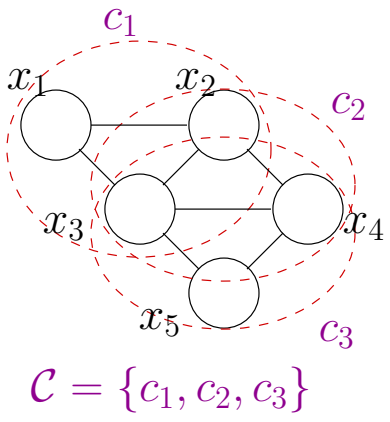
if it satisfies all the conditional independence properties we can read from the graph. The result is again in terms of how the distribution has to factor. For the above example, we can write the distribution as a product of (non-negative) *potential functions* over pairs of variables that specify how the variables depend on each other

$$P(x_1, x_2, x_3, x_4) = \frac{1}{Z}\psi_{12}(x_1, x_2)\psi_{13}(x_1, x_3)\psi_{24}(x_2, x_4)\psi_{34}(x_3, x_4) \tag{1}$$

where Z is a normalization constant (required since the potential functions can be any non-negative functions). The distribution is therefore *globally normalized*. More generally, an undirected graph places no constraints on how any fully connected subset of the variables, variables in a *clique*, depend on each other. In other words, we are free to associate any potential function with such variables. Without loss of generality we can restrict ourselves to *maximal cliques*, i.e., not consider separately cliques that are subsets of other cliques. In the above example, the maximal cliques where the pairs of connected variables. Now, in general, the Hammersley-Clifford theorem states that the joint distribution has to factor according to (maximal) cliques in the graph:

$$P(x_1, \ldots, x_n) = \frac{1}{Z}\prod_{c\in\mathcal{C}}\psi_c(x_c) \tag{2}$$

where $c \in \mathcal{C}$ is a (maximal) clique in the graph and $x_c = \{x_i\}_{i\in c}$ denotes the set of variables in the clique. The normalization constant Z could be easily absorbed into one of the potential functions but we will write it explicitly here as a reminder that the model has to be normalized globally (is not automatically normalized as Bayesian networks). Figure below provides an example of a graph with three maximal cliques.



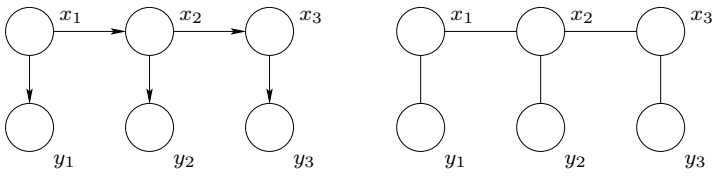
Bayesian networks as undirected models

We can always turn a Bayesian network into a MRF via *moralization*, i.e., connecting all the parents of a common child and dropping the directions on the edges. After the transformation we naturally still have the same probability distribution but may no longer capture all the independence properties explicitly in the graph. For example, in

$P(x_1, x_2, x_3) = P(x_1)P(x_2)P(x_3|x_1, x_2)$ $P(x_1, x_2, x_3) = \psi(x_1, x_2, x_3)$

where, clearly, $\psi(x_1, x_2, x_3) = P(x_1)P(x_2)P(x_3|x_1, x_2)$ so that the underlying distributions are the same (only the representation changed). The undirected graph is fully connected, however, and the marginal independence of x_1 and x_2 is no longer visible in the graph. In terms of probabilistic inference, i.e., calculating various probabilities, little is typically lost by turning a Bayesian network first into an undirected model. For example, we would often have some evidence pertaining to the variables, something would be known about x_3 and, as a result, x_1 and x_2 would become dependent. The advantage from the transformation is that the inference algorithms will run uniformly on both types of models.

Let's consider one more example of turning Bayesian networks into MRFs. The figure below gives a simple HMM with the associated probability model and the same for the undirected version after moralization.



$P(x_1)P(x_2|x_1)P(x_3|x_2)\times$
$\times P(y_1|x_1)P(y_2|x_2)P(y_3|x_3)$

$\frac{1}{Z}\psi_{12}(x_1, x_2)\psi_{23}(x_2, x_3)\times$
$\times\phi_1(x_1, y_1)\phi_2(x_2, y_2)\phi_3(x_3, y_3)$

Each conditional probability on the left can be assigned to any potential function that contains the same set of variables. For example, $P(x_1)$ could be included in $\psi_{12}(x_1, x_2)$ or in $\phi_1(x_1, y_1)$. The objective is merely to maintain the same distribution when we take the

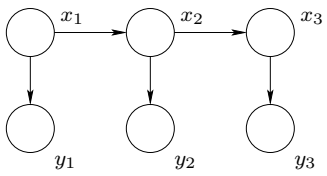
product (it doesn't matter how we reorder terms in a product). Here's a possible complete setting of the potential functions:

$$
\begin{aligned}
Z &= 1 \text{ (already normalized as we start with a Bayesian network)} && (3)\\
\psi_{12}(x_1, x_2) &= P(x_1)P(x_2|x_1) && (4)\\
\psi_{23}(x_2, x_3) &= P(x_3|x_2) && (5)\\
\phi_1(x_1, y_1) &= P(y_1|x_1) && (6)\\
\phi_1(x_2, y_2) &= P(y_2|x_2) && (7)\\
\phi_1(x_3, y_3) &= P(y_3|x_3) && (8)
\end{aligned}
$$

Probabilistic inference

Once we have the graph and the associated distribution (either learned from data or given to us), we would like to make use of this distribution. For example, in HMMs discussed above, we could try to compute $P(y_3|y_1, y_2)$, i.e., predict what we expect to see as the next observation having already seen y_1 and y_2. Note that the sequence of observations in an HMM does not satisfy the Markov property, i.e., y_3 is not independent of y_1 given y_1. This is easy to see from either Bayesian network or the undirected version via the separation criteria. You can also understand it by noting that y_1 may influence the state sequence, and therefore which value x_3 takes provided that y_2 does not fully constraint x_2 to take a specific value. We are also often interested in *diagnostic* probabilities such as $P(x_2|y_1, y_2, y_3)$, the posterior distribution over the states x_2 at $t = 2$ when we have already observed y_1, y_2, y_3. Or we may be interested in the most likely hidden state sequence and need to evaluate max-probabilities. All of these are probabilistic inference calculations.

If we can compute basic conditional probabilities such as $P(x_2|y_2)$, we can also evaluate various other quantities. For example, suppose we have an HMM



and we are interesting known which y's we should observe (query) so as to obtain the most information about x_2. To this end we have to define a value of new information. Consider,

for example, defining

$$\text{Value}(\{P(x_2|y_i)\}_{x_2=1,\ldots,m}) = -\text{Entropy}(x_2|y_i) \quad (9)$$

$$= \sum_{x_2} P(x_2|y_i) \log P(x_2|y_i) \quad (10)$$

In other words, if given a specific observation y_i, we can evaluate the value of the resulting conditional distribution $P(x_2|y_i)$ where y_i is known. There are many ways to define the value and, for simplicity, we defined it in terms of the uncertainty about x_2. The value is zero if we know x_2 perfectly and negative otherwise. Since we cannot know y_i prior to querying its value, we will have to evaluate its expected value assuming our HMM is correct: *the expected value of information* in response to querying a value for y_i is given by

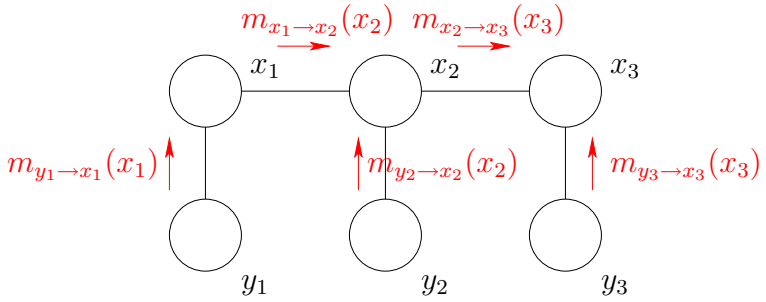
$$\sum_{y_i} P(y_i) \left[\sum_{x_2} P(x_2|y_i) \log P(x_2|y_i) \right] \quad (11)$$

where $P(y_i)$ is a marginal probability computed from the same HMM. We could now use the above criterion to find the observation most helpful in determining the value of x_2. Note that all the probabilities we needed were simple marginal and conditional probabilities. We still need to discuss how to evaluate such probabilities efficiently from a given distribution.

Belief propagation

Belief propagation is a simple message passing algorithm that generalizes the forward-backward algorithm. It is exact on undirected graphs that are trees (a graph is a tree if any pair of nodes have a unique path connecting them, i.e., has no loops). In case of more general graphs, we can cluster variables together so as to obtain a tree of clusters, and apply the same algorithm again, now on the level of clusters.

We will begin by demonstrating how messages are computed in the belief propagation algorithm. Consider the problem of evaluating the marginal probability of x_3 in an undirected HMM



$$\frac{1}{Z}\psi_{12}(x_1, x_2)\psi_{23}(x_2, x_3)\times$$
$$\times\phi_1(x_1, y_1)\phi_2(x_2, y_2)\phi_3(x_3, y_3)$$

We can perform the required marginalizations (summing over the other variables) in order: first y_1, then x_1, then y_2, and so on. Each of such operations will affect the variables they interact with. This effect is captured in terms of messages that are shown in red in the above figure. For example, $m_{y_1 \to x_1}(x_1)$ is a message, a function of x_1, that summarizes the effect of marginalizing over y_1. It is computed as

$$m_{y_1 \to x_1}(x_1) = \sum_{y_1} \phi_1(\mathbf{x}_1, y_1) \tag{12}$$

Note that in the absence of any evidence about y_1, $\phi(x_1, y_1) = P(y_1|x_1)$ and we would simply get a constant function of x_1 as the message. Suppose, instead, that we had already observed the value of y_1 and denote this value as $\hat{y}_1$. We can incorporate this observation (evidence about y_1) into the potential function $\phi_1(x_1, y_1)$ by redefining it as

$$\phi_1(x_1, y_1) = \delta(y_1, \hat{y}_1)P(y_1|x_1) \tag{13}$$

This way the message would be calculated as before but the value of the message as a function of x_1 would certainly change:

$$m_{y_1 \to x_1}(x_1) = \sum_{y_1} \phi_1(\mathbf{x}_1, y_1) = \sum_{y_1} \delta(y_1, \hat{y}_1)P(y_1|x_1) = P(\hat{y}_1|x_1) \tag{14}$$

After completing the marginalization over y_1, we turn to x_1. This marginalization results in a message $m_{x_1 \to x_2}(x_2)$ as x_2 relates to x_1. In calculating this message, we will have to take into account the message from y_1 so that

$$m_{x_1 \to x_2}(x_2) = \sum_{x_1} m_{y_1 \to x_1}(x_1)\psi_{12}(x_1, x_2) \tag{15}$$

More generally, in evaluating such messages, we incorporate (take a product of) all the incoming messages except the one coming from the variable we are marginalizing towards. Thus, x_2 will send the following message to x_3

$$m_{x_2 \to x_3}(x_3) = \sum_{x_2} m_{x_1 \to x_2}(x_2) m_{y_2 \to x_2}(x_2) \psi_{23}(x_2, x_3) \tag{16}$$

Finally, the probability of x_3 is obtained by collecting all the messages into x_3

$$P(x_3, D) = m_{x_2 \to x_3}(x_3) m_{y_3 \to x_3}(x_3) \tag{17}$$

where D refers to any data or observations incorporated into the potential functions (as illustrated above). The distribution we were after is then

$$P(x_3|D) = \frac{P(x_3, D)}{\sum_{x'_3} P(x'_3, D)} \tag{18}$$

It might seem that to evaluate similar distributions for all the other variables we would have to perform the message passing operations (marginalizations) in a particular order in each case. This is not necessary, actually. We can simply initialize all the messages to all one functions, and carry out the message passing operations asynchronously. For example, we can pick x_2 and calculate its message to x_1 based on the other available incoming messages (that may be wrong at this time). The messages will converge to the correct ones provided that the undirected model is a tree, and we repeatedly update each message (i.e., send messages from each variable in all directions). The asynchronous message updates will propagate the necessary information across the graph. This exchange of information between the variables is a bit more efficient to carry out synchronously, however. In other words, we designate a root variable and imagine the edges oriented outwards from this variable (this orientation has nothing to do with Bayesian networks). Then "collect" all the messages toward the root, starting from the leaves. Once the root has all its incoming messages, the direction is switched and we send (or "distribute") the messages outwards from the root, starting with the root. These two passes suffice to get the correct incoming messages for all the variables.